

INFORMATIQUE • 3ÈME

INFORMATIQUE – CLASSE DE 3ÈME

LEÇON N16

INTRODUCTION AUX
APPROCHES DE
DÉVELOPPEMENT LOGICIEL

DURÉE CONSEILLÉE : 1 SÉANCE (FICHE DE PROGRESSION HARMONISÉE)

DANS CETTE LEÇON

Cette leçon introduit les notions de base sur le cycle de développement des logiciels : les étapes de vie d'un logiciel, de l'analyse du besoin à la maintenance, et les principales approches de développement (cycle en cascade, cycle en V, approches itératives et agiles). L'élève apprend à appliquer ces notions à des situations concrètes de la vie courante et à rédiger et justifier une démarche.

- À la fin de cette leçon, je sais définir ce qu'est un logiciel et un cycle de développement logiciel.
- À la fin de cette leçon, je sais citer et ordonner les étapes du cycle de vie d'un logiciel.
- À la fin de cette leçon, je sais décrire le rôle de chaque étape (analyse, conception, programmation, tests, déploiement, maintenance).
- À la fin de cette leçon, je sais distinguer les principales approches de développement : cascade, cycle en V, itératif/incrémental et agile.
- À la fin de cette leçon, je sais comparer les avantages et limites de ces approches dans un tableau.
- À la fin de cette leçon, je sais choisir et justifier une approche adaptée à un projet concret de la vie courante.

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|---|---|
| 1. Logiciel et cycle de développement : premières définitions | 6. Choisir et justifier une démarche de développement |
| 2. Les étapes du cycle de vie d'un logiciel | 7. Activité d'intégration |
| 3. L'approche en cascade (cycle en cascade) | 8. Méthodes et exercices résolus |
| 4. Le cycle en V | 9. Exercices |
| 5. Approches itératives et agiles | 10. Corrigés des exercices |
| | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir ce qu'est un logiciel et un cycle de développement logiciel.
- À la fin de cette leçon, je sais citer et ordonner les étapes du cycle de vie d'un logiciel.
- À la fin de cette leçon, je sais décrire le rôle de chaque étape (analyse, conception, programmation, tests, déploiement, maintenance).
- À la fin de cette leçon, je sais distinguer les principales approches de développement : cascade, cycle en V, itératif/incrémental et agile.
- À la fin de cette leçon, je sais comparer les avantages et limites de ces approches dans un tableau.
- À la fin de cette leçon, je sais choisir et justifier une approche adaptée à un projet concret de la vie courante.
- À la fin de cette leçon, je sais rédiger et justifier une démarche de développement pour un mini-projet simple.
- À la fin de cette leçon, je sais identifier les acteurs d'un projet logiciel (client, analyste, développeur, testeur, utilisateur).

PRÉREQUIS

- Notion d'algorithme et de programme (classes de 4ème et début de 3ème)
- Notion de logiciel et distinction logiciel système / logiciel d'application

- Utilisation élémentaire d'un ordinateur ou d'un smartphone (applications usuelles)
- Notion de problème et de résolution de problème étape par étape
- Lecture et interprétation d'un schéma simple (organigramme, diagramme)

1 LOGICIEL ET CYCLE DE DÉVELOPPEMENT : PREMIÈRES DÉFINITIONS

Imagine Amina, élève de 3ème dans un collège de Yaoundé. Sa mère tient un petit restaurant très fréquenté au quartier Mvog-Ada. Chaque midi, c'est la course : les clients commandent à l'oral, la mère note sur un carnet qui s'effiloche, les additions se font à la main, et il arrive qu'on oublie un plat ou qu'on se trompe sur le total. Amina, qui suit les cours d'informatique, se dit : « Et si je faisais une petite application sur téléphone pour l'aider ? » Elle en parle à son grand frère, étudiant en génie logiciel. Il sourit et lui répond : « Bonne idée, ma sœur ! Mais attention, on ne fabrique pas un logiciel comme on prépare un ndolé : on ne jette pas tout dans la marmite n'importe comment. Il y a une recette précise, des étapes à respecter, des gens à impliquer. Sinon, ton application plantera au premier rush du midi. » Il lui montre alors que les informaticiens suivent des **méthodes organisées** — cycle en cascade, cycle en V, méthodes agiles — pour transformer une idée en un outil fiable, utilisable et durable.

Problématique : Comment passe-t-on d'un besoin réel (gérer les commandes d'un restaurant) à un logiciel fonctionnel, testé et maintenable ? Quelles sont les étapes, les acteurs et les règles du jeu ? C'est toute la question de cette leçon.

1.1 Qu'est-ce qu'un logiciel ?

Avant de parler de cycle, posons les bases. Tu utilises des logiciels tous les jours, souvent sans t'en rendre compte.

DÉFINITION Logiciel

Un logiciel est un ensemble de programmes (instructions exécutables par la machine), de données (informations manipulées) et de documentation (notices, manuels, commentaires) qui permet de réaliser une ou plusieurs tâches précises sur un ordinateur, un smartphone ou tout système numérique.

À RETENIR

Un logiciel n'est pas un simple programme. C'est un produit complet : code source + données + documentation + procédures d'installation et d'utilisation.

EXEMPLE Logiciels du quotidien au Cameroun

- Applications Mobile Money (MTN MoMo, Orange Money) : transfert d'argent, paiement de factures, achat de crédit. Le programme gère l'interface et la sécurité ; les données sont tes contacts, ton solde, l'historique des transactions.

- Logiciel de gestion scolaire : gestion des notes, bulletins, emplois du temps, inscriptions. Les données : élèves, professeurs, matières, notes, absences.
- Applications de transport (Yango, applications locales de moto-taxi) : mise en relation client/chauffeur, géolocalisation, calcul du tarif. Données : courses en cours, positions GPS, historique.
- Traitement de texte (LibreOffice Writer, MS Word) : rédaction, mise en forme, impression. Données : le texte, les images, les styles du document.

ATTENTION Erreur fréquente : « Logiciel = Code »

Beaucoup d'élèves pensent que développer un logiciel, c'est seulement écrire des lignes de code. Faux. Le code n'est que la partie visible de l'iceberg. Sans analyse du besoin, sans conception, sans tests, sans documentation et sans maintenance, le code ne sert à rien ou devient ingérable. Un logiciel professionnel passe plus de temps en maintenance qu'en écriture initiale du code.

1.2 Le développement logiciel et son cycle de vie

DÉFINITION Développement logiciel

Le développement logiciel est l'ensemble des activités (techniques, organisationnelles, humaines) qui permettent de passer d'un besoin exprimé par un client ou un utilisateur à un logiciel fonctionnel, livré, utilisé et maintenu.

DÉFINITION Cycle de développement (ou cycle de vie) d'un logiciel

Le cycle de développement (ou cycle de vie logiciel) est la succession ordonnée d'étapes qui jalonnent l'existence d'un logiciel, depuis la naissance de l'idée (expression du besoin) jusqu'à son retrait (désinstallation, archivage, remplacement).

Pourquoi ne pas coder directement ? Parce que sans méthode, on obtient :

- un logiciel qui **ne répond pas au besoin** (fonctionnalités manquantes ou inutiles) ;
- un logiciel **bourré de bugs** (erreurs non détectées) ;
- un **dépassement de budget** et de **délais** (on refait trois fois la même chose) ;
- un code **inmaintenable** (personne ne comprend comment ça marche, pas même l'auteur six mois plus tard) ;

- une **mauvaise collaboration** en équipe (qui fait quoi ? qui a modifié quoi ?).

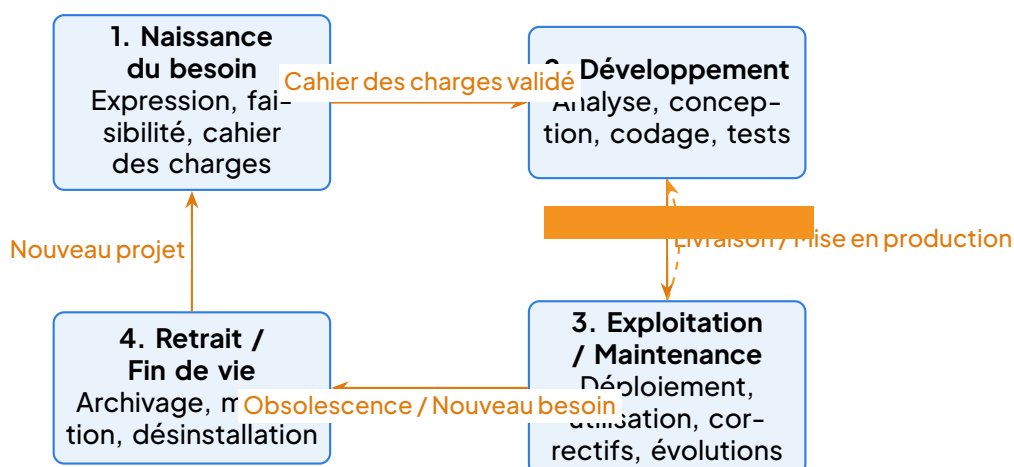
PROPRIÉTÉ Intérêts d'une méthode de développement structurée

Suivre un cycle de vie formel garantit :

1. La qualité : le logiciel fait ce qu'on attend, de façon fiable et sécurisée.
2. La maîtrise des coûts et des délais : on détecte les erreurs tôt (quand elles coûtent peu) plutôt qu'après la livraison (quand elles coûtent très cher).
3. Le travail en équipe : rôles clairs, livrables attendus à chaque étape, communication facilitée.
4. La traçabilité : on sait pourquoi une décision a été prise, qui l'a validée, quand.

1.3 Les grandes phases du cycle de vie

Quel que soit le modèle choisi (cascade, V, agile), tout logiciel traverse quatre **macro-phases** fondamentales. Le schéma ci-dessous les présente sous forme cyclique car la fin d'un cycle (retrait) prépare souvent le suivant (nouveau besoin).



Les quatre macro-phases du cycle de vie d'un logiciel (vue cyclique). La flèche orange en pointillés rappelle qu'il y a des allers-retours permanents (évolutions demandées pendant l'exploitation).

MÉTHODE Repères pour identifier la phase d'un projet

1. Phase 1 — Besoin : on parle métier, pas technique. Questions : « Quel problème résout-on ? Pour qui ? Avec quel budget ? Quelle date limite ? » → Livrable : cahier des charges.
2. Phase 2 — Développement : on passe au technique : analyse (modélisation), concep-

tion (architecture, choix techniques), codage, tests. → Livrable : code source + jeux de tests + documentation technique.

3. Phase 3 — Exploitation : installation chez le client, formation des utilisateurs, support, corrections de bugs (maintenance corrective), améliorations (maintenance évolutive). → Livrable : logiciel en production + rapports d'incidents + versions successives.
4. Phase 4 — Retrait : le logiciel ne sert plus (métier changé, technologie obsolète). On archive les données, on migre vers le remplaçant. → Livrable : plan de migration + archives.

EXEMPLE Le projet d'Amina : où en est-on ?

Amina a identifié le besoin : « Maman perd du temps et fait des erreurs sur les additions. » Elle a discuté avec sa mère (le client) : « Il me faut : liste des plats et prix, ajout d'une commande par table, calcul automatique du total, impression du ticket, historique journalier. » C'est la Phase 1. Elle n'a pas encore ouvert son éditeur de code ! Si elle codait tout de suite, elle risquerait d'oublier la gestion des boissons, ou de faire une interface trop complexe pour sa mère qui n'est pas à l'aise avec le tactile.

1.4 Les acteurs d'un projet logiciel

Un logiciel ne se fait pas tout seul. C'est un **travail d'équipe** où chaque rôle a sa responsabilité. Même pour un petit projet comme celui d'Amina, elle devra « jouer » plusieurs rôles tour à tour.

DÉFINITION Rôles clés (à retenir pour le BEPC)

- Client / Commanditaire : celui qui a le besoin, qui finance et qui valide le résultat final.
- Chef de projet : responsable de la tenue des délais, du budget, de la coordination de l'équipe et de la communication avec le client.
- Analyste / Concepteur : traduit le besoin métier en spécifications techniques (modèles de données, algorithmes, maquettes d'écrans).
- Développeur (Programmeur) : écrit le code source dans le langage choisi et réalise les tests unitaires.
- Testeur : vérifie que le logiciel correspond aux spécifications (tests d'intégration, recette).
- Utilisateur final : celui qui utilisera le logiciel au quotidien. Son retour est crucial pour la maintenance évolutive.

EXEMPLE Répartition typique du temps sur un projet (ordres de grandeur)

Phase	Activités principales	Temps total
Analyse et conception	Besoins, modélisation, maquettes	environ 20 %
Programmation (codage)	Écriture du code, tests unitaires	environ 30 %
Tests et recette	Intégration, tests système, recette client	environ 30 %
Mise en route et maintenance initiale	Installation, formation, corrections	environ 20 %

Leçon : le codage ne représente qu'environ un tiers du temps total. Négliger l'analyse ou les tests, c'est garantir l'échec ou l'explosion des coûts plus tard.

LESAVAIS-TU ? Le coût réel d'un logiciel : la loi de l'iceberg

Les études menées sur les logiciels d'entreprise montrent que, sur la durée de vie totale d'un logiciel (souvent 10 à 15 ans), la maintenance représente 60 % à 80 % du coût global, contre seulement 20 % à 40 % pour le développement initial. On distingue :

- la maintenance corrective : corriger les bugs découverts après livraison ;
- la maintenance adaptative : adapter le logiciel à un nouvel environnement (nouveau système d'exploitation, nouvelle règle fiscale) ;
- la maintenance évolutive : ajouter des fonctionnalités demandées par les utilisateurs (« Maman veut maintenant gérer les livraisons à domicile ») ;
- la maintenance préventive : améliorer le code pour éviter qu'il ne devienne ingérable.

Moralité : écrire du code propre, documenté et testé dès le début, ce n'est pas une perte de temps, c'est un investissement.

2 LES ÉTAPES DU CYCLE DE VIE D'UN LOGICIEL

Zoomons maintenant sur la **Phase 2 (Développement)**, celle qui transforme le cahier des charges en logiciel. Elle se décompose elle-même en étapes précises, que l'on retrouve dans tous les modèles de développement.

2.1 Les étapes détaillées

DÉFINITION Les étapes classiques du développement

1. Étude de faisabilité : peut-on réaliser le projet ? Avec quels moyens (humains, matériels, financiers) ? En combien de temps ?
2. Analyse des besoins : recueillir et rédiger de façon précise ce que le logiciel doit faire. Livrable : le cahier des charges.
3. Conception : décrire comment le logiciel va réaliser ces fonctions : architecture générale, modèle des données, maquettes des écrans, algorithmes principaux.
4. Codage (implémentation) : traduction de la conception en programmes, dans un langage de programmation.
5. Tests : vérification que chaque morceau fonctionne (tests unitaires), que les morceaux fonctionnent ensemble (tests d'intégration) et que le tout satisfait le client (re-cette).
6. Déploiement : installation du logiciel chez le client, formation des utilisateurs.
7. Maintenance : corrections et évolutions pendant toute la durée d'utilisation.

ATTENTION Ne pas confondre analyse et conception

L'analyse répond à la question « QUOI faire ? » (les besoins, les fonctions attendues). La conception répond à la question « COMMENT le faire ? » (choix techniques, organisation du code, structure des données). Confondre les deux, c'est comme commencer à acheter des briques avant de savoir si l'on construit une maison ou un hangar.

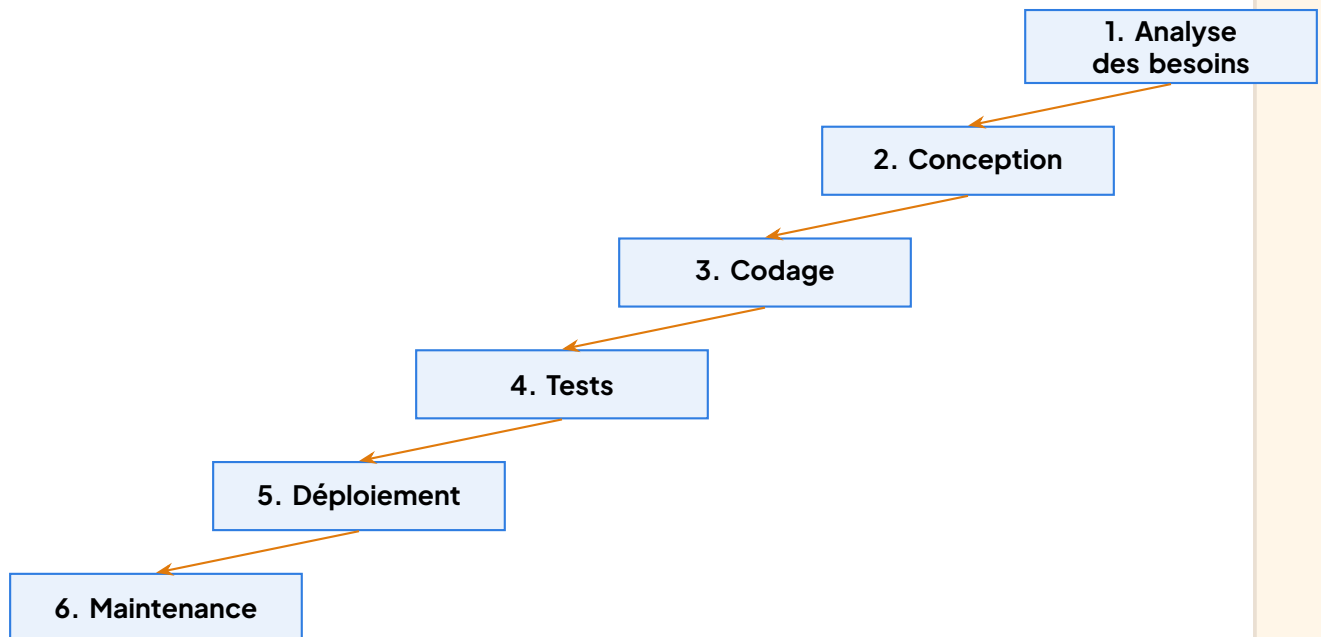
EXEMPLE Application au restaurant de la mère d'Amina

Étape	Ce que fait Amina
Faisabilité	« J'ai un vieux smartphone Android, 2 semaines de vacances, pas de budget : c'est faisable. »
Analyse	Liste des fonctions : carte des plats, commande par table, total automatique, ticket, recette du jour.
Conception	Maquettes des 3 écrans sur papier ; choix : application web simple (HTML + JavaScript) ; données stockées dans le navigateur.
Codage	Écriture des pages HTML et du code JavaScript de calcul.
Tests	Elle simule un service complet : 10 commandes, avec erreurs volontaires (plat inexistant, table vide).
Déploiement	Installation sur le téléphone de sa mère, 30 minutes de formation.
Maintenance	Semaine suivante : ajout de la gestion des boissons, correction d'un bug d'arrondi.

3 L'APPROCHE EN CASCADE (CYCLE EN CASCADE)

3.1 Principe

Le **modèle en cascade** (en anglais waterfall) est le plus ancien et le plus simple des modèles de développement. Son idée : **chaque étape doit être totalement terminée et validée avant de passer à la suivante**, comme l'eau d'une cascade qui descend de marche en marche sans jamais remonter.



Le cycle en cascade : les étapes s'enchaînent de haut en bas, sans retour en arrière prévu.

PROPRIÉTÉ Avantages et limites du cycle en cascade

Avantages :

- simple à comprendre et à gérer (étapes claires, livrables datés) ;
- adapté aux projets dont les besoins sont stables et bien connus dès le départ.

Limites :

- pas de retour en arrière prévu : si l'on découvre une erreur d'analyse pendant les tests, il est très coûteux de la corriger ;
- le client ne voit le logiciel qu'à la fin : risque de « surprise » (« ce n'est pas ce que je voulais ! ») ;
- mal adapté aux projets dont les besoins évoluent.

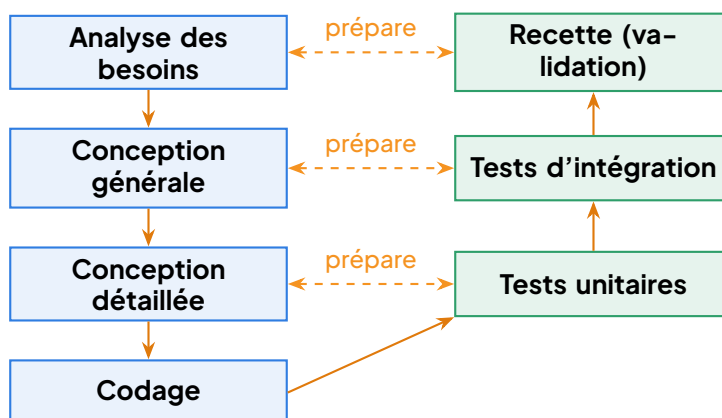
EXEMPLE Quand la cascade convient

Le proviseur d'un collège commande un logiciel de calcul des moyennes et d'impression des bulletins. Les règles de calcul (coefficients, moyennes, rangs) sont fixées par les textes officiels et ne changeront pas. Les besoins sont parfaitement connus et stables : le cycle en cascade est un bon choix.

4 LE CYCLE EN V

4.1 Principe

Le cycle en V est une amélioration du cycle en cascade. L'idée centrale : **à chaque étape de construction (descente) correspond une étape de vérification (remontée)**. Les tests ne sont plus une surprise de fin de projet : ils sont préparés dès les premières étapes.



Le cycle en V : la branche gauche construit (besoins → conception → codage), la branche droite vérifie (tests unitaires → intégration → recette). Chaque test est préparé par l'étape qui lui fait face.

DÉFINITION Les trois niveaux de tests du cycle en V

- Tests unitaires : on vérifie chaque petit morceau du programme séparément (par exemple la fonction qui calcule le total d'une commande). Ils sont préparés pendant la conception détaillée.
- Tests d'intégration : on vérifie que les morceaux fonctionnent bien ensemble (la commande s'enregistre et le ticket s'imprime correctement). Ils sont préparés pendant la conception générale.
- Recette (tests de validation) : le client vérifie que le logiciel répond bien au cahier des charges initial. C'est lui qui « réceptionne » le produit.

PROPRIÉTÉ Avantages et limites du cycle en V

Avantages : qualité renforcée (tests planifiés dès le début, à tous les niveaux) ; traçabilité entre besoins et tests. Limites : comme la cascade, il reste rigide : le client ne découvre le

produit qu'à la recette, et un changement de besoin en cours de route reste coûteux.

5 APPROCHES ITÉRATIVES ET AGILES

5.1 Le développement itératif

Pour corriger la rigidité des modèles précédents, l'**approche itérative** propose de construire le logiciel **par petits morceaux successifs** appelés **itérations**. Chaque itération est un mini-cycle complet (analyse, conception, codage, tests) qui produit une **version utilisable** du logiciel, enrichie à chaque tour.

EXEMPLE Le projet d'Amina en itératif

- Version 1 (semaine 1) : saisie d'une commande et calcul automatique du total. Sa mère l'utilise déjà !
- Version 2 (semaine 2) : ajout de l'impression du ticket et de la gestion des tables.
- Version 3 (semaine 3) : ajout de la recette journalière et de l'historique.

À chaque version, sa mère donne son avis, et Amina ajuste la suite. Si un problème apparaît, il ne concerne qu'un petit morceau, facile à corriger.

5.2 Les méthodes agiles

Les **méthodes agiles** poussent cette logique encore plus loin. Leurs principes essentiels, à retenir :

À RETENIR Principes des méthodes agiles

- Le client au centre : il participe tout au long du projet, pas seulement au début et à la fin.
- Livrer tôt, livrer souvent : des versions courtes et fonctionnelles (toutes les 1 à 4 semaines), plutôt qu'un gros livrable final.
- Accueillir le changement : si le besoin évolue, on adapte le projet au lieu de figer le cahier des charges.
- Communication directe : petites équipes, échanges quotidiens, moins de documents lourds, plus de dialogue.

PROPRIÉTÉ Comparaison des trois approches

Critère	Cascade	Cycle en V	Itératif / Agile
Enchaînement	Linéaire, sans retour	Linéaire + tests à chaque niveau	Cycles courts répétés
Place du client	Début et fin	Début et recette	Pendant tout le projet
Besoins changeants	Très mal gérés	Mal gérés	Bien gérés
Livraison	Une seule, à la fin	Une seule, à la fin	Versions successives
Projet adapté	Besoins stables et connus	Projets exigeant une forte qualité	Besoins évolutifs, équipes agiles

6 CHOISIR ET JUSTIFIER UNE DÉMARCHE DE DÉVELOPPEMENT

Il n'existe pas de « meilleure » méthode dans l'absolu : il faut **choisir en fonction du projet** et savoir **justifier** ce choix — c'est exactement le savoir-faire attendu au BEPC.

MÉTHODE Choisir une approche de développement

1. Les besoins sont-ils clairs et stables ? Si oui (règles officielles, métier bien connu) → cascade ou cycle en V.
2. La qualité et la fiabilité sont-elles critiques ? (logiciel de paie, notes d'examen) → cycle en V, car les tests y sont systématiques et tracés.
3. Les besoins risquent-ils d'évoluer ? Le client n'est-il pas sûr de ce qu'il veut ? → approche itérative ou agile.
4. Le client est-il disponible ? Une méthode agile exige sa présence régulière ; s'il est injoignable, préférer un cahier des charges détaillé et un cycle en V.
5. Rédiger la justification : « J'ai choisi telle approche parce que caractéristique du projet, ce qui garantit avantage attendu. »

EXERCICE RÉSOLU 1**Analyse d'une situation : le cybercafé de M. Kouam**

Énoncé : M. Kouam gère un cybercafé à Douala. Il note l'heure d'arrivée et de départ de chaque client sur un cahier et calcule le prix à la main (50 FCFA les 15 minutes). Il demande à son neveu, élève en 3ème, de lui faire un logiciel « vite fait ce week-end ». Le neveu code une page en 3 heures sans rien noter. Résultat : le logiciel perd les sessions si l'on rafraîchit la page, ne gère pas les pauses, et M. Kouam ne sait pas l'installer sur ses 10 postes.

Questions :

1. Identifie les acteurs manquants ou mal joués dans ce projet.
2. Quelles étapes du cycle de vie ont été sautées ?
3. Propose une démarche minimaliste mais structurée pour refaire ce logiciel correctement, et justifie le choix de l'approche.

SOLUTION

Solution détaillée :

1. Acteurs : le neveu a joué seul tous les rôles : client (il a deviné les besoins), chef de projet (sans planning), analyste (sans spécifications), développeur (code rapide), testeur (sans tests). Manquent : la validation par le vrai client (M. Kouam), les tests réels, la documentation et la formation de l'utilisateur final.
2. Étapes sautées :
 - Analyse des besoins : pas de cahier des charges (gestion des pauses, multi-postes, sauvegarde des sessions, ticket).
 - Conception : pas de réflexion sur les données à stocker ni sur le choix technique (comment conserver les sessions?).
 - Tests : aucun test (rafraîchissement de la page, coupure de courant, plusieurs postes).
 - Déploiement : pas d'installation sur les 10 postes, pas de formation, pas de mode d'emploi.
3. Démarche minimaliste structurée (1 semaine) :
 - (a) Lundi : entretien avec M. Kouam → liste des besoins priorités (sessions sauvegardées, calcul automatique, gestion des pauses, ticket).
 - (b) Mardi : conception simple : description des données (numéro de poste, heure de début, heure de fin, pauses, montant) et maquettes des écrans sur papier.
 - (c) Mercredi-jeudi : codage, avec sauvegarde des données pour survivre au rafraîchissement ; tests unitaires (calcul du prix, pause, reprise).
 - (d) Vendredi : tests sur 2 postes, recette avec M. Kouam sur des scénarios réels, installation, rédaction d'un mode d'emploi d'une page.
 - (e) Semaine suivante : support et corrections des bugs remontés.

Justification de l'approche : j'ai choisi une approche itérative simplifiée parce que M. Kouam ne sait pas exprimer précisément ses besoins à l'avance ; livrer une première version simple (chronométrage + calcul du prix) puis l'enrichir avec ses retours garantit un logiciel vraiment adapté à son usage.

EXPÉRIENCE**Mini-atelier : « De l'idée au cahier des charges » (15 min)**

Objectif : s'exercer à l'étape d'expression du besoin pour un micro-projet réaliste.

Matériel : papier, stylo, ou traitement de texte.

Consigne : en binôme. L'un joue le client (choisis un métier : coiffeur, vendeuse de beignets, bibliothécaire, gérant de pressing), l'autre l'analyste.

1. Client : décris ton activité et 3 problèmes concrets que tu veux résoudre avec un logiciel (ex. : « J'oublie qui a payé », « Je ne sais plus quel stock il me reste »).
2. Analyste : pose des questions pour préciser : « Combien de clients par jour ? », « Quelles informations sur le ticket ? », « As-tu un PC, un smartphone, Internet ? », « Quel budget, quel délai ? ».
3. Ensemble : rédigez un mini cahier des charges (une demi-page) : contexte, objectifs, fonctionnalités principales (5 maximum), contraintes, utilisateurs.
4. Partage : deux binômes lisent leur cahier des charges à la classe. Discussion : est-ce clair ? complet ? réaliste ? Quelle approche (cascade, V, itératif) choisiriez-vous, et pourquoi ?

Observation attendue : les élèves constatent que « faire une application » commence par écouter, questionner, écrire, bien avant la première ligne de code.

EXERCICE 1**Entraînement BEPC**

1. Complète : un logiciel est un ensemble de _____, de _____ et de _____.
2. Vrai ou Faux ? Justifie en une phrase.
 - a) Le cycle de vie d'un logiciel s'arrête à la livraison chez le client.
 - b) Le développeur est le seul acteur d'un projet logiciel.
 - c) Dans le cycle en V, les tests sont préparés dès les étapes de conception.
3. Mets dans l'ordre chronologique : (1) Codage, (2) Expression du besoin, (3) Maintenance corrective, (4) Tests d'intégration, (5) Conception, (6) Déploiement.
4. Situation : une commune veut un logiciel de suivi des naissances. Les règles sont fixées par la loi et ne changeront pas ; la fiabilité est critique. Quelle approche conseilles-tu ? Justifie en deux phrases.

5. Situation : Léa code une application de gestion de stock pour la boutique de son oncle. Elle la livre sans mode d'emploi ni formation. Deux semaines plus tard, l'oncle appelle : « Ça ne marche plus, j'ai effacé un truc. » Quelle phase du cycle a été négligée ? Quel acteur n'a pas fait son travail ?

CORRIGÉ Exercice Entraînement BEPC

1. programmes, données, documentation.
2. a) Faux. Le cycle continue par l'exploitation (utilisation, maintenance) et se termine par le retrait. b) Faux. Un projet réunit au minimum : client, chef de projet, analyste, développeur, testeur et utilisateur final. c) Vrai. Dans le cycle en V, chaque étape de la branche descendante prépare le niveau de tests correspondant dans la branche montante.
3. Ordre : (2) Expression du besoin → (5) Conception → (1) Codage → (4) Tests d'intégration → (6) Déploiement → (3) Maintenance corrective.
4. Cycle en V. Les besoins sont stables et parfaitement connus (fixés par la loi), donc un modèle linéaire convient. La fiabilité critique exige des tests systématiques et tracés à chaque niveau, ce qui est précisément la force du cycle en V.
5. Phase négligée : la phase d'exploitation, plus précisément le déploiement accompagné (installation, formation, documentation utilisateur). Acteurs défaillants : le chef de projet (n'a pas planifié la formation ni la livraison de la documentation) et le développeur (n'a pas fourni de mode d'emploi). L'utilisateur final n'a pas été mis en condition de réussir.

7 LES ÉTAPES DU CYCLE DE VIE D'UN LOGICIEL

Dans la section précédente, nous avons défini ce qu'est un logiciel et introduit l'idée qu'il ne naît pas par hasard : il suit un processus structuré, le **cycle de vie**. Avant d'étudier les différentes manières d'organiser ce processus (cascade, V, agile), il est indispensable de maîtriser le contenu de chaque étape. Imagine que tu doives construire une maison : tu ne commences pas par poser les briques (programmation) sans avoir dessiné les plans (conception) ni même sans savoir combien de chambres le client désire (analyse des besoins). Le développement logiciel obéit à la même logique rigoureuse.

7.1 Étape 1 — Analyse des besoins : comprendre le problème avant de le résoudre

C'est la fondation de tout le projet. Si cette étape est bâclée, le logiciel final, même techniquement parfait, ne servira à personne.

DÉFINITION Analyse des besoins

L'**analyse des besoins** est l'activité qui consiste à identifier, comprendre, structurer et documenter ce que le client (ou l'utilisateur final) attend du logiciel. Elle répond à la question fondamentale : « Quel problème le logiciel doit-il résoudre ? ».

MÉTHODE Comment mener une analyse des besoins efficace

Pour ne rien oublier, utilise la grille QQQQCP (Qui, Quoi, Où, Quand, Comment, Combien, Pourquoi) adaptée à l'informatique :

1. Les acteurs (Qui) : Qui va utiliser le logiciel? (Secrétaire, comptable, élève, client web). Quels sont leurs niveaux de compétence informatique?
2. Les fonctions (Quoi) : Quelles tâches le logiciel doit-il effectuer? (Saisir une note, calculer une moyenne, imprimer un bulletin, gérer un stock).
3. Les données (Quoi aussi) : Quelles informations le logiciel manipule-t-il? (Noms, notes, dates, montants). Quelles sont les règles de gestion? (Ex. : une note est entre 0 et 20).
4. Les contraintes (Comment/Combien) : Délais, budget, matériel disponible (vieux PC au cybercafé?), système d'exploitation cible (Windows, Linux, Android), langue (français, anglais), normes légales (protection des données au Cameroun).
5. L'environnement (Où/Quand) : Le logiciel tourne-t-il en local? En réseau local (intranet d'un lycée)? Sur le web? Doit-il fonctionner hors ligne?

PROPRIÉTÉ Le cahier des charges

Le livrable principal de cette étape est le **cahier des charges** (ou spécifications fonctionnelles). C'est un document contractuel (souvent signé par le client et l'équipe de développement) qui liste :

- Les besoins fonctionnels : ce que le logiciel doit faire (ex. : « Le système permet d'ajouter un élève »).
- Les besoins non fonctionnels : qualités attendues (ex. : « Le temps de réponse < 2 s », « Interface en français », « Accessible aux malvoyants », « Sauvegarde automatique chaque heure »).
- Les contraintes techniques et légales.

ATTENTION Piège fréquent : confondre besoin et solution

Le client dit souvent : « Je veux un bouton rouge en haut à droite pour exporter en PDF. »
Erreur : Noter « Bouton rouge PDF » dans le cahier des charges. Bonne pratique : Comprendre le vrai besoin : « L'utilisateur a besoin d'imprimer ou d'archiver la liste des élèves hors ligne. » La solution technique (bouton, menu, raccourci, PDF, Excel) sera décidée plus tard lors de la conception.

EXEMPLE Contexte camerounais : Gestion d'une bibliothèque scolaire

Le proviseur du Lycée Bilingue d'Etoug-Ebe veut informatiser la bibliothèque.

- Acteurs : Bibliothécaire (gestion), Élèves/Professeurs (emprunt/retour), Proviseur (statistiques).
- Fonctions clés : Enregistrer un livre, gérer les adhérents, prêter/retourner un livre, alerter les retards, éditer la liste des livres disponibles.
- Règles de gestion : Max 3 livres par élève, durée max 14 jours, pas de prêt si retard en cours.
- Contraintes : 5 postes Windows 10 en réseau local, pas d'internet, budget serré, formation du bibliothécaire incluse.

Le cahier des charges validera ces points avant d'écrire la première ligne de code.

7.2 Étape 2 — Conception : architecturer la solution

Une fois le **quoi** validé, on définit le **comment**. On ne code pas encore, on modélise.

DÉFINITION Conception

La **conception** (ou design) transforme les besoins en une architecture technique détaillée. Elle produit des modèles (schémas, diagrammes) qui guideront les programmeurs. On distingue deux niveaux :

- Conception générale (architecturale) : Choix de l'architecture (client-serveur, monolithique, web), des technologies (langage, SGBD, framework), découpage en modules.
- Conception détaillée : Pour chaque module : algorithmes précis, structure de la base de données (MCD/MLD), maquettes d'écrans (IHM), formats d'échanges (API, fichiers).

EXEMPLE Exemple de conception pour la bibliothèque

- Architecture : Application locale en C (niveau 3ème) ou Python, base de données SQ-Lite (fichier unique, pas de serveur à installer).
- Base de données (MCD simplifié) :

Entité	Attributs principaux
LIVRE	ISBN (PK), Titre, Auteur, Éditeur, Année, Disponible (booléen)
ADHERENT	NumCarte (PK), Nom, Prénom, Classe, Téléphone
EMPRUNT	# (PK), DateEmprunt, DateRetourPrévue, DateRetourRéelle, ISBN (FK), NumCarte (FK)

- Algorithme principal (Prêter un livre) :

```

Algorithme PreterLivre
Variables
    isbn, numCarte : Chaîne
    livre, adherent : Enregistrement
    nbEmpruntsEnCours : Entier
Début
    Afficher "Scanner ISBN : " ; Saisir isbn
    livre <- RechercherLivre(isbn)
    Si livre = NUL Alors
        Afficher "Livre inconnu"
    Sinon Si Non livre.Disponible Alors
        Afficher "Déjà emprunté"
    Sinon
        Afficher "Scanner carte adhérent : " ; Saisir numCarte
        adherent <- RechercherAdherent(numCarte)
        Si adherent = NUL Alors
            Afficher "Adhérent inconnu"
        Sinon
            nbEmpruntsEnCours <- CompterEmpruntsNonRendus(numCarte)
            Si nbEmpruntsEnCours >= 3 Alors
                Afficher "Limite 3 livres atteinte"
            Sinon Si AUnRetard(numCarte) Alors
                Afficher "Retard en cours, prêt bloqué"
            Sinon
                CreerEmprunt(isbn, numCarte, DateDuJour, DateDuJour+14)
                ModifierLivre(isbn, Disponible <- Faux)
                Afficher "Prêt enregistré. Retour prévu le ", DateDuJour+14
            FinSi
        FinSi
    FinSi
Fin
  
```

- Maquettes IHM : Écran d'accueil (menu), Écran « Nouveau prêt » (champs ISBN, Num-Carte, bouton Valider), Écran « Liste retards » (tableau trié par date).

7.3 Étape 3 — Programmation (implémentation) : donner vie au modèle

C'est l'écriture du code source dans le langage choisi (C, Python, HTML/JS, etc.). C'est l'étape la plus visible, mais elle ne doit être que la traduction fidèle de la conception.

PROPRIÉTÉ**Bonnes pratiques de programmation au niveau 3ème**

- Respecter la conception : Ne pas improviser une structure de table différente du MCD.
- Style et lisibilité : Indentation rigoureuse (4 espaces), noms de variables explicites (`dateRetourPrevue` et pas `drp`), commentaires pour les parties complexes.
- Modularité : Une fonction = une tâche précise (ex. `calculerMoyenne()`, `verifierDisponibilite()`).
- Gestion des erreurs : Vérifier les saisies utilisateur (entrée invalide, division par zéro, fichier introuvable).
- Contrôle de version : Même en solo, utiliser `git` (`init`, `add`, `commit`) pour garder l'historique.

ATTENTION**Erreur fréquente : coder sans avoir fini la conception**

« Je vais commencer à coder l'écran de login, on verra pour la base de données plus tard. »
→ Cela conduit à du spaghetti code, des retours en arrière coûteux et des incohérences. La conception doit être validée (revue de conception) avant le début du codage.

7.4 Étape 4 — Tests et validation : prouver que ça marche

Un logiciel non testé est un logiciel qui ne marche pas. Les tests ne servent pas à « voir si ça marche » mais à « trouver des erreurs ».

DÉFINITION**Niveaux de tests**

- Tests unitaires : On teste chaque fonction/module isolément. Ex. : `calculerMoyenne(10, 12, 14)` renvoie-t-il 12? Et avec une liste vide? Et avec une note > 20?
- Tests d'intégration (ou d'ensemble) : On teste l'enchaînement des modules. Ex. : Le prêt met-il bien à jour la table LIVRE et la table EMPRUNT dans la même transaction?
- Tests système : Le logiciel complet dans son environnement réel (réseau, imprimante, scanner code-barres).

- **Recette client (UAT - User Acceptance Testing)** : Le client (proviseur, bibliothécaire) teste lui-même ses scénarios quotidiens. C'est lui qui valide (signe la fiche de recette) ou refuse la livraison.

ATTENTION Distinction cruciale : Test vs Débogage

- **Tester** = Provoquer une erreur (exécuter le code avec des cas limites, des données invalides, charger le système). Objectif : révéler la présence d'un bug.
- **Déboguer (debugger)** = Localiser et corriger la cause de l'erreur trouvée. Objectif : supprimer le bug.

Ne dis jamais « j'ai testé et j'ai corrigé » comme si c'était une seule action. Ce sont deux phases distinctes : on teste d'abord (souvent par un autre développeur ou un script automatique), on débogue après.

EXERCICE RÉSOLU 2 Cas de test pour la fonction « Calculer moyenne »

Énoncé : La fonction `moyenne(notes[])` calcule la moyenne d'un tableau de notes (réels entre 0 et 20). Proposer un jeu de tests unitaires complet.

SOLUTION

Solution détaillée : On utilise la technique des partitions d'équivalence et des valeurs limites.

Cas de test	Données d'entrée	Résultat attendu	Objectif (partition)
TC-01	[10, 12, 14]	12	Cas nominal (moyenne simple)
TC-02	[0, 0, 0]	0	Borne inférieure (notes minimales)
TC-03	[20, 20, 20]	20	Borne supérieure (notes maximales)
TC-04	[10, 15]	12,5	Nombre pair d'éléments, moyenne non entière
TC-05	[] (vide)	Erreur / Exception / -1	Cas limite : tableau vide (division par 0)
TC-06	[-5, 10]	Erreur / Rejet	Donnée invalide : note < 0
TC-07	[10, 25]	Erreur / Rejet	Donnée invalide : note > 20
TC-08	[10, 10, 10, 10, 10, 10, 10, 10, 10, 10]	10	Tableau grand (performance, débordement?)

Remarque : Si le code plante au TC-05 (division par zéro), le test a réussi sa mission (trouver le bug). Le développeur corrige ensuite (débugage).

7.5 Étape 5 — Déploiement (mise en production) : livrer l'outil à l'utilisateur

Le code est testé, validé, signé. Il faut maintenant l'installer là où il sera utilisé.

DÉFINITION Déploiement

Le **déploiement** comprend toutes les opérations pour rendre le logiciel opérationnel chez le client :

- Installation sur les postes cibles (exécutable, scripts, raccourcis, base de données initiale).
- Configuration (chemins d'accès, paramètres réseau, imprimante par défaut).
- Migration des données anciennes (ex. : importer le fichier Excel des 500 adhérents existants dans la nouvelle base SQLite).
- Formation des utilisateurs (mode d'emploi, session pratique de 1h au lycée).
- Documentation : Guide utilisateur (PDF), guide administrateur (sauvegarde, restau-

ration), fiche de recette signée.

EXPÉRIENCE TP guidé : Déployer l'application « BiblioLycee » sur le poste du bibliothécaire

Objectif : Mettre en production l'application C/SQLite développée. Matériel : Poste Windows 10 (bibliothécaire), clé USB avec l'exécutable `BiblioLycee.exe`, la base vide `biblio.db`, le guide utilisateur PDF. Manipulation :

1. Créer le dossier `C:\BiblioLycee` sur le poste.
2. Copier `BiblioLycee.exe` et `biblio.db` dans ce dossier.
3. Clic droit sur `BiblioLycee.exe` → « Créer un raccourci » → déplacer le raccourci sur le Bureau.
4. Lancer l'application via le raccourci. Vérifier l'écran d'accueil.
5. Menu « Fichier > Importer adhérents » : sélectionner `adherents_2024.xlsx` (fourni par le proviseur). Vérifier que 487 lignes sont importées.
6. Faire un test complet : Prêt d'un livre à un élève existant → Vérifier que le livre passe en « Indisponible ».
7. Former le bibliothécaire : montrer comment faire une sauvegarde (copier `biblio.db` sur clé USB) et comment restaurer.
8. Faire signer la fiche de recette par le proviseur et le bibliothécaire.

Observation : L'importation a échoué la première fois car le fichier Excel avait des entêtes en ligne 3 au lieu de ligne 1. Correction du script d'importation (conception → programmation → test → redéploiement) puis succès. Interprétation : Le déploiement révèle souvent des problèmes d'environnement (chemins, droits admin, format de données) invisibles en développement. Prévoir une période de rodage (1 à 2 semaines) avec support développeur réactif.

7.6 Étape 6 — Maintenance : la vie continue après la livraison

Contrairement à une maison qu'on « finit », un logiciel vit, évolue, vieillit. La maintenance est l'activité la plus longue et la plus coûteuse.

DÉFINITION Types de maintenance

- Maintenance corrective : Corriger des bugs découverts après la livraison (ex. : l'alerte de retard ne s'affiche pas pour les prêts du 31 décembre).

- Maintenance évolutive : Ajouter de nouvelles fonctionnalités demandées par l'utilisateur (ex. : « On veut aussi gérer les réservations de livres » ou « Ajouter un export CSV pour le fournisseur »).
- Maintenance adaptative : Adapter le logiciel à un environnement qui change (ex. : migration Windows 10 → Windows 11, changement d'imprimante, nouvelle loi sur les données personnelles au Cameroun, passage à la fibre CAMTEL modifiant l'IP du serveur).
- Maintenance préventive : Améliorer la structure du code (refactoring), mettre à jour les bibliothèques, optimiser les requêtes SQL lentes, documenter le code pour les futurs développeurs, avant que ça ne casse.

LESAVAIS-TU ? Le coût caché de la maintenance

Des études classiques (Boehm, Lientz & Swanson) montrent que la maintenance représente 50% à 75% du coût total d'un logiciel sur toute sa durée de vie (souvent 10 à 15 ans). Le développement initial n'est que la pointe de l'iceberg. C'est pourquoi un code propre, documenté et testé (étapes 2, 3, 4) est un investissement rentable : il divise le coût de maintenance par 2 ou 3.

7.7 Le coût de l'erreur selon l'étape de détection : la loi de l'amplification

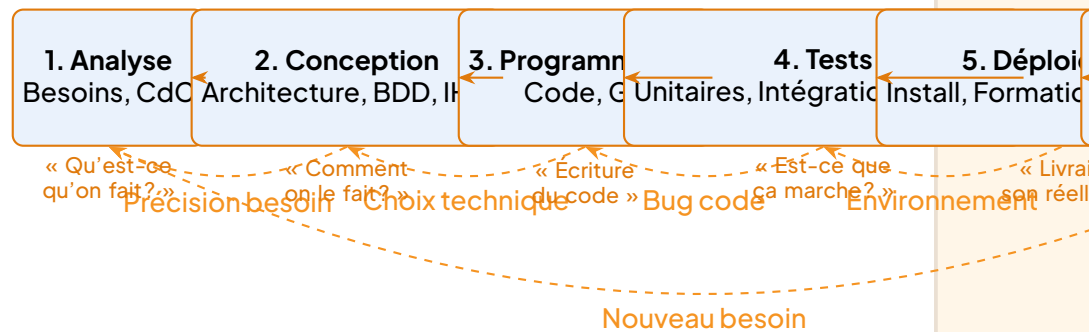
C'est la raison d'être des cycles structurés (Cascade, V) : **plus on détecte une erreur tard, plus elle coûte cher à corriger.**

PROPRIÉTÉ Règle du coût de correction (ordre de grandeur industriel)

Si corriger une erreur d'analyse (étape 1) pendant l'analyse coûte 1 unité (ex. : 1 heure de discussion), alors :

- La corriger pendant la conception (étape 2) coûte 3 à 5 unités (il faut modifier les diagrammes, la BDD).
- La corriger pendant la programmation (étape 3) coûte 10 unités (refaire le code, les tests unitaires).
- La corriger pendant les tests (étape 4) coûte 20 à 50 unités (re-test complet, régression, délai livraison).
- La corriger en production (étape 5/6) coûte 100 unités et plus (arrêt service, perte données, image client, urgence, hotfix).

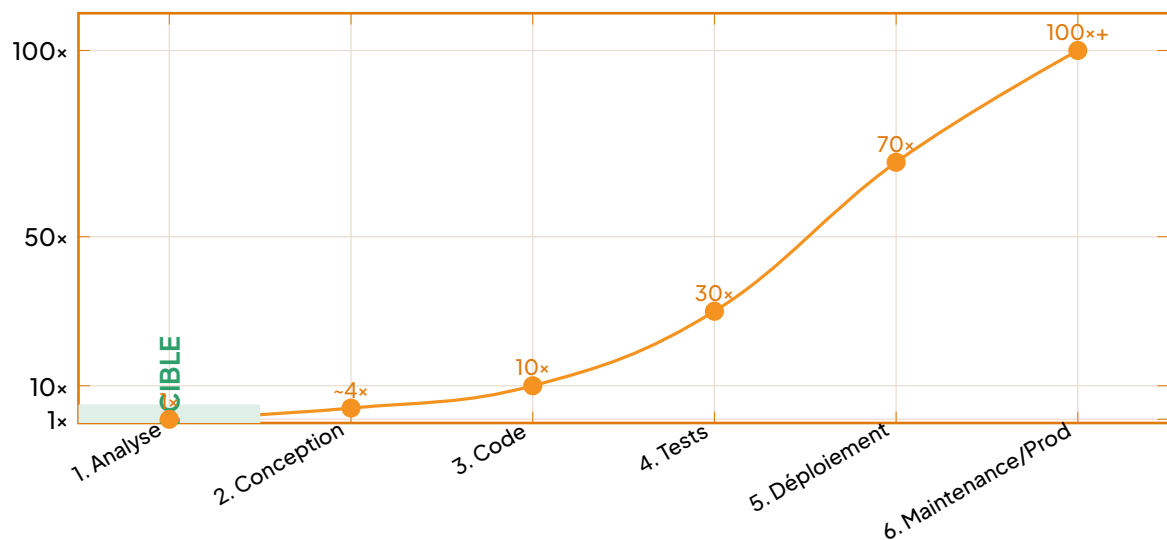
Le cycle de vie d'un logiciel : 6 étapes fondamentales



Frise chronologique des 6 étapes du cycle de vie. Les flèches pleines montrent l'avancement principal; les flèches orange pointillées rappellent qu'un retour en arrière est toujours possible (et fréquent), mais coûteux.

Coût relatif de correction (échelle logarithmique)

Coût exponentiel de la correction d'une erreur selon l'étape de détection



Étape de détection de l'erreur

Courbe qualitative du coût de correction. Une erreur d'analyse (ex. : « on a oublié de gérer les réservations ») corrigée à l'étape 1 coûte 1. La même erreur découverte en production (étape 6) coûte 100 fois plus cher : il faut modifier la BDD, le code, refaire tous les tests, redéployer, former à nouveau, gérer les données existantes.

À RETENIR Synthèse : Les 6 piliers du cycle de vie

1. Analyse des besoins → Cahier des charges (Quoi ? Pour qui ? Contraintes). **QQQQCP**.
2. Conception → Modèles (Architecture, MCD/MLD, Algorithmes, Maquettes IHM). Ne pas coder avant validation.
3. Programmation → Code source (Propre, modulaire, commenté, versionné Git).
4. Tests → Preuve de qualité (Unitaires, Intégration, Système, Recette Client). Test ≠ Dé-bogage.
5. Déploiement → Mise en production (Installation, Migration données, Formation, Do-cumentation, Signature recette).
6. Maintenance → Longévité (Corrective, Évolutive, Adaptative, Préventive). Repré-sente > 50% du coût total.

Règle d'or : « Une erreur détectée tôt est une erreur maîtrisée. » Chaque étape valide la précédente avant de passer à la suivante.

EXERCICE 2 Mise en situation : Le cybercafé « NetPlus » à Douala

Le gérant de « NetPlus » veut un logiciel pour gérer ses 20 postes : temps de connexion par client, facturation à la minute, vente de boissons/snacks, impression de reçus, sta-tistiques journalières.

1. Rédige une fiche d'analyse des besoins (acteurs, 5 fonctions principales, 3 règles de gestion, 3 contraintes techniques).
2. Propose une structure de base de données simplifiée (3 tables minimum avec clés pri-maires/étrangères).
3. Liste 4 cas de tests unitaires pour la fonction « Calculer montant total (temps + snacks) ».
4. Le gérant appelle 3 mois plus tard : « L'imprimante thermique a changé, les reçus sortent illisibles. » De quel type de maintenance s'agit-il ? Justifie.
5. Pourquoi est-il beaucoup plus cher de corriger l'oubli de la gestion des snacks s'il est découvert lors de la recette client plutôt qu'en analyse ?

CORRIGÉ Exercice : Cybercafé NetPlus

1. Analyse :

- Acteurs : Gérant (stats, config), Employé caisse (vente, session), Client (utilise poste).
- Fonctions : Démarrer/Arrêter session, Ajouter article snack, Calculer facture, Imprimer reçu, Tableau de bord journalier.
- Règles : Facturation à la minute indivisible (ex. 100 FCFA/min), Snack déduit du stock, Session auto-coupée si solde insuffisant (prépayé).
- Contraintes : 20 postes Windows en LAN, 1 serveur central (BDD), coupures électrique fréquentes (onduleur), budget faible, formation employé 30 min.

2. BDD (MCD) :

- POSTE (IdPoste, Numero, Statut)
- SESSION (IdSession, Debut, Fin, IdPoste#, IdClient?, MontantTemps)
- ARTICLE (Code, Libelle, Prix, Stock)
- LIGNE_VENTE (IdSession#, CodeArticle#, Quantite, PrixUnitaire)
- CLIENT (IdClient, Nom, Telephone, SoldePrepayé) (optionnel)

3. Tests unitaires « CalculFacture » :

Cas	Entrées (Min, Snacks)	Attendu
TC1	30 min, 0 snack	3000 FCFA
TC2	0 min, 1 Coca (500) + 1 Pain (300)	800 FCFA
TC3	45 min, 2 Cafés (400)	4500 + 800 = 5300 FCFA
TC4	-5 min (erreur), 1 snack	Erreur / Rejet

- Maintenance adaptative. L'environnement matériel change (nouvelle imprimante, nouveau pilote, nouveau format d'impression), le besoin métier (facturer) n'a pas changé.
- Coût : En analyse : 1 discussion + modification CdC (1×). En recette : la BDD n'a pas de table ARTICLE/LIGNE_VENTE, le code de facturation ne gère que le temps, les écrans n'ont pas de bouton « Snack ». Il faut refaire Conception (BDD, IHM) + Code (nouveaux modules) + Tests (tous les scénarios) + Redéploiement (maj BDD existante, formation) ≈ 30 à 50× le coût initial.

8

L'APPROCHE EN CASCADE (CYCLE EN CASCADE)

8.1 Une réponse naturelle au besoin d'ordre et de rigueur

Dans la section précédente, nous avons identifié les grandes étapes communes à tout projet

logiciel : analyse, conception, programmation, tests, déploiement et maintenance. Une question se pose immédiatement : comment organiser ces étapes dans le temps ? La réponse la plus intuitive, et historiquement la première formalisée, consiste à les enchaîner les unes après les autres, sans retour en arrière, comme l'eau qui tombe d'une cascade. C'est l'**approche en cascade** (ou cycle en cascade, waterfall model en anglais).

Imagine que tu dois construire une maison. Tu ne commences pas par poser le toit avant d'avoir coulé les fondations. Tu ne changes pas le plan des murs une fois la toiture terminée. Chaque corps de métier intervient à son tour, valide son travail, et passe la main au suivant. En informatique, cette logique s'applique quand les besoins sont **clairs, stables et bien compris dès le départ** : un logiciel de calcul de bulletins scolaires dont les règles de moyennes sont fixées par le ministère, un programme de facturation pour une boutique dont les taxes et remises ne changent pas, un module de gestion de stock pour la bibliothèque de l'établissement.

DÉFINITION Cycle en cascade (modèle en cascade)

Le **cycle en cascade** est un modèle de développement logiciel séquentiel et linéaire dans lequel les étapes du cycle de vie (analyse, conception, programmation, tests, déploiement, maintenance) s'enchaînent dans un ordre strict et immuable. Une étape ne débute que lorsque l'étape précédente est totalement terminée et validée (livraison d'un document, d'un code, d'un rapport de test). Il n'y a, par principe, aucun chevauchement ni retour en arrière prévu entre les phases.

8.2 Le schéma en escalier : visualiser l'enchaînement strict

Le diagramme ci-dessous représente la structure canonique du cycle en cascade. Chaque « marche » correspond à une phase ; la flèche descendante unique symbolise l'avancement irréversible. Observe bien : la validation (le palier) conditionne le passage à la marche suivante.

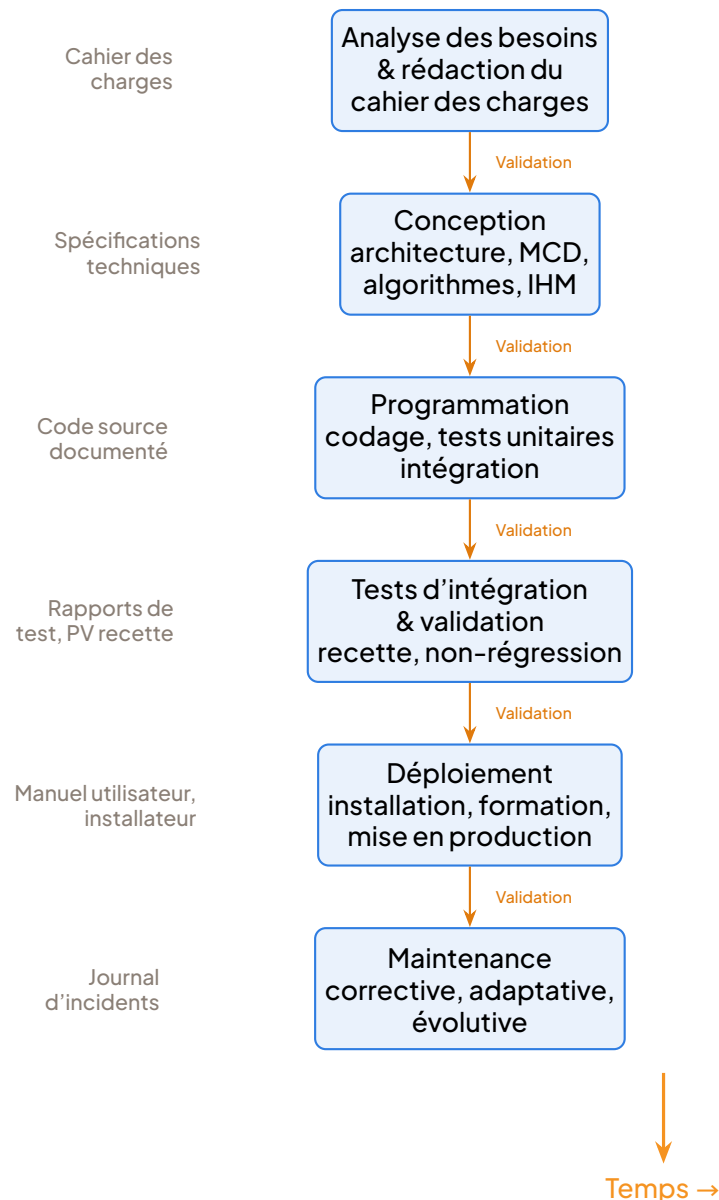


Schéma en escalier (waterfall) du cycle en cascade : chaque phase produit un livrable validé avant de passer à la suivante.

8.3 Les six phases en détail

Rappelons brièvement ce qui se joue à chaque marche, car la rigueur du cascade repose sur la **qualité des livrables** produits à chaque étape.

1. **Analyse des besoins** : Rencontres avec le client (proviseur, gestionnaire, bibliothécaire), recueil des exigences fonctionnelles (calcul des moyennes, édition de bulletins, export PDF) et non-fonctionnelles (temps de réponse, sécurité). Livrable : **cahier des charges** (CdC) signé.

2. **Conception** : Traduction du CdC en architecture technique. Choix du langage (C, Python, etc.), de la structure de données (fichiers, base SQLite), des algorithmes (tri, moyenne pondérée), des écrans (IHM). Livrable : **dossier de spécifications techniques** (MCD, organigrammes, maquettes).
3. **Programmation** : Codage pur. Chaque module est écrit, testé unitairement par le développeur, puis intégré. Respect strict des spécifications — pas d'improvisation. Livrable : **code source complet, commenté, compilé**.
4. **Tests d'intégration et validation** : Tests boîte noire (selon CdC), tests de non-régression, recette formelle avec le client. Livrable : **rapports de test, procès-verbal de recette** signé.
5. **Déploiement** : Installation sur les postes cibles (salle informatique, bureau du secrétariat), formation des utilisateurs, migration des données existantes (anciens bulletins). Livrable : **logiciel en production, manuel utilisateur**.
6. **Maintenance** : Corrections d'anomalies résiduelles (corrective), adaptations (changement de taux de TVA, nouveau format d'export — adaptative), nouvelles demandes mineures (évolutive). Cette phase dure toute la vie du logiciel.

PROPRIÉTÉ Principe de validation séquentielle (admis)

Dans le cycle en cascade, on ne passe à l'étape E_{i+1} qu'après validation complète et formelle de l'étape E_i . Cette validation repose sur un livrable tangible (document, code, rapport) examiné et accepté par les parties prenantes. Tout retour en arrière est considéré comme une anomalie de processus (coût élevé, rupture de planning) et non comme une itération normale.

8.4 Avantages : simplicité, traçabilité, maîtrise documentaire

Pourquoi ce modèle a-t-il dominé l'ingénierie logicielle pendant des décennies (années 1970–1990) et reste-t-il pertinent pour certains projets ?

- **Simplicité conceptuelle** : Facile à expliquer à un client non technique (« on fait l'étape 1, puis 2, puis 3... »). Idéal pour un premier projet d'élève en 3ème.
- **Jalons clairs et mesurables** : Chaque phase a un début, une fin, un livrable. Le chef de projet sait exactement où il en est.
- **Documentation exhaustive** : Le CdC, les spécifications, les rapports de test sont obligatoires. Cela assure la **maintenabilité** à long terme (un autre développeur peut reprendre le code dans 3 ans).
- **Adapté aux besoins stables** : Quand le client sait exactement ce qu'il veut et que cela ne bougera pas (ex. : calcul de bulletins selon arrêté ministériel fixe), le cascade évite le surcoût d'itérations inutiles.
- **Gestion contractuelle aisée** : Les paiements peuvent être calés sur les validations de phases (acompte à la signature du CdC, solde à la recette).

8.5 Limites : le piège de l'irréversibilité

C'est ici que le bât blesse pour la majorité des projets réels. Le cascade suppose qu'on peut **tout savoir, tout prévoir, tout figer** avant d'écrire la première ligne de code. La réalité en décide souvent autrement.

ATTENTION Erreur fréquente : croire qu'on peut « juste modifier un petit truc » en cours de route

Beaucoup d'élèves (et de clients !) pensent : « On est en phase de programmation, mais le proviseur veut changer la règle de calcul de la moyenne. Ce n'est qu'un *if* de plus, on le fait vite fait. » Faux. Dans un cycle en cascade strict, cela impose de :

1. Revenir en phase d'analyse (nouveau besoin),
2. Mettre à jour le cahier des charges (nouvelle version, nouvelle signature),
3. Répercuter sur la conception (algorithme, structure de données, IHM),
4. Re-coder et re-tester les modules impactés.

Le coût exponentiel du changement tardif est la critique n°1 du modèle en cascade. Négliger l'analyse pour « gagner du temps » revient à payer 10 à 100 fois plus cher la correction en phase de test ou de maintenance.

- **Le client ne voit le logiciel qu'à la fin** (phase de recette). S'il a mal exprimé son besoin, ou si son besoin a évolué (nouveau décret, changement de directeur), le logiciel livré ne correspond plus à l'attente. C'est l'effet « tunnel ».
- **Les erreurs d'analyse ou de conception sont découvertes tardivement** (en test ou en production). Corriger une erreur de modélisation de données (MCD mal fait) après le codage complet coûte extrêmement cher.
- **Pas de livraison de valeur intermédiaire** : L'établissement n'a aucun outil utilisable avant 6-12 mois. Risque de démotivation de l'équipe et du client.
- **Mauvaise gestion des risques techniques** : Si une technologie choisie en conception s'avère inadaptée (ex. : bibliothèque graphique trop lente), on le découvre en programmation, trop tard pour changer sans tout casser.

8.6 Quand choisir le cascade ? Situations d'usage en 3ème

Le cycle en cascade n'est pas « dépassé » : il est **spécialisé**. Vois-le comme un outil parmi d'autres (nous verrons les approches itératives et agiles en sections 5 et 6). Choisis-le quand **TOUTES** ces conditions sont réunies :

Condition	Exemple concret au Cameroun
Besoins clairs, complets, stables	Logiciel de bulletins : règles de calcul fixées par le MINESEC, pas de surprise.
Technologie maîtrisée, sans risque	Développement en C console ou Python + Tkinter, pas de nouvelle librairie complexe.
Projet court (quelques semaines)	TP noté sur 2–3 séances, équipe de 2–3 élèves.
Client disponible seulement au début et à la fin	Proviseur valide le CdC en septembre, fait la recette en décembre.
Exigence forte de documentation	Projet de fin d'année à transmettre au prof de l'an prochain.

8.7 Exemple concret : logiciel de bulletins scolaires pour le Collège Bilingue de Nkol-bisson

EXEMPLE Développement en cascade : « BulletinsFacile v1.0 »

Contexte : Le proviseur veut un logiciel pour éditer les bulletins du 1er trimestre. Les règles sont figées : moyenne pondérée (coeff. officiels), mention (Assez Bien ≥ 12 , Bien ≥ 14 , Très Bien ≥ 16), rang, appréciation du professeur principal. Pas d'option, pas de spécialité. Délai : 4 semaines. Équipe : 3 élèves de 3ème.

Phase 1 — Analyse (Semaine 1)

Entretien avec le surveillant général (gestionnaire des notes). Recueil : format des notes (sur 20, 2 décimales), liste des matières par série (C, D, E), coefficients, export PDF A4 paysage. Livrable : Cahier des charges v1.0 (5 pages, signé proviseur + surveillant + chef de projet élève).

Phase 2 — Conception (Semaine 1–2)

- MCD : Entités `Eleve`, `Matiere`, `Note`, `Trimestre`. Cardinalités : un élève a plusieurs notes (1,n) ; une matière a plusieurs notes (1,n).
- Algorithme principal : Pour chaque élève, pour chaque matière, lire notes → calculer moyenne → calculer moyenne générale pondérée → déterminer mention → classer → générer PDF.
- Choix technique : Python 3.11, `sqlite3` (fichier unique, pas de serveur), `fpdf2` pour PDF. IHM en console (menu texte) — simple, robuste.

Livrable : Dossier de specs techniques (MCD, organigramme principal, structure BD, maquette écran console).

Phase 3 — Programmation (Semaine 2–3)

Codage modulaire : `modele.py` (BD), `calcul.py` (moyennes, mentions, rangs), `export_pdf.py`, `main.py` (menu). Tests unitaires sur chaque fonction (ex. : `calculer_moyenne_ponderee([(12,3),(14,2)]) == 12.8`). Intégration progressive. Li-

vrable : Code source complet (dépôt Git local, tag `v1.0-dev`).

Phase 4 — Tests & Recette (Semaine 3–4)

Jeu de test : 50 élèves fictifs (série C, D, E), notes variées (entiers, décimales, absents). Vérification manuelle de 10 bulletins au calculateur. Recette formelle avec le surveillant : il valide 5 bulletins réels. Livrable : Rapport de test + PV de recette signé.

Phase 5 — Déploiement (Fin semaine 4)

Installation sur le PC du secrétariat (Windows 10, Python embarqué via `pyinstaller --onefile`). Formation 30 min au surveillant (lancer `BulletinsFacile.exe`, choisir trimestre, cliquer « Générer »). Livrable : Exécutable + mode d'emploi 1 page.

Phase 6 — Maintenance (Trimestres suivants)

Corrective : correction bug arrondi (affichage 11,9999 au lieu de 12,00). Adaptative : ajout export Excel pour le proviseur (demande mineure, nouvelle version `v1.1`). Évolutive : pas prévue (changement de règles = nouveau projet `v2.0` en cascade).

8.8 Exercice résolu : identifier la phase et le livrable

EXERCICE RÉSOLU 3

Situation : où en sommes-nous ?

Énoncé. L'équipe « CodeStars » (3 élèves de 3ème) développe un logiciel de gestion de stock pour la cantine du lycée (entrées/sorties riz, haricots, huile, alertes seuil). Le proviseur a validé le cahier des charges la semaine dernière. Aujourd'hui, l'équipe dessine le MCD (entités `Produit`, `Mouvement`, `Fournisseur`) et écrit l'organigramme de la procédure « Alerte stock bas ». Dans quelle phase du cycle en cascade sont-ils ? Quel livrable doivent-ils produire avant de passer à la phase suivante ?

Solution détaillée.

1. Identification de la phase : Le cahier des charges est validé → phase d'Analyse terminée. L'équipe travaille sur le MCD et les organigrammes → c'est la phase de Conception.
2. Livrable attendu : Pour valider la conception et passer à la programmation, l'équipe doit produire le Dossier de spécifications techniques complet, contenant au minimum :
 - Le MCD finalisé (avec cardinalités, identifiants, attributs typés).
 - Les organigrammes des algorithmes principaux (alerte, entrée, sortie, inventaire).
 - Le dictionnaire des données (tables, champs, types, contraintes).
 - Les maquettes des écrans console (menus, saisies, affichages).
 - Le choix technique justifié (langage, SGBD, bibliothèques).
3. Validation : Ce dossier doit être relu par l'équipe (auto-revue) et idéalement présenté au professeur (tuteur technique) pour validation formelle avant d'écrire la première ligne de code.

Point clé : Ne pas commencer le codage (phase 3) tant que le MCD n'est pas stabilisé et validé. Un attribut manquant (ex. : `date_peremption` sur `Produit`) découvert en phase de test aurait obligé à refaire le MCD, les requêtes SQL, le code Python, les tests... Coût multiplié par 10 minimum.

8.9 Synthèse : le cascade, un choix délibéré, pas un défaut

À RETENIR Ce qu'il faut retenir du cycle en cascade

- C'est un modèle séquentiel, linéaire, documenté : Analyse → Conception → Programmation → Tests → Déploiement → Maintenance.
- Règle d'or : On ne passe à l'étape suivante qu'après validation formelle du livrable de l'étape courante.
- Atouts : Simplicité, visibilité, traçabilité, documentation complète, adaptée aux petits projets aux besoins figés et bien compris.
- Risque majeur : Coût exponentiel du changement tardif. Le client ne voit le résultat qu'à la fin (effet tunnel). Erreurs d'analyse découvertes en test = catastrophe budgétaire.
- En 3ème : Utilise le cascade pour tes TPs notés (besoins clairs, délai court, technologie connue). Pour ton projet personnel ou un vrai client aux besoins flous → vois les sections 5 et 6 (itératif, agile).

8.10 Exercice d'application

EXERCICE 3 Choix de démarche : cascade ou pas ?

Pour chacun des projets ci-dessous, dis si le cycle en cascade est adapté ou inadapté, et justifie en une phrase en te basant sur les critères vus (stabilité des besoins, risque technique, visibilité client, documentation).

1. Un module de calcul de moyenne pondérée pour les bulletins de 3ème (coeff. officiels inchangés depuis 10 ans), à rendre dans 3 semaines.
2. Un site web de e-commerce pour une boutique de vêtements à Yaoundé : le gérant veut « voir au fur et à mesure » et change souvent d'avis sur le design.
3. Un programme en C qui trie un tableau d'entiers par tri à bulles, tri rapide et tri fusion, et affiche les temps d'exécution — sujet d'examen imposé, spécifications figées.
4. Une application mobile de géolocalisation des pharmacies de garde à Douala : l'API

de localisation est nouvelle pour l'équipe, la mairie n'a pas encore livré la base de données officielle.

5. Un script Python d'import CSV → base SQLite pour la bibliothèque du collège : format CSV stable, structure BD simple, pas d'IHM, livraison dans 5 jours.

CORRIGÉ Exercice : Choix de démarche

1. Adapté. Besoins stables (règles officielles), technologie connue, délai court, documentation exigée pour la correction.
2. Inadapté. Besoins instables (design changeant), client veut visibilité continue → approche itérative/agile nécessaire.
3. Adapté. Spécifications figées (sujet d'examen), algorithmes connus, pas de client externe, livrable unique (code + rapport).
4. Inadapté. Risque technique fort (API nouvelle, données manquantes), besoins flous → il faut prototyper, itérer, valider l'API d'abord.
5. Adapté. Besoins clairs (format CSV, schéma BD), pas d'IHM, technologie maîtrisée, court délai, documentation simple (script + mode d'emploi).

9 LE CYCLE EN V

Dans la section précédente, nous avons découvert le cycle en cascade : les étapes se succèdent comme l'eau qui descend une cascade, sans jamais remonter. Nous avons aussi identifié sa grande faiblesse : on ne teste le logiciel qu'à la fin, et si l'on découvre une erreur commise au tout début (par exemple un besoin mal compris), il faut tout recommencer. Imagine un tailleur de Douala qui coud un complet sans jamais le faire essayer au client : si la mesure du premier jour était fautive, tout le travail est perdu. Le **cycle en V** a été inventé pour corriger ce défaut, tout en gardant la simplicité de la cascade.

9.1 Présentation du cycle en V

L'idée du cycle en V est toute simple : puisque les tests arrivent trop tard dans la cascade, on va préparer chaque test en même temps que l'étape qu'il devra vérifier. Le schéma n'est plus une ligne droite qui descend, mais un **V** :

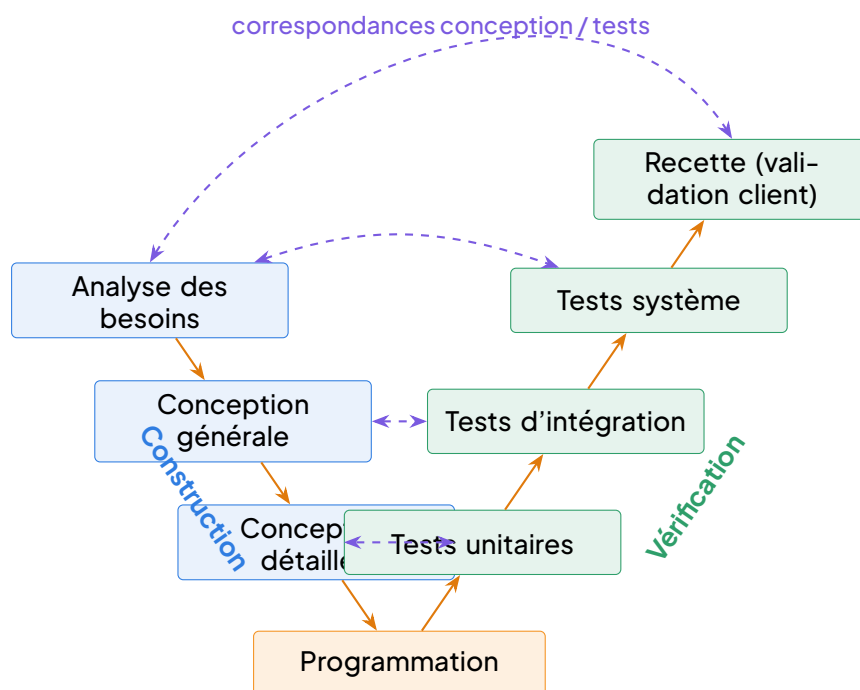
- la **branche descendante** (à gauche) contient les étapes de **construction** du logiciel : on part des besoins du client et on descend jusqu'à la programmation, qui se trouve à la **pointe du V** (en bas) ;
- la **branche montante** (à droite) contient les étapes de **vérification** : on remonte de la pointe

jusqu'à la validation finale par le client ;

- des **flèches horizontales** relient chaque étape de construction à l'étape de test qui la vérifie.

DÉFINITION Cycle en V

Le **cycle en V** est une variante du cycle en cascade dans laquelle chaque étape de construction (branche descendante) correspond à une étape de vérification (branche montante). Les tests ne sont plus une simple étape finale : ils sont préparés dès la phase de conception correspondante et exécutés après la programmation, en remontant le V.



Le cycle en V : la branche gauche construit le logiciel, la branche droite le vérifie, et chaque test correspond à une étape de conception.

9.2 La branche descendante : construire le logiciel

La branche descendante reprend les étapes de la cascade, en les détaillant davantage :

1. **Analyse des besoins** : on rencontre le client et on rédige le cahier des charges. Exemple : « l'application du lycée doit calculer les moyennes trimestrielles et imprimer les bulletins ».
2. **Conception générale** : on découpe le logiciel en grands blocs (modules). Exemple : un module « saisie des notes », un module « calcul des moyennes », un module « impression ».

3. **Conception détaillée** : on décrit précisément le contenu de chaque module : les données utilisées, les algorithmes, les formules (par exemple la formule de la moyenne pondérée avec les coefficients).
4. **Programmation** : les développeurs écrivent le code, module par module. C'est la **pointe du V**, le moment où tout ce qui a été conçu devient un vrai programme.

9.3 La branche montante : vérifier le logiciel

Une fois la programmation terminée, on remonte le V en exécutant des tests de plus en plus larges :

1. **Tests unitaires** : on teste chaque module seul. Par exemple, on vérifie que la fonction qui calcule une moyenne donne bien 12,5 pour les notes 10 et 15 de coefficient 1.
2. **Tests d'intégration** : on assemble les modules deux à deux, puis tous ensemble, et on vérifie qu'ils communiquent correctement. Par exemple, les notes saisies dans le module « saisie » arrivent-elles bien au module « calcul » ?
3. **Tests système** : on teste le logiciel complet, dans des conditions réelles, sur un vrai ordinateur du lycée. On vérifie que l'ensemble répond aux spécifications techniques issues de la conception générale.
4. **Recette** : le client (ici, le proviseur et les enseignants) essaie le logiciel et vérifie qu'il répond bien aux besoins exprimés au départ. S'il l'accepte, le logiciel est **validé** et peut être livré.

9.4 Les correspondances : le cœur du cycle en V

Voici l'idée la plus importante du cycle en V : **chaque test de la branche montante vérifie une étape précise de la branche descendante.**

Étape de construction (descente)	Test correspondant (montée)
Conception détaillée	Tests unitaires
Conception générale	Tests d'intégration et Tests système
Analyse des besoins	Recette (validation client)

Ainsi, les **tests unitaires** vérifient que chaque module respecte la conception détaillée ; les **tests d'intégration** et les **tests système** vérifient que l'architecture prévue en conception générale fonctionne correctement ; la **recette** vérifie que le logiciel répond aux besoins exprimés au tout début. Conséquence pratique très précieuse : pendant que l'on rédige la conception détaillée, on écrit déjà les tests unitaires. Les tests sont donc prêts avant même que le programme existe !

MÉTHODE Lire un schéma en V

Pour lire ou expliquer un schéma de cycle en V, procède en trois temps :

1. Descends la branche gauche, des besoins jusqu'à la programmation (la pointe du V) : c'est la construction.

2. Monte la branche droite, des tests unitaires jusqu'à la recette : c'est la vérification.
3. Relie chaque test à sa conception par une flèche horizontale : tests unitaires ↔ conception détaillée, tests d'intégration/système ↔ conception générale, recette ↔ analyse des besoins.

Si tu sais faire ces trois gestes, tu maîtrises le cycle en V.

ATTENTION Erreur fréquente : la programmation n'est pas au sommet !

Beaucoup d'élèves placent la programmation en haut du V, comme si c'était l'étape la plus importante. C'est faux : la programmation se trouve à la pointe du V, tout en bas, car c'est l'étape la plus concrète et la plus proche de la machine. En haut, on trouve les étapes les plus proches de l'utilisateur : les besoins (à gauche) et la recette (à droite). Retiens : on descend vers la machine, on remonte vers le client.

9.5 Avantages et limites du cycle en V

À RETENIR Avantages et limites

Avantages :

- les tests sont préparés tôt, en même temps que la conception : on gagne du temps et on oublie moins de cas ;
- chaque étape de construction a son test : la qualité du logiciel est meilleure et les erreurs sont repérées plus facilement ;
- le schéma reste simple à comprendre et à suivre.

Limites :

- le cycle en V reste rigide, comme la cascade : on ne passe à l'étape suivante que lorsque la précédente est terminée ;
- le client ne voit le logiciel qu'à la fin, lors de la recette : s'il a changé d'avis entre-temps, les modifications sont coûteuses.

9.6 Comparaison rapide : cascade et cycle en V

Critère	Cycle en cascade	Cycle en V
Forme du schéma	Ligne qui descend	Deux branches en V
Place des tests	Une seule étape, à la fin	Une branche entière, préparée dès la conception
Correspondance conception / tests	Aucune	Chaque test vérifie une étape de conception
Qualité du logiciel	Moyenne (erreurs découvertes tard)	Meilleure (erreurs traitées étape par étape)
Souplesse	Faible	Faible (reste rigide)

EXERCICE RÉSOLU 4 Associer tests et conceptions

Énoncé. Une entreprise de Yaoundé développe une application de gestion de stock pour une boutique. Lors d'un test, on vérifie que la fonction « calculer la quantité restante » renvoie bien 37 quand on retire 13 articles d'un stock de 50. (a) De quel type de test s'agit-il ? (b) Quelle étape de la branche descendante ce test vérifie-t-il ? (c) À quel moment ce test a-t-il été préparé dans un cycle en V ?

SOLUTION

Solution. (a) On teste une seule fonction, isolément : c'est un test unitaire. (b) Le test unitaire vérifie la conception détaillée, c'est-à-dire l'étape où l'on a décrit précisément l'algorithme de la fonction (quantité restante = stock initial - quantité retirée, soit $50 - 13 = 37$). (c) Dans le cycle en V, ce test a été préparé pendant la conception détaillée, bien avant la programmation : c'est le principe des correspondances entre les deux branches du V.

LESAVAIS-TU ? Le cycle en V dans les industries critiques

Le cycle en V est très utilisé dans les industries où une panne peut coûter des vies ou beaucoup d'argent : le transport (logiciels des avions, des trains, des voitures), la santé (appareils médicaux) et la banque (transactions financières). Dans ces domaines, chaque étape de conception doit être prouvée par un test documenté, ce qui est exactement la force du cycle en V. C'est pour cela que, même s'il est ancien, il reste enseigné et pratiqué partout dans le monde.

EXERCICE 4 À toi de jouer

Le service informatique d'un hôpital de Douala développe un logiciel de gestion des rendez-vous des patients en suivant un cycle en V.

1. Place dans l'ordre les étapes suivantes sur le V : recette, conception détaillée, tests d'intégration, analyse des besoins, programmation, tests unitaires, conception générale, tests système.
2. Le médecin-chef essaie le logiciel complet et déclare : « C'est bien ce que nous avons demandé. » À quelle étape sommes-nous ? Quelle étape de la branche descendante cette étape vérifie-t-elle ?
3. Cite un avantage et une limite du cycle en V pour ce projet.

CORRIGÉ Exercice : À toi de jouer

1. Branche descendante : analyse des besoins → conception générale → conception détaillée → programmation (pointe). Branche montante : tests unitaires → tests d'intégration → tests système → recette.
2. Il s'agit de la recette (validation par le client). Elle vérifie l'analyse des besoins, la toute première étape : le logiciel livré doit correspondre à ce qui avait été demandé.
3. Avantage : les tests sont préparés dès la conception, donc la qualité est meilleure — essentiel dans un hôpital. Limite : le cycle reste rigide ; si les médecins changent un besoin en cours de route, il est difficile et coûteux de revenir en arrière.

En résumé, le cycle en V améliore la cascade en associant à chaque étape de construction une étape de vérification : on descend pour construire, on remonte pour tester, et chaque test correspond à une conception. Mais il reste rigide : le client attend la fin pour voir le produit. C'est précisément ce problème que les approches itératives et agiles, que nous étudierons dans la prochaine section, cherchent à résoudre.

10 APPROCHES ITÉRATIVES ET AGILES

Dans la section précédente, nous avons découvert le **cycle en V** : on descend d'abord par les étapes d'analyse et de conception, puis on remonte par les étapes de réalisation et de tests, chaque test vérifiant l'étape qui lui fait face. Cette méthode est rigoureuse, mais elle garde un défaut hérité du cycle en cascade : **l'utilisateur ne voit le logiciel qu'à la toute fin**, parfois des mois après avoir exprimé son besoin. Que se passe-t-il si, entre-temps, ses besoins ont changé ? Ou si, en découvrant le logiciel, il s'exclame : « ce n'est pas du tout ce que je voulais ! » ?

C'est précisément pour répondre à ce problème que sont nées les **approches itératives** puis les **méthodes agiles**. L'idée est simple et très concrète : au lieu de construire le logiciel d'un seul coup, on le construit **par petits morceaux, version après version**, en montrant régulièrement le résultat à l'utilisateur. C'est exactement ce qui se passe avec les applications de ton téléphone : tu les installes, puis elles sont mises à jour régulièrement avec de nouvelles fonc-

tions. Étudions cela en détail.

10.1 L'approche itérative et incrémentale

Intuition. Imagine que tu dois rédiger un long exposé. Deux stratégies s'offrent à toi : soit tu écris tout l'exposé d'un trait, sans le relire, et tu le déposes ; soit tu écris un premier plan simple, tu le montres au professeur, tu l'améliores, tu ajoutes une partie, tu le remontres, etc. La deuxième stratégie est plus sûre : les erreurs d'orientation sont corrigées très tôt. L'approche itérative applique cette idée au développement logiciel.

DÉFINITION Approche itérative et incrémentale

Une **approche itérative** est une démarche de développement dans laquelle on répète plusieurs fois les étapes du cycle de vie (analyser, concevoir, programmer, tester), chaque répétition s'appelant une **itération**. Elle est dite **incrémentale** lorsque chaque itération ajoute de nouvelles fonctionnalités au logiciel : on obtient ainsi des versions successives de plus en plus complètes (version 1, version 2, version 3, ...).

Contrairement au cycle en cascade où l'on traverse **une seule fois** les étapes, ici on traverse les mêmes étapes **plusieurs fois**, en boucle :

Analyser → Concevoir → Programmer → Tester → (on recommence en améliorant)

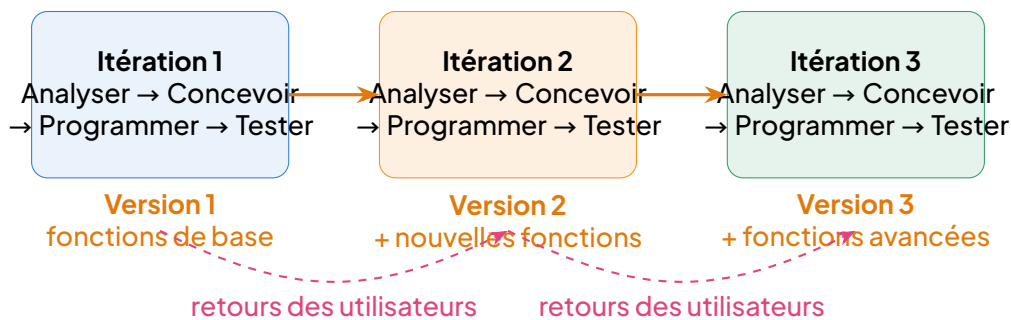
À la fin de chaque boucle, on obtient une **version utilisable** du logiciel, que l'on peut montrer aux utilisateurs. Leurs remarques servent de point de départ à la boucle suivante.

EXEMPLE Une application de livraison à Douala

Une petite entreprise de Douala veut créer une application mobile de livraison de repas. Au lieu de tout développer pendant un an, elle procède par versions :

- Version 1 : le client choisit un restaurant, commande et paie à la livraison. C'est tout, mais l'application fonctionne déjà !
- Version 2 : après les retours des utilisateurs, on ajoute le paiement par mobile money et le suivi du livreur sur une carte.
- Version 3 : on ajoute les notes et commentaires sur les restaurants, puis un programme de fidélité.

Chaque version est réellement utilisée par les clients, et leurs réactions guident la version suivante. L'entreprise gagne de l'argent dès la version 1, au lieu d'attendre un an.



Les itérations successives : chaque boucle reprend les étapes du cycle de vie et produit une version plus riche, enrichie grâce aux retours des utilisateurs.

10.2 Les méthodes agiles

Intuition. L'approche itérative améliore déjà beaucoup les choses, mais on peut aller plus loin : raccourcir encore les itérations (quelques jours ou quelques semaines au lieu de plusieurs mois) et faire participer le client **en permanence**, pas seulement à la fin de chaque version. C'est l'esprit des méthodes agiles.

DÉFINITION Approche agile

Une **approche agile** est une démarche de développement fondée sur des cycles très courts appelés **itérations**, une collaboration permanente avec le client et une grande capacité d'adaptation au changement : si le besoin évolue en cours de route, on ajuste le travail à l'itération suivante au lieu de figer tout le projet au départ.

Un principe agile typique se résume ainsi : **livrer vite une première version utilisable, puis l'améliorer régulièrement selon les retours des utilisateurs**. On préfère un logiciel simple mais disponible tout de suite à un logiciel parfait livré dans deux ans.

PROPRIÉTÉ Détecter tôt coûte moins cher (admise)

Plus une erreur est détectée tôt dans le développement, moins elle coûte cher à corriger. Une erreur d'analyse découverte au tout début se corrige en modifiant quelques lignes d'un document ; la même erreur découverte après des mois de programmation oblige à réécrire une grande partie du logiciel. C'est l'intérêt majeur des cycles courts : en testant et en montrant le logiciel très souvent, on détecte les erreurs au plus tôt.

ATTENTION Agile ne veut pas dire désorganisé

Erreur fréquente : croire qu'une méthode agile signifie travailler sans organisation ni do-

cumentation. C'est faux ! Une équipe agile planifie chaque itération, note les besoins, teste chaque version et tient des réunions régulières. La différence est que la documentation reste légère et utile, et que la planification accepte le changement au lieu de le refuser.

10.3 Avantages et limites

Avantages	Limites
Grande souplesse : on peut changer d'orientation en cours de route.	Demande une forte implication du client , qui doit être disponible très souvent.
Retour rapide des utilisateurs, qui utilisent le logiciel dès les premières versions.	Moins adapté aux projets dont les besoins sont totale-ment figés dès le départ (ex. logiciel embarqué très réglementé).
Les erreurs sont détectées tôt , donc corrigées à moindre coût.	Sans bonne organisation, le projet peut manquer de vision globale à long terme.

EXERCICE RÉSOLU 5 Choisir la bonne approche

Énoncé. La mairie d'une petite ville veut un site web d'information sur ses services (horaires, annonces, actes de naissance). Le contenu exact du site n'est pas encore fixé : les élus souhaitent voir des maquettes régulièrement et faire évoluer leurs idées. Quelle approche de développement conseilles-tu ? Justifie ta réponse.

SOLUTION

Solution. Les besoins ne sont pas figés : ils vont évoluer au fil des maquettes et des discussions. Les élus sont disponibles pour donner leur avis régulièrement. On conseille donc une approche itérative ou agile : on livre d'abord une version simple du site (page d'accueil et horaires), on la montre aux élus, puis on l'enrichit à chaque itération (annonces, démarches en ligne, ...). Le cycle en cascade ou le cycle en V seraient mal adaptés ici, car ils supposent que tous les besoins sont définis précisément dès le départ, ce qui n'est pas le cas.

10.4 Tableau comparatif des quatre approches

Critère	Cascade	Cycle en V	Itératif	Agile
Ordre des étapes	Linéaire, une seule fois, sans retour en arrière	Linéaire, avec tests préparés dès la conception	Étapes répétées en boucles, versions successives	Très courtes boucles, ajustées en permanence
Souplesse	Très faible	Faible	Bonne	Excellente
Implication du client	Au début seulement	Au début et à la recette finale	À la fin de chaque version	Permanente
Cas d'usage	Petit projet aux besoins clairs et stables	Projet exigeant une forte qualité et des tests rigoureux	Projet évolutif, livré par versions	Projet aux besoins changeants, client très disponible

LESAVAIS-TU ? Les mises à jour de ton téléphone

Les applications de nos téléphones (messageries, réseaux sociaux, jeux) sont mises à jour très régulièrement, parfois toutes les semaines. Ce rythme est possible grâce aux cycles courts de développement : les équipes publient une version, observent comment elle est utilisée, corrigent les erreurs signalées et ajoutent de petites nouveautés à chaque mise à jour. Au Cameroun comme ailleurs, les applications de mobile money évoluent ainsi en permanence pour répondre aux besoins des utilisateurs.

À RETENIR L'essentiel

- L'approche itérative et incrémentale développe le logiciel par versions successives, en répétant la boucle : analyser → concevoir → programmer → tester.
- Les méthodes agiles poussent cette idée plus loin : itérations très courtes, collaboration permanente avec le client, adaptation au changement.
- Principe clé : livrer vite une première version utilisable, puis l'améliorer selon les retours des utilisateurs.
- Plus une erreur est détectée tôt, moins elle coûte cher à corriger : c'est tout l'intérêt des cycles courts.
- Agile ne signifie pas désorganisé : il faut toujours planifier, tester et documenter.

EXERCICE 5 À toi de jouer

Un groupe d'élèves de 3ème veut créer une petite application de quiz de révision pour le

BEPC. Ils ne savent pas encore exactement quelles matières ni quelles fonctions inclure, mais leurs camarades sont prêts à tester chaque version et à donner leur avis.

1. Quelle approche de développement leur conseilles-tu ? Justifie.
2. Propose le contenu d'une version 1, d'une version 2 et d'une version 3 de cette application.
3. Cite un avantage et une limite de l'approche choisie dans ce contexte.

CORRIGÉ Exercice

1. Une approche itérative ou agile, car les besoins ne sont pas figés et les utilisateurs (les camarades) sont disponibles pour tester régulièrement et donner leur avis.
2. Version 1 : quiz de mathématiques à choix multiples avec score final. Version 2 : ajout du français et de l'anglais, correction détaillée après chaque question. Version 3 : suivi des progrès, minuteur, classement entre camarades.
3. Avantage : les erreurs et les mauvaises idées sont détectées très tôt grâce aux tests des camarades, et l'application est utilisable dès la version 1. Limite : il faut que les camarades restent disponibles et motivés pour tester chaque version, sinon les retours manquent.

11 CHOISIR ET JUSTIFIER UNE DÉMARCHE DE DÉVELOPPEMENT

Après avoir découvert les approches itératives et agiles, tu disposes maintenant d'une « boîte à outils » méthodologique complète : le cycle en cascade, le cycle en V, les démarches itératives et l'agilité. Mais posséder les outils ne suffit pas : il faut savoir lequel sortir pour quel travail. Un marteau est parfait pour un clou, mais inutile pour visser une ampoule. De même, imposer un cycle en V rigide à une start-up qui doit livrer une démo dans deux semaines serait une erreur stratégique ; à l'inverse, gérer la construction du système de pilotage d'un satellite avec une approche « on verra au fur et à mesure » serait irresponsable.

Cette section t'apprendra à **analyser le contexte d'un projet** pour choisir l'approche la plus adaptée, et surtout à **rédigier une justification écrite, structurée et convaincante** — une compétence clé pour l'examen du BEPC et pour ta future vie professionnelle.

11.1 Critères de choix : lire le projet comme une carte d'identité

Avant de trancher, un chef de projet (ou toi, en situation d'examen) doit interroger le projet sur cinq dimensions fondamentales. C'est l'équivalent d'un diagnostic médical avant de prescrire le traitement.

DÉFINITION Critères de choix d'une approche de développement

Le choix d'une démarche de développement logiciel repose sur l'analyse de cinq critères principaux :

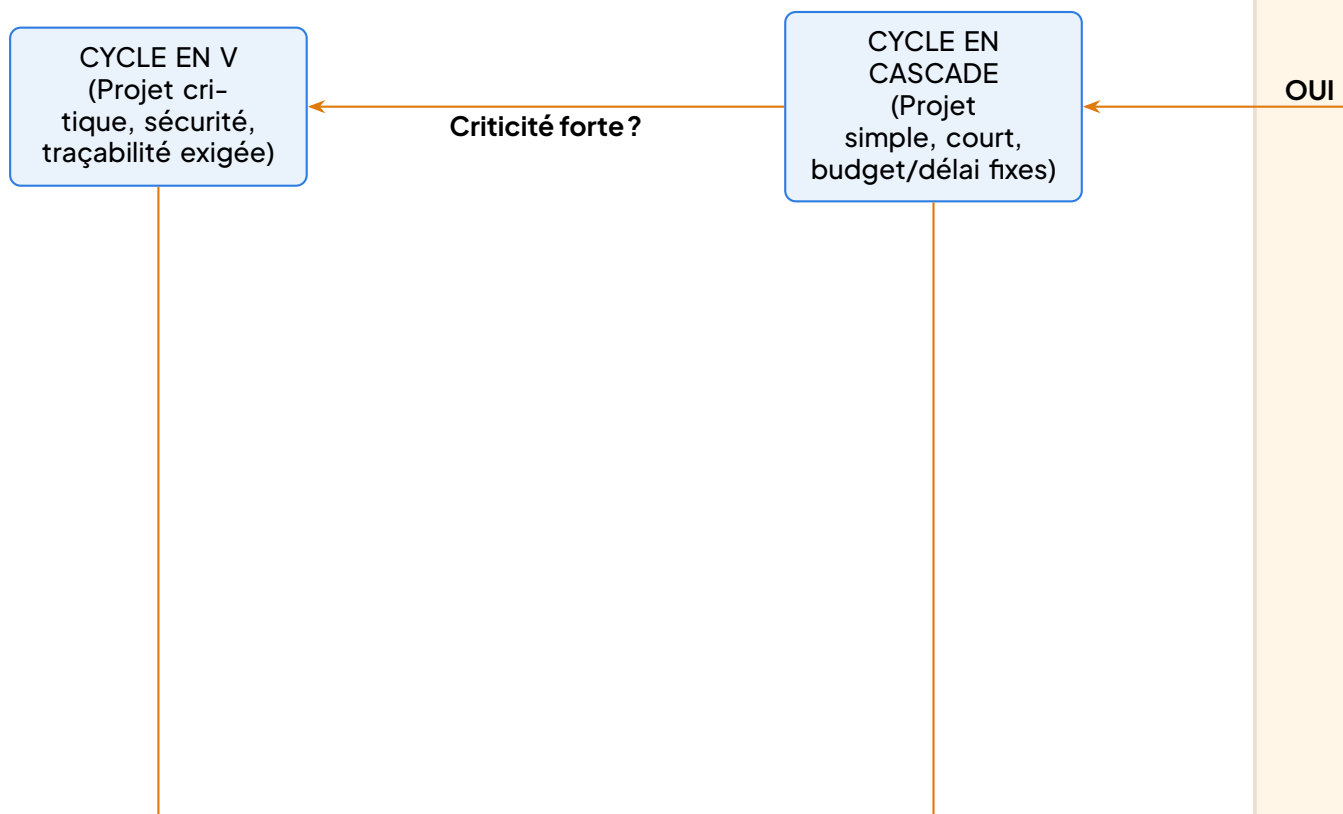
1. La clarté et la stabilité des besoins : Le client sait-il exactement ce qu'il veut ? Les besoins sont-ils figés par un contrat ou une réglementation (ex. : logiciel bancaire, système embarqué) ou évoluent-ils sans cesse (ex. : site e-commerce, application mobile grand public) ?
2. La taille et la complexité du projet : S'agit-il d'un petit module (quelques centaines de lignes, 1-2 développeurs) ou d'un système critique de grande envergure (millions de lignes, équipes distribuées, interfaces multiples) ?
3. Les délais (Time-to-market) : La date de livraison est-elle imposée et courte (avantage aux approches itératives/agiles qui livrent de la valeur tôt) ou le projet a-t-il un horizon long avec des jalons fixes (avantage au cycle en V) ?
4. Le budget et la tolérance au risque : Le budget est-il verrouillé dès le départ ? Le coût du changement en fin de projet est-il prohibitif (matériel, certification, sécurité) ?
5. La disponibilité du client / utilisateur final : Le client peut-il valider des versions intermédiaires chaque semaine (agile) ou n'est-il disponible qu'au début (cadrage) et à la fin (recette) ?

11.2 Règle pratique : le grand partage « Besoins stables vs Besoins évolutifs »

Parmi les cinq critères, **la stabilité des besoins** est le discriminant le plus puissant. Elle dicte l'architecture même du processus.

PROPRIÉTÉ Règle de décision principale

- Besoins clairs, complets, stables et contractuels → Cycle en cascade (projets courts, simples) ou Cycle en V (projets critiques, besoin de traçabilité forte, validation à chaque niveau).
- Besoins flous, incomplets, changeants ou découverts progressivement → Approche itérative (livraisons successives de sous-ensembles fonctionnels) ou Approche Agile (Scrum, Kanban : cycles courts, client intégré à l'équipe, adaptation continue).



Arbre de décision pour choisir une approche de développement selon la stabilité des besoins et le contexte projet.

11.3 Rédiger et justifier une démarche : la méthode en 4 étapes

Au BEPC, on ne te demandera pas seulement de cocher une case. Il te faudra **produire un texte argumenté**. Voici la méthode standard, utilisable en examen comme en entreprise.

MÉTHODE Rédiger une justification de choix méthodologique (4 étapes)

Pour justifier le choix d'une approche de développement, structure ton paragraphe ainsi :

1. Présenter le besoin (Situation) : Résume en une phrase le contexte, le commanditaire, l'objectif du logiciel et les contraintes majeures (délai, budget, criticité).
2. Lister les étapes prévues (Démarche) : Décris brièvement comment le projet va s'organiser (phases successives, itérations, sprints, livraisons intermédiaires).
3. Nommer l'approche choisie : Cite explicitement le nom de la démarche (Cycle en V, Scrum, Itératif incrémental, etc.).
4. Justifier par au moins deux arguments ancrés dans le contexte : Relie chaque argument à un critère du projet (ex. : « Car le client ne peut valider qu'à la fin, donc le cycle en V assure la traçabilité » ; « Car les besoins évolueront, donc Scrum permet l'adaptation »).

ATTENTION Erreur fréquente : le choix « au feeling » sans justification contextuelle

- Erreur : « Je choisis l'agile car c'est moderne et flexible. » → 0 point. L'agile n'est pas « mieux », il est adapté (ou pas).
- Erreur : « Je choisis le cycle en V car c'est rigoureux. » → 0 point. La rigueur n'est pas un argument si le projet demande de la souplesse.
- Bonne pratique : « Je choisis le cycle en V car le cahier des charges est figé par la réglementation bancaire (besoins stables) et car la certification impose une traçabilité complète des tests à chaque niveau de conception (criticité forte). »

11.4 Applications concrètes : trois situations camerounaises

Appliquons la méthode à des cas réalistes que tu pourrais croiser au BEPC ou dans ton environnement.

11.5 Logiciel de gestion de la cantine scolaire (Collège de Ngoa-Ekellé)

EXERCICE RÉSOLU 6 Choix d'approche pour une cantine scolaire

Énoncé : Le principal du Collège de Ngoa-Ekellé veut informatiser la gestion de la cantine : achats de vivres, gestion des stocks, édition des tickets repas pour 800 élèves, comptabilité journalière. Le budget est voté par le conseil d'administration, le cahier des charges est validé par l'intendant et le comptable. La livraison est impérative pour la rentrée de septembre (dans 4 mois). L'intendant ne sera disponible que pour la recette finale.

Solution détaillée :

1. Situation : Projet de gestion administrative scolaire, besoins métiers stables (règles de gestion comptable immuables), délai fixe (4 mois), client (intendant) peu disponible en cours de projet, budget verrouillé.
2. Démarche prévue : 1. Spécifications détaillées figées (CDC). 2. Conception architecture/base de données. 3. Codage modules (Stocks, Compta, Tickets). 4. Tests unitaires puis intégration. 5. Recette finale avec intendant. 6. Déploiement formation.
3. Approche choisie : Cycle en V.
4. Justification : J'opte pour le cycle en V car les besoins sont parfaitement définis et stables dès le départ (règles comptables, nombre d'élèves connu), ce qui permet une conception complète en amont sans risque de retour arrière coûteux. De plus, la faible disponibilité de l'intendant (client) pendant le développement impose une validation unique en fin de projet (recette), ce que structure nativement le cycle en V par sa symétrie conception/validation.

11.6 Application mobile de suivi des notes pour parents d'élèves (Start-up « EduConnect » à Douala)

EXERCICE RÉSOLU 7 Choix d'approche pour une application mobile

Énoncé : Une jeune start-up doulaïse développe « NoteFacile », une appli Android/iOS pour que les parents consultent les notes, absences et bulletins de leurs enfants. Le concept de base est clair, mais les fonctionnalités précises (notifications push, graphiques d'évolution, messagerie prof/parents) ne sont pas figées : les fondateurs veulent tester le marché vite (MVP Minimum Viable Product dans 6 semaines) et adapter selon les retours utilisateurs. L'équipe est de 3 développeurs + 1 designer. Les fondateurs sont disponibles quotidiennement.

Solution détaillée :

1. Situation : Projet innovant B2C, besoins flous/évolutifs (découverte marché), délai très court pour première version (MVP), équipe petite et collocalisée, client (fondateurs/PO) hyper-disponible.
2. Démarche prévue : Organisation en Sprints de 2 semaines. Sprint 1 : Authentification + Affichage notes (MVP). Livraison bêta sur Play Store/TestFlight. Retours utilisateurs (parents volontaires). Sprint 2 : Graphiques + Notifications. Sprint 3 : Messagerie. Revue et adaptation continue du backlog.
3. Approche choisie : Approche Agile (Scrum).
4. Justification : Je choisis Scrum car les besoins ne sont pas stabilisés : l'équipe doit découvrir ce que veulent vraiment les parents par itérations successives (MVP puis enrichissements). Par ailleurs, la disponibilité quotidienne des fondateurs (Product Ow-

ner) permet les cérémonies Scrum (Daily, Review, Planning) et une réorientation rapide du backlog, condition sine qua non de l'agilité.

11.7 Site web de la coopérative agricole « Cacao Vert » (Région du Sud)

EXERCICE RÉSOLU 8 Choix d'approche pour un site web vitrine + catalogue

Énoncé : La coopérative « Cacao Vert » (200 producteurs) veut un site web : présentation de la coop, catalogue des produits (cacao, café, banane plantain) avec fiches techniques, formulaire de contact commande, espace adhérent (comptes rendus AG, paiements). Le président a rédigé un cahier des charges précis de 15 pages. Le budget est une subvention fixe (500 000 FCFA). Le développeur est un prestataire externe (agence web à Yaoundé). Le président valide les maquettes (2 réunions prévues) puis la version finale. Délai : 8 semaines.

Solution détaillée :

1. Situation : Projet web standard (vitrine + catalogue + espace membre), besoins bien cadrés par CDC écrit, budget fixe, prestataire externe (relation contractuelle), client disponible seulement 2 fois (validation maquettes, recette), délai modéré.
2. Démarche prévue : 1. Atelier cadrage + maquettes Figma (Validation 1). 2. Développement Back/Front complet. 3. Tests interne agence. 4. Recette finale avec président (Validation 2). 5. Mise en production + formation admin.
3. Approche choisie : Cycle en cascade (ou Cycle en V simplifié).
4. Justification : Le cycle en cascade est pertinent car le cahier des charges est complet, validé et contractuel (subvention), éliminant le risque d'évolution majeure des besoins. Ensuite, la relation client/prestataire externe avec seulement deux points de validation imposés (maquettes, recette) correspond parfaitement à la structure séquentielle du cascade : on fige les spécifications, on développe, on livre.

11.8 Exemple chiffré : mini-projet 2 semaines, client dispo chaque jour

EXEMPLE Mini-projet « Bibliothèque Express » — Approche itérative en 3 versions

Contexte : Dans le cadre du club informatique, tu dois coder en Python (ou C) une mini-appli console de gestion de prêts de livres (ajouter livre, prêter, rendre, lister). Durée : 2 semaines (6 séances de 1h30). Ton professeur (client) est présent à chaque séance pour tester et valider.

Analyse critères :

- Besoins : Simples, mais le professeur veut voir l'interface évoluer (besoins découverts progressivement).
- Taille : Très petite (1 développeur, 200 lignes).
- Délai : Court, découpé en séances fixes.
- Client : Disponible chaque jour (critère décisif).

Choix : Approche Itérative Incrémentale (3 itérations de 2 séances).

Planification :

Itération	Objectif (Incrément livré)	Validation Prof
#1 (Séances 1-2)	Noyau : Structure 'Livre' (titre, auteur, dispo), Menu principal, Ajouter/Lister livres.	./biblio → Ajoute 3 livres → Liste OK.
#2 (Séances 3-4)	Prêt/Retour : Fonction 'preter(id, eleve)', 'rendre(id)', Gestion erreurs (déjà prêté, inexistant).	Scénario : Prêt « Germinal » à « Paul » → OK. Rendre → OK. Erreur double prêt → Message géré.
#3 (Séances 5-6)	Finition : Sauvegarde fichier CSV (persistance), Recherche par auteur, Rapport « livres en retard ».	Redémarrage appli → données conservées. Recherche « Zola » → trouve Germinal.

Justification type BEPC : « Pour ce mini-projet de 2 semaines, je choisis une approche itérative incrémentale en 3 versions. Premier argument : la disponibilité quotidienne du client (professeur) permet une validation continue à la fin de chaque itération de 2 séances, ce qui est la condition de succès de l'itératif. Deuxième argument : la taille réduite du projet autorise une réécriture ou un remaniement (refactoring) rapide entre itérations sans risque planning, ce qui aurait été impossible sur un gros projet. »

11.9 Synthèse : ton aide-mémoire pour l'examen

À RETENIR

Check-list « Choix & Justification » pour le BEPC

- Identifie le critère roi : Besoins stables? → Cascade / V. Besoins changeants? → Itératif / Agile.
- Vérifie le critère « Client dispo » : Si client absent → pas d'Agile/Scrum (pas de Daily, pas de Review). Si client présent → Agile/Itératif favorisé.
- Structure ta réponse : Contexte → Démarche (étapes) → Nom approche → Justification (2 arguments min. liant critère projet → propriété approche).

- Vocabulaire précis : Utilise « Cahier des charges », « Recette », « MVP », « Sprint/Itération », « Traçabilité », « Incrément », « Backlog », « Product Owner ».
- Piège : Ne confonds pas « Itératif » (livraisons successives planifiées) et « Agile » (cadre Scrum/Kanban, valeurs manifesto, adaptation continue). Au niveau 3ème, « Approche itérative » est le terme générique attendu pour les cycles courts avec validation intermédiaire.

EXERCICE 6 Entraînement BEPC — Choix et justification

Sujet : L'administration d'un hôpital de district (150 lits) souhaite informatiser la gestion des dossiers patients (admission, antécédents, prescriptions, facturation). La réglementation médicale impose des règles strictes de confidentialité et de traçabilité (loi sur les données de santé). Le directeur a validé un cahier des charges détaillé de 50 pages. Le budget est une dotation annuelle fixe. L'équipe médicale (médecins, infirmiers) ne pourra tester le logiciel qu'à la fin du développement (disponibilité nulle en cours de projet). Le délai est de 6 mois.

1. Cite les trois critères principaux qui orientent le choix de l'approche dans ce contexte.
2. Nomme l'approche la plus adaptée.
3. Rédige une justification structurée en appliquant la méthode en 4 étapes (Situation, Démarche, Approche, Justification 2 arguments).

CORRIGÉ Exercice n°1 — Hôpital de district

1. Critères déterminants : (1) Besoins stables et réglementés (loi santé, CDC 50 pages validé). (2) Criticité forte / Traçabilité exigée (données sensibles, vie humaine). (3) Client (corps médical) indisponible pendant le développement.
2. Approche choisie : Cycle en V.
3. Justification rédigée : Situation : Il s'agit de développer un système d'information hospitalier (SIH) pour un hôpital de district, soumis à une réglementation stricte sur les données de santé, avec un cahier des charges figé de 50 pages, un budget fixe et un corps médical indisponible pour des validations intermédiaires. Démarche prévue : Le projet suivra les phases symétriques du V : Spécifications générales → Spécifications détaillées → Conception architecture/BD → Codage modules (Admission, Dossier, Facturation) → Tests unitaires → Tests d'intégration → Tests système → Recette utilisateur finale (médecins) → Mise en production. Approche : Cycle en V. Justification : Je choisis le cycle en V car la stabilité totale des besoins (cadré par la loi et le CDC validé) permet une conception complète et figée en amont, évitant des retours arrière coûteux sur l'architecture de la base de données médicale. De plus, la criticité

forte du domaine (santé, confidentialité, traçabilité légale) impose une correspondance rigoureuse entre chaque niveau de conception et son niveau de test (traçabilité exigée par la réglementation), ce que garantit nativement la structure en V. Enfin, l'indisponibilité du corps médical pendant la phase de réalisation rend impossible toute approche itérative ou agile nécessitant des validations fréquentes ; le cycle en V concentre la validation utilisateur sur la phase de recette finale, unique moment où les médecins sont disponibles.

LESAVAIS-TU ? Le « Manifeste Agile » au Cameroun ?

Tu entends souvent « Agile » dans les tech hubs de Douala (Silicon Mountain) ou Yaoundé (Jongo Hub). Mais sais-tu que le Manifeste Agile (2001) privilégie « la collaboration avec le client plutôt que la négociation contractuelle » ? Au Cameroun, beaucoup de marchés publics ou projets ONG imposent des cahiers des charges fermes et des appels d'offres rigides. Résultat : L'agile pur (Scrum) y est souvent difficile à appliquer « by the book ». On voit donc émerger des hybrides : un cycle en V pour le contrat/cadrage (spécifs figés), puis des sprints internes en agile pour la construction technique, avec des démos régulières au MOA (Maître d'Ouvrage) quand il est dispo. Choisir une démarche, c'est aussi savoir l'adapter à la réalité du terrain !

12 ACTIVITÉ D'INTÉGRATION

13 LEÇON 16 : INTRODUCTION AUX APPROCHES DE DÉVELOPPEMENT LOGICIEL

13.1 Logiciel et cycle de développement : premières définitions

DÉFINITION Logiciel

Un **logiciel** est un ensemble de programmes, de procédures, de règles et de documentation associés au fonctionnement d'un système de traitement de l'information. Contrairement au matériel (hardware), il est immatériel.

DÉFINITION Cycle de vie d'un logiciel

Le **cycle de vie d'un logiciel** (ou cycle de développement) décrit la succession des phases par lesquelles passe un logiciel, depuis l'idée initiale jusqu'à son retrait du service. Il structure le travail de l'équipe de développement pour garantir la qualité du produit final.

DÉFINITION Cahier des charges

Le **cahier des charges** (ou spécifications des besoins) est le document de référence qui traduit les besoins du client en fonctions techniques précises, contraintes (délais, budget, performance) et critères d'acceptation. Il sert de contrat entre le client et le développeur.

À RETENIR Vocabulaire clé

- **Analyse des besoins** : comprendre « quoi » et « pourquoi ».
- **Conception** : décider « comment » (architecture, algorithmes, interfaces, base de données).
- **Programmation (ou codage)** : écrire le code source dans un langage (C, Python, HTML/JS, etc.).

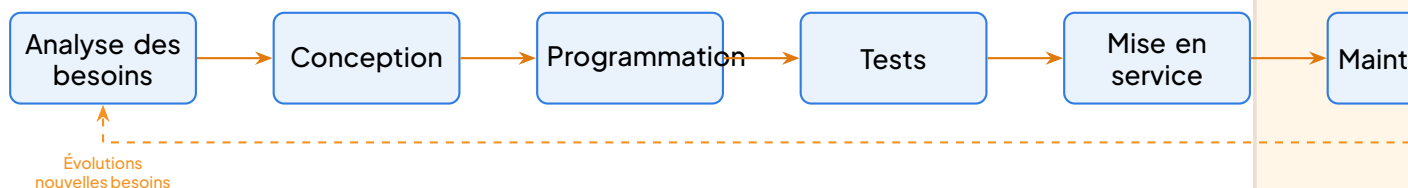
- **Tests** : vérifier que le logiciel fait ce qui est demandé (tests unitaires, d'intégration, de validation).
- **Mise en service (déploiement)** : installer chez le client, former les utilisateurs, migrer les données.
- **Maintenance** : corriger les bogues (corrective), adapter aux évolutions (évolutive), améliorer (parfaite).

13.2 Les étapes du cycle de vie d'un logiciel

Le cycle de vie classique comporte **six étapes séquentielles**. Même si les approches modernes les revisitent, ces six activités restent indispensables.

PROPRIÉTÉ Les 6 étapes du cycle de vie (ordre logique)

1. Analyse des besoins : recueil auprès des utilisateurs, rédaction du cahier des charges.
2. Conception : architecture globale (macro-conception) et détaillée (micro-conception : algorithmes, MCD, maquettes d'écrans).
3. Programmation : écriture du code source, respect des normes de codage.
4. Tests : exécution du logiciel avec des jeux de données pour détecter les anomalies (tests unitaires, d'intégration, système, acceptation).
5. Mise en service : installation en production, formation, démarrage assisté.
6. Maintenance : corrections, adaptations, améliorations tout au long de la vie du logiciel.



Cycle de vie classique en cascade (flèche continue) et retour possible en maintenance évolutive (flèche pointillée).

13.3 L'approche en cascade (cycle en cascade)

DÉFINITION Cycle en cascade

Le **cycle en cascade** (ou modèle en cascade) est une approche séquentielle stricte : chaque étape doit être totalement terminée et validée avant de passer à la suivante. Il n'y a pas de retour en arrière prévu (ou alors très coûteux).

PROPRIÉTÉ Caractéristiques de la cascade

- Avancement linéaire : Analyse → Conception → Code → Tests → Déploiement.
- Validation formelle à la fin de chaque étape (jalons).
- Documentation exhaustive produite dès le début.

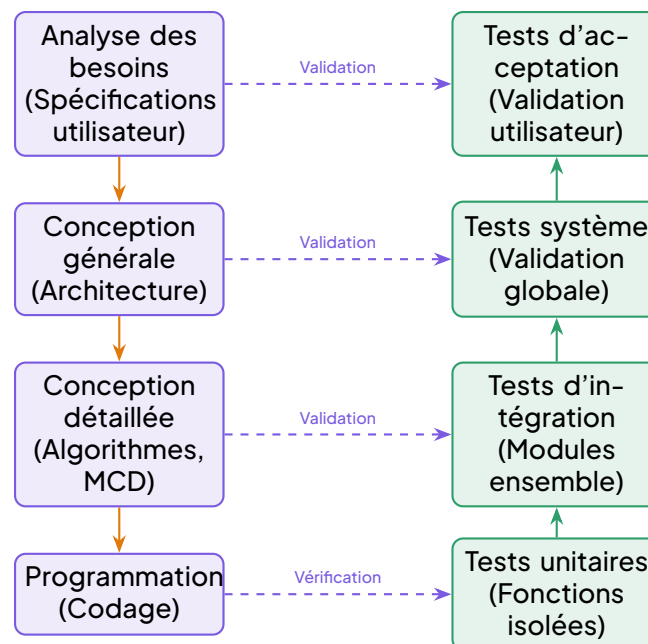
À RETENIR Quand choisir la cascade ?

- Besoins clairs, stables, complets et figés dès le départ.
- Client peu disponible pendant le développement (seulement au début et à la fin).
- Technologie maîtrisée, pas de risque technique majeur.
- Délai et budget fixes, une seule livraison attendue.

13.4 Le cycle en V

DÉFINITION Cycle en V

Le **cycle en V** (modèle en V) est une variante de la cascade qui met l'accent sur la validation et la vérification (V&V). À chaque phase de descente (spécifications) correspond une phase de remontée (tests) symétrique.



Modèle en V : la branche gauche (descente) définit le « quoi », la branche droite (remontée) vérifie le « comment ».

À RETENIR Quand choisir le cycle en V ?

- Besoins bien définis et stables (comme la cascade).
- Fiabilité et qualité critiques (logiciels bancaires, médicaux, embarqués, gestion de paiements).
- Nécessité de tracer chaque test vers une exigence précise (traçabilité).
- Client disponible pour la validation finale (tests d'acceptation), mais peu pendant la conception.

13.5 Approches itératives et agiles

DÉFINITION Approche itérative

L'**approche itérative** décompose le projet en **versions successives** (itérations). Chaque itération traverse un mini-cycle de vie complet (analyse → code → test) et livre un sous-ensemble fonctionnel. Le logiciel grandit par enrichissements successifs.

DÉFINITION Approche agile

L'**approche agile** (ex. Scrum, XP) pousse l'itération à l'extrême : cycles très courts (**sprints** de 2 à 4 semaines), collaboration quotidienne avec le client (Product Owner), adaptation permanente au changement, priorité au logiciel fonctionnel sur la documentation exhaustive.

PROPRIÉTÉ Différences clés Itératif vs Agile

Critère	Itératif (ex. RUP simplifié)	Agile (ex. Scrum)
Durée d'un cycle	Quelques semaines à quelques mois	Court : 2 à 4 semaines (Sprint)
Planification	Globale au début, affinée par itération	Juste-à-temps (Sprint Planning)
Client	Disponible aux jalons (fin d'itération)	Implication quotidienne (Product Owner)
Changements	Acceptés entre itérations	Bienvenus à tout moment
Documentation	Structurée, maintenue	Minimale, « juste assez »

À RETENIR Quand choisir Itératif / Agile ?

- Itératif : Besoins évolutifs mais globalement connus, architecture complexe à valider tôt, plusieurs livraisons possibles.
- Agile : Besoins flous ou changeants, client très disponible et engagé, équipe autonome, besoin de valeur métier rapide (Time-to-market).

13.6 Choisir et justifier une démarche de développement

Il n'existe pas de « meilleure » méthode universelle. Le choix est une **décision d'ingénierie** fondée sur le contexte du projet.

MÉTHODE Comment choisir son approche ?

1. Analyser la stabilité des besoins : Sont-ils figés (cascade/V) ou changeants (itératif/agile) ?

2. Évaluer la disponibilité du client : Peu dispo (cascade/V) vs très dispo (agile).
3. Identifier les contraintes de délai/budget : Date fixe + budget verrouillé → cascade/V. Délai court pour une première valeur → agile.
4. Mesurer l'importance de la fiabilité : Critique (santé, finance, paiements) → cycle en V (traçabilité tests). Standard → cascade ou agile.
5. Rédiger la justification : « J'ai choisi X car [argument 1 issu du contexte] et [argument 2 issu du contexte]. »

Approche	Principe clé	Contexte favorable (Critères de choix)
Cascade	Étapes successives, sans retour arrière	Besoins clairs/stables, client peu dispo, délai/budget fixes, 1 seule livraison
Cycle en V	Descente specs / Remontée tests (V&V)	Fiabilité cruciale, besoins stables, traçabilité exigée
Itératif	Versions successives enrichies	Besoins évolutifs, architecture à valider, plusieurs livraisons acceptées
Agile	Sprints courts, client immergé	Besoins flous/changeants, client très dispo, valeur rapide, équipe autonome

13.7 Activité d'intégration : La cantine du collège de Bafoussam

13.8 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- identifier les besoins d'un utilisateur et les formuler sous la forme d'un **mini cahier des charges** ;
- ordonner et décrire les **six étapes du cycle de vie d'un logiciel** (analyse des besoins, conception, programmation, tests, mise en service, maintenance) appliquées à un projet concret ;
- choisir une **approche de développement** (cascade, cycle en V, itératif, agile) adaptée à une situation donnée ;
- **rédiger et justifier une démarche** par des arguments précis, liés au contexte du projet ;
- présenter oralement et comparer plusieurs démarches pour construire une réponse de référence.

Cette activité mobilise donc à la fois les savoirs de la leçon (notions de base sur le cycle de développement des logiciels) et les savoir-faire attendus au BEPC : appliquer les notions de l'unité à une situation concrète de la vie courante et rédiger une démarche justifiée.

13.9 Situation

La cantine du collège de Bafoussam.

Le collège bilingue de Bafoussam, dans la région de l'Ouest, compte 1200 élèves. Chaque midi, environ 450 élèves déjeunent à la cantine scolaire. Jusqu'à présent, tout se fait à la main : l'intendant inscrit les noms des élèves abonnés dans un grand cahier, encaisse les paiements en espèces (le repas coûte 300 FCFA, le ticket mensuel 6000 FCFA), et coche chaque jour les présences. Les problèmes s'accumulent :

- le cahier des abonnés est parfois égaré ou taché, et des noms disparaissent ;
- des parents paient mais l'élève n'est pas retrouvé sur la liste, ce qui provoque des disputes ;
- l'intendant passe deux heures chaque soir à compter l'argent et à vérifier les listes ;
- en fin de trimestre, le proviseur ne sait jamais exactement combien de repas ont été servis ni combien d'argent a été encaissé.

Le proviseur, M. Kamdem, décide donc de faire développer un petit logiciel de gestion de la cantine. Il contacte un jeune informaticien de la ville et lui pose ses conditions :

- le logiciel doit être **prêt avant la fin du trimestre**, soit dans **trois mois** ;
- le proviseur et l'intendant sont **très occupés** : ils ne pourront rencontrer le développeur que **deux ou trois fois** au maximum pendant le projet ;
- les besoins sont **clairs et stables** : on sait exactement ce que le logiciel doit faire, et cela ne changera pas d'ici la fin de l'année scolaire ;
- le budget est limité : le collège ne peut payer qu'**une seule version** du logiciel, pas plusieurs essais successifs.

Le développeur hésite : quelle approche de développement choisir ? Le cycle en cascade ? Le cycle en V ? Une approche itérative ou agile ? C'est à ta classe de l'aider à décider.

13.10 Consignes

Par groupes de 4 élèves, réponds aux tâches suivantes, dans l'ordre. Chaque production sera rédigée proprement sur une fiche, car elle sera présentée à la classe.

1. **Mini cahier des charges.** Rédige un mini cahier des charges du logiciel comportant **au maximum 5 besoins**. Pour chaque besoin, précise : qui utilise le logiciel, quelle fonction il doit offrir, et quelle contrainte il doit respecter (par exemple : rapidité, sécurité des paiements, simplicité pour l'intendant).
2. **Cycle de vie du projet.** Sur une fiche, écris dans le bon ordre les **6 étapes du cycle de vie d'un logiciel** appliquées à ce projet, et accompagne chaque étape d'**une phrase** expliquant concrètement ce que le développeur y ferait pour la cantine (par exemple, à l'étape des tests : « vérifier qu'un paiement de 6000 FCFA est bien enregistré au bon nom »).
3. **Choix de l'approche.** Choisis, parmi les quatre approches étudiées (cascade, cycle en V, itératif, agile), celle qui te semble la **mieux adaptée** à ce projet. Un seul choix par groupe.

4. **Justification rédigée.** Rédige un paragraphe de **8 à 10 lignes** justifiant ton choix par **au moins deux arguments** tirés de la situation : besoins stables ou évolutifs, délai d'un trimestre, disponibilité limitée du proviseur, budget ne permettant qu'une seule version.
5. **Présentation orale.** Chaque groupe présente sa démarche en **2 minutes** : cahier des charges, fiche des étapes, choix et justification. La classe compare ensuite les choix des différents groupes et construit ensemble, avec l'enseignant, la **démarche de référence**.

ATTENTION Pièges à éviter

- Ne confonds pas les étapes : la conception (on dessine et on planifie) n'est pas la programmation (on écrit le code).
- Un argument comme « c'est plus facile » n'est pas une justification : appuie-toi toujours sur une donnée de la situation (délai, budget, disponibilité, stabilité des besoins).
- Il n'y a pas une seule bonne réponse possible : c'est la qualité de la justification qui est évaluée, pas seulement le nom de l'approche choisie.

13.11 Démarche et corrigé commenté**EXERCICE RÉSOLU 9** Corrigé commenté de l'activité

Énoncé. Concevoir la démarche de développement de l'application de gestion de la cantine du collège de Bafoussam : mini cahier des charges, cycle de vie appliqué, choix d'une approche et justification rédigée.

SOLUTION

Étape 1 : le mini cahier des charges (5 besoins).

1. Inscription des abonnés : l'intendant enregistre chaque élève abonné (nom, classe, type de ticket) ; contrainte : recherche d'un élève en moins de 10 secondes.
2. Paiements : l'intendant enregistre chaque paiement (montant, date, reçu) ; contrainte : aucun paiement ne doit être perdu, même en cas de coupure de courant.
3. Pointage quotidien : le surveillant de cantine coche les élèves présents chaque midi ; contrainte : manipulation très simple, sans formation informatique.
4. État des comptes : le logiciel signale les élèves dont le ticket est épuisé ou impayé ; contrainte : mise à jour automatique après chaque repas.
5. Bilan trimestriel : le proviseur obtient le nombre de repas servis et le total encaissé ; contrainte : bilan imprimable pour le conseil d'établissement.

Commentaire : chaque besoin précise bien qui (intendant, surveillant, proviseur), quoi (la fonction) et avec quelle contrainte. C'est exactement ce qu'on attend d'un cahier des charges simplifié.

Étape 2 : les 6 étapes du cycle de vie appliquées au projet.

1. Analyse des besoins : le développeur rencontre le proviseur et l'intendant, observe le cahier actuel et rédige le cahier des charges.
2. Conception : il dessine les écrans (liste des élèves, page de paiement, bilan) et organise les données (fichier des élèves, fichier des paiements).
3. Programmation : il écrit le code du logiciel, écran par écran, fonction par fonction.
4. Tests : il vérifie chaque fonction avec des cas réels, par exemple : inscrire un élève de 3^e C, payer 6000 FCFA, pointer 3 repas, puis contrôler que le solde restant est correct.
5. Mise en service : il installe le logiciel sur l'ordinateur de l'intendance, forme l'intendant pendant une journée et saisit les 450 abonnés.
6. Maintenance : il corrige les pannes signalées (par exemple un reçu mal imprimé) et ajoute, l'année suivante, la gestion des demi-pensions.

Étape 3 : choix de l'approche.

Nous choisissons le **cycle en cascade** (le cycle en V est également défendable : voir le bilan).

Étape 4 : justification rédigée (8 à 10 lignes).

« Nous avons choisi le cycle en cascade pour deux raisons principales. D'abord, les besoins du collège sont clairs et stables : le proviseur sait exactement ce qu'il veut (inscriptions, paiements, pointage, bilan) et ces besoins ne changeront pas pendant l'année scolaire ; il est donc inutile de prévoir plusieurs versions successives comme dans une approche itérative. Ensuite, le proviseur et l'intendant sont très occupés et ne pourront rencontrer le développeur que deux ou trois fois : or l'approche agile exige une collaboration permanente avec le client, ce qui est impossible ici. De plus, le délai d'un trimestre et le budget limité à une seule version conviennent bien à la cascade, où tout est planifié dès le départ puis réalisé étape par étape. Enfin, une phase de tests sérieuse avant la mise en service garantira que les paiements des parents sont correctement enregistrés. »

Commentaire : le paragraphe cite trois arguments (besoins stables, client peu disponible, délai et budget) et chacun est rattaché à une donnée précise de la situation. C'est cela, justifier une démarche.

13.12 Bilan de l'activité

Cette activité t'a permis de mettre en évidence plusieurs idées essentielles de la leçon :

- Un logiciel ne se fabrique pas au hasard : il suit toujours un **cycle de vie** en six étapes, de l'analyse des besoins jusqu'à la maintenance.

- Le **cahier des charges** est le point de départ de tout projet : c'est lui qui traduit les besoins des utilisateurs (ici le proviseur, l'intendant, le surveillant) en fonctions précises.
- Il n'existe pas « la meilleure » approche de développement en absolu : le bon choix dépend du **contexte** (stabilité des besoins, délai, budget, disponibilité du client).
- Dans notre situation, la **cascade** et le **cycle en V** sont tous deux défendables : la cascade parce que les besoins sont stables et le client peu disponible, le cycle en V parce que la fiabilité des paiements exige des tests rigoureux. En revanche, l'approche **agile** est mal adaptée ici, car elle suppose un client disponible en permanence.
- Enfin, tu as pratiqué un savoir-faire attendu au BEPC : **rédiger et justifier une démarche** en appuyant chaque argument sur une donnée concrète du problème, puis défendre ton point de vue à l'oral et accepter de le comparer à celui des autres groupes.

À RETENIR**L'essentiel**

Pour choisir une approche de développement, pose-toi toujours quatre questions : les besoins sont-ils stables ? Quel est le délai ? Le client est-il disponible ? Quelle importance accorder aux tests ? Ta réponse doit ensuite être justifiée par des arguments tirés de la situation, jamais par une simple préférence.

14 MÉTHODES ET EXERCICES RÉSOLUS

15 INTRODUCTION AUX APPROCHES DE DÉVELOPPEMENT LOGICIEL

15.1 Logiciel et cycle de développement : premières définitions

DÉFINITION Logiciel

Un **logiciel** est un ensemble de programmes, de procédures, de règles et de documentation associés au fonctionnement d'un système de traitement de l'information. Contrairement au matériel (hardware), il est immatériel.

DÉFINITION Cycle de vie d'un logiciel (Cycle de développement)

Le **cycle de vie d'un logiciel** (ou cycle de développement) décrit la succession des étapes traversées par un logiciel, depuis l'idée initiale jusqu'à son retrait définitif. Il structure le travail de l'équipe de développement pour produire un logiciel de qualité, dans les délais et le budget.

À RETENIR Les grandes phases universelles

Quel que soit l'approche choisie, tout développement logiciel passe par ces activités fondamentales :

1. Étude préliminaire / Faisabilité : Le projet est-il possible ? (Technique, budget, délais).
2. Analyse des besoins : Que doit faire le logiciel ? (Recueil auprès du client, cahier des charges).
3. Conception : Comment le logiciel va-t-il le faire ? (Architecture, algorithmes, base de données, interfaces).
4. Implémentation (Codage) : Écriture du code source dans un langage de programmation (C, Python, HTML/JS...).
5. Tests : Vérification que le logiciel fonctionne correctement (tests unitaires, tests d'intégration, tests utilisateur).
6. Déploiement (Mise en production) : Installation chez le client, formation, démarrage.

7. Maintenance : Corrections, adaptations, améliorations après la livraison.

15.2 Les étapes du cycle de vie d'un logiciel

EXEMPLE Exemple concret : Gestion d'une bibliothèque scolaire

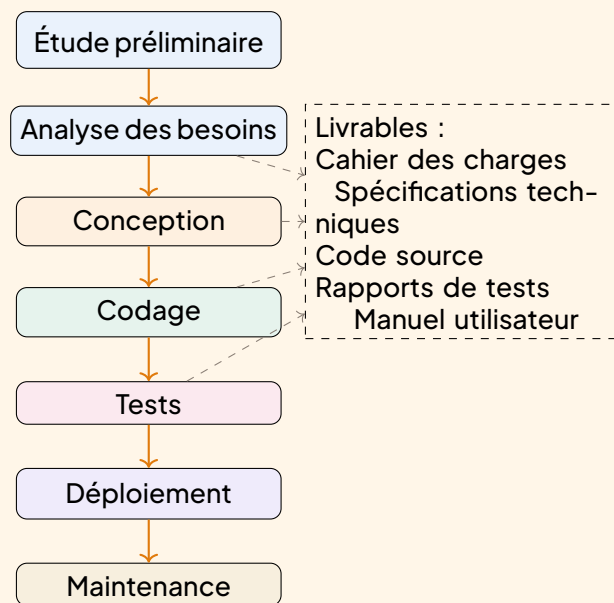
Imaginons le développement d'un logiciel pour la bibliothèque du lycée de Ngaoundéré.

- Faisabilité : Le proviseur valide le budget (500 000 FCFA) et le délai (3 mois). L'équipe (2 élèves en stage + 1 prof) a les compétences (Python, SQLite).
- Besoins : Le bibliothécaire veut : enregistrer les livres, gérer les emprunts/retours, alerter les retards, imprimer la liste des livres perdus.
- Conception : Choix d'une base de données SQLite (tables `Livre`, `Eleve`, `Emprunt`). Algorithme de calcul de la date de retour (14 jours). Interface graphique simple (Tkinter).
- Codage : Écriture des fonctions `ajouter_livre()`, `emprunter()`, `verifier_retards()`.
- Tests : On simule 50 emprunts. On vérifie qu'un élève ne peut pas emprunter 3 fois le même livre. On corrige un bug sur le calcul de date (année bissextile).
- Déploiement : Installation sur le PC de la bibliothèque. Formation du bibliothécaire (1 heure).
- Maintenance : Mois 1 : correction bug affichage accents. Mois 6 : ajout champ "ISBN" (évolutive). An 2 : migration vers Windows 11 (adaptative).

15.3 L'approche en cascade (cycle en cascade)

PROPRIÉTÉ Modèle en cascade (Waterfall)

Le modèle en **cascade** est une approche séquentielle et linéaire. Les phases s'enchaînent strictement : la phase $n+1$ ne démarre qu'après la validation formelle (signature, revue) de la phase n . Il n'y a pas de retour en arrière prévu dans le modèle pur.



Modèle en cascade : enchaînement strict des phases.

À RETENIR Avantages et limites de la cascade

- Avantages : Simple à comprendre, à gérer, à planifier. Documentation complète imposée. Adapté aux projets courts, besoins figés et stables, technologie maîtrisée, contrainte réglementaire forte.
- Limites : Rigide. Le client ne voit le logiciel qu'à la fin (effet tunnel). Coût élevé en cas de changement de besoins découverts tardivement. Risques techniques découverts tard (phase test).

MÉTHODE Identifier les étapes du cycle de vie (Modèle en cascade)

Objectif : Ordonner chronologiquement les phases fondamentales et associer livrables et acteurs.

1. Lister les phases canoniques dans l'ordre : Étude préliminaire → Analyse des besoins → Conception → Codage → Tests → Déploiement → Maintenance.
2. Vérifier la séquentialité : Aucune phase ne commence avant la validation de la précédente.
3. Associer livrables et acteurs :

Phase	Livrable principal	Acteurs clés
Étude préliminaire	Rapport de faisabilité	Chef de projet, Commanditaire
Analyse des besoins	Cahier des charges (CDC)	Analyste, Client/Utilisateur
Conception	Spécifications techniques (MCD, Algos, IHM)	Concepteur, Architecte
Codage	Code source	Développeur
Tests	Rapports de tests (unitaires, intégration, acceptation)	Testeur, Utilisateur (recette)
Déploiement	Logiciel installé + Manuel utilisateur	Admin système, Formateur
Maintenance	Correctifs, Nouvelles versions	Équipe support, Développeur

EXERCICE RÉSOLU 10 Ordonnancement des phases pour un cybercafé à Yaoundé

Énoncé : Le gérant d'un cybercafé veut un logiciel pour gérer les postes (temps, impression, facturation). L'équipe choisit le modèle en cascade. Numérotez de 1 à 7 les tâches ci-dessous dans l'ordre strict :

- A. Écriture du code C du module facturation.
- B. Réunion avec le gérant pour lister tarifs, abonnements, règles de blocage.
- C. Installation sur le serveur et formation du gérant.
- D. Correction d'un bug d'affichage signalé 2 semaines après mise en service.
- E. Élaboration du schéma de la base de données (tables Poste, Client, Session, Tarif) et algorithmes de calcul.
- F. Vérification que le logiciel bloque l'accès internet quand le crédit est épuisé (test avec le gérant).
- G. Étude de faisabilité : coût matériel, compétences équipe, délai 3 mois.

SOLUTION

Solution détaillée :

1. 1. Étude préliminaire (Faisabilité) : G. On évalue la viabilité avant tout engagement.
2. 2. Analyse des besoins : B. Recueil des règles de gestion auprès du gérant. Production du CDC.
3. 3. Conception : E. Architecture technique : MCD, algorithmes, IHM. Spécifications

techniques.

4. 4. Codage (Implémentation) : A. Traduction des algorithmes en C.
 5. 5. Tests : F. Exécution scénarios (unitaires, intégration, acceptation). Ici test d'acceptation métier.
 6. 6. Déploiement : C. Mise en production, installation, formation, démarrage assisté.
 7. 7. Maintenance : D. Intervention post-livraison. Maintenance corrective (bug).
- Ordre final : $G \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow D$.

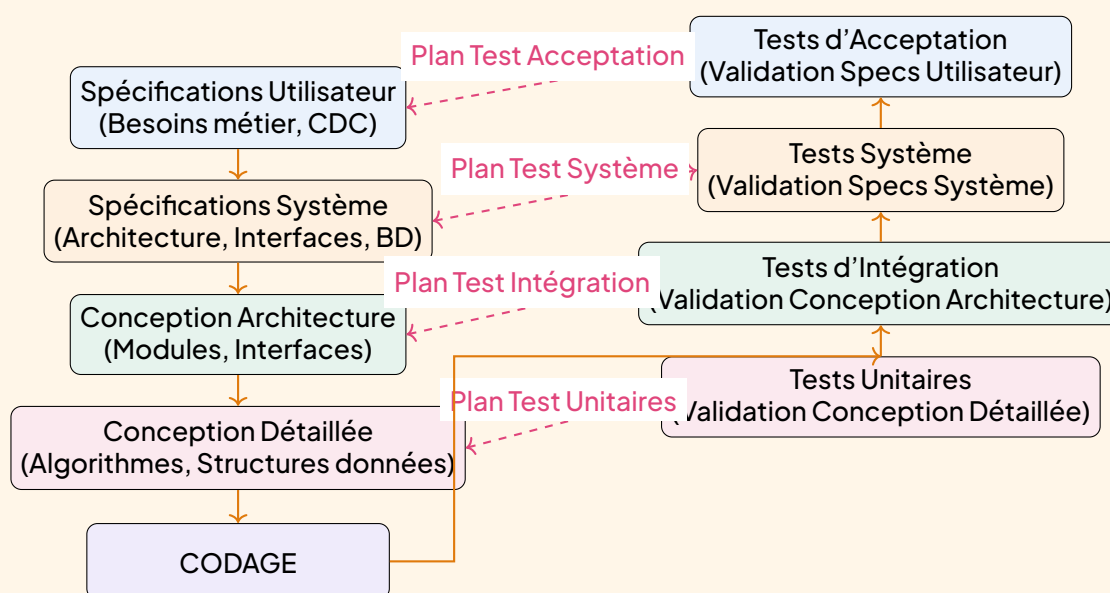
15.4 Le cycle en V

PROPRIÉTÉ Modèle en V

Le **cycle en V** est une variante du modèle en cascade qui met l'accent sur la validation et la traçabilité. Il se représente par un V :

- Branche descendante (Définition) : Spécifications Utilisateur → Spécifications Système → Conception Architecture → Conception Détaillée.
- Branche montante (Validation) : Tests Unitaires → Tests d'Intégration → Tests Système → Tests d'Acceptation (Recette).

Principe clé : Chaque niveau de définition a son niveau de validation symétrique (flèches horizontales). Les plans de tests sont rédigés en parallèle des spécifications (pas après le code).



Cycle en V : symétrie définition/validation et rédaction anticipée des plans de test.

MÉTHODE Appliquer le cycle en V : correspondances validation/spécification

Objectif : Faire correspondre chaque document de définition à son activité de validation.

1. Spécifications Utilisateur (CDC, besoins métier) ↔ Tests d'Acceptation (Recette utilisateur) : Le client valide : "Le logiciel fait-il ce que j'ai demandé?"
2. Spécifications Système (Architecture, API, SGBD) ↔ Tests Système : L'équipe technique valide : "Le système respecte-t-il les specs techniques globales (perf, sécurité, interfaces)?"
3. Conception Architecture (Modules, Interfaces) ↔ Tests d'Intégration : On vérifie : "Les modules communiquent-ils correctement via les interfaces définies?"
4. Conception Détaillée (Algorithmes, Fonctions) ↔ Tests Unitaires : On teste chaque fonction isolée : "L'algorithme de tri/calcul fonctionne-t-il pour tous les cas (limites, erreurs)?"

EXERCICE RÉSOLU 11 Correspondance pour une application de bulletins scolaires

Énoncé : Application de notes/bulletins pour un collège à Douala. Cycle en V imposé. Associez définition (1-4) et validation (A-D).

Branche descendante (Définition)	Branche montante (Validation)
1. Expression besoins : « Bulletin officiel MINESEC avec moyennes pondérées, rangs, signature numérique. »	A. Tests sur fonction <code>calculerMoyenne(notes[], coeffs[])</code> avec cas limites (coeff 0, note > 20).
2. Specs techniques : « Architecture 3 couches (Web, Java, PostgreSQL). API REST import notes. »	B. Simulation saisie 60 élèves, génération PDF, vérif mise en page et calculs par proviseur adjoint.
3. Conception détaillée module « Calculs » : « Fonction <code>rangClasse()</code> : tri sélection décroissante, gestion ex-aequo. »	C. Test API REST : envoi JSON, vérif insertion base, intégrité référentielle.
4. Conception architecture : « Module « Édition PDF » indépendant, appelle service « Calculs » via interface <code>IBulletinService</code> . »	D. Vérification tri sélection : bon rang pour 1, 5, 60 élèves, avec doublons.

SOLUTION

Solution : On applique la symétrie horizontale (haut niveau ↔ haut niveau, bas niveau ↔ bas niveau).

- 1 ↔ B : Besoins utilisateur (métier) → Tests d'Acceptation. Le proviseur adjoint (utilisateur) valide le PDF final.
- 2 ↔ C : Specs techniques (architecture, API, BD) → Tests Système. Test du système intégré (API, BD, enchaînement global).
- 3 ↔ D : Conception détaillée (algo précis, fonction `rangClasse`) → Tests Unitaires. Test fonction isolée, cas nominaux/limites/ex-aequo.
- 4 ↔ A : Conception architecture (modules, interface `IBulletinService`) → Tests d'Intégration. Vérif appel module PDF → service Calculs via l'interface.

Rappel : Les plans de test B, C, D, A sont rédigés en même temps que les docs 1, 2, 3, 4.

15.5 Approches itératives et agiles

DÉFINITION Approche itérative et incrémentale

Contrairement à la cascade (une seule livraison finale), l'approche **itérative** découpe le projet en cycles courts (itérations) de durée fixe (ex : 2 à 4 semaines). Chaque itération produit un **incrément** : une version partielle mais fonctionnelle, testée et intégrable. On affine le produit par boucles successives.

À RETENIR Différences clés : Cascade vs Itératif/Agile

Critère	Cascade (Séquentielle)	Itératif / Agile (ex : Scrum)
Livraison valeur métier	Une seule à la fin (Big Bang)	Progressive (chaque itération/Sprint)
Gestion des changements	Coûteux, formel (avenant contrat)	Naturel, bienvenu (Backlog dynamique)
Découverte des risques	Tardive (phase tests)	Précoce (dès 1ère itération)
Retour utilisateur (Feedback)	Final (recette)	Continu (démonstration fin de sprint)
Visibilité avancement	Documents (specs, % code)	Logiciel fonctionnel (Démonstration)
Adapté pour	Besoins stables, technologie connue, réglementation forte	Besoins flous/évolutifs, innovation, time-to-market court

LESAVAIS-TU ? Le Manifeste Agile (2001) – Simplifié pour le BEPC

L'agilité repose sur 4 valeurs fondamentales :

1. Individus et interactions plus que processus et outils.
2. Logiciel opérationnel plus que documentation exhaustive.
3. Collaboration client plus que négociation contractuelle.
4. Adaptation au changement plus que suivi d'un plan.

Repères Scrum (cadre le plus connu) :

- Rôles : Product Owner (représente le client), Scrum Master (facilitateur), Équipe Dev (auto-organisée).
- Artefacts : Product Backlog (liste priorisée des besoins), Sprint Backlog (sélection pour le sprint), Incrément (logiciel fini).
- Cérémonies : Sprint Planning (planif), Daily Stand-up (synchro 15min quotidien), Sprint Review (démon client), Sprint Rétrospective (amélioration processus).

EXERCICE RÉSOLU 12 Choix de l'approche : Application mobile « Emploi Jeunes CM »

Énoncé : Startup camerounaise, appli mobile (offres emploi, CV vidéo, alertes SMS). Marché concurrentiel, besoins évoluent vite (nouvelles fonctions/mois), équipe 5 devs juniors, time-to-market critique (MVP 2 mois). Choix : Cascade vs Itératif/Agile.

1. 3 arguments techniques pour rejeter la cascade pure.
2. Organisation de la 1ère itération (2 semaines) pour livrer un MVP.

SOLUTION

Solution :

1. Arguments contre la Cascade :
 - (a) Besoins instables : "Besoins évoluent vite". Cascade fige le CDC au départ. Tout changement = avenant long/coûteux, incompatible avec adaptation mensuelle.
 - (b) Risque Time-to-market : "MVP en 2 mois". Cascade livre valeur qu'à la fin (6-12 mois). Concurrent sort fonction clé avant = échec commercial. Itératif livre MVP utilisable dès Sprint 1.
 - (c) Risque technique/équipe : Équipe junior, techno mobile nouvelle (Flutter/React Native). Cascade valide architecture sur papier. Itératif valide architecture par le code (Proof of Concept) dès Sprint 1, évitant erreur majeure découverte 6 mois plus tard.
2. Sprint 1 (2 semaines) – Objectif MVP : « Candidature simple à une offre »

- Objectif (Sprint Goal) : Jeune crée compte, consulte offres (mockées), postule avec CV texte, reçoit confirmation SMS.
- Périmètre (User Stories priorisées) :
 - (a) US1 : Inscription/Connexion (Auth simple, stockage local).
 - (b) US2 : Affichage liste offres (Appel API mock, liste simple).
 - (c) US3 : Détail offre + Bouton « Postuler » (Envoi données + CV texte).
 - (d) US4 : Notification SMS confirmation (Intégration API SMS sandbox Orange/MTN).
- Livrable (Incrément) : APK installable sur téléphone test, démo bout-en-bout (Inscription → Postulation → SMS reçu). Code versionné (Git), tests unitaires critiques passés.

Justification : Valide architecture (Client ↔ API ↔ SMS), donne visibilité, feedback réel UX/réseau 3G/4G pour Sprint 2.

15.6 Choisir et justifier une démarche de développement

MÉTHODE Structurer une justification de choix méthodologique (Modèle examen BEPC)

Objectif : Rédiger un argumentaire clair, noté sur la structure et la pertinence.

1. Analyser le contexte (Grille de lecture) : Stabilité besoins ? Criticité/Réglementation ? Taille équipe/Durée ? Technologie connue ? Visibilité client souhaitée ?
2. Rédiger en 4 étapes (Structure « CQQCOQP » adaptée) :
 - (a) Thèse / Choix : « Au regard du contexte [résumé], l'approche [Nom] est la plus adaptée. »
 - (b) Argument 1 (Contrainte bloquante) : « En premier lieu, [Contrainte clé] impose [Caractéristique méthode] car [Raison technique]. »
 - (c) Argument 2 (Risque/Équipe) : « Par ailleurs, [Contrainte équipe/technique] trouve réponse dans [Pratique méthode] permettant [Bénéfice]. »
 - (d) Argument 3 (Valeur client/Délai) : « Enfin, [Attente client] est satisfaite par [Mécanisme méthode] garantissant [Résultat]. »
 - (e) (Optionnel) Condition de succès : « Cela suppose toutefois [PO dispo, équipe stable, outillage...]. »
 - (f) Conclusion : « En synthèse, [Méthode] offre le meilleur compromis entre [Objectif 1] et [Objectif 2]. »

3. Vocabulaire de liaison : En effet, Par conséquent, En outre, Néanmoins, Ainsi, Ce qui permet de...

EXERCICE RÉSOLU 13 Justification courte : Site e-commerce « Made in Cameroon » (Foumban)

Énoncé : Coopérative artisans (sculpteurs, tisserands) vente en ligne. Budget serré (ONG, 6 mois max). Équipe : 2 juniors + 1 senior. Besoins : catalogue, panier, paiement Mobile Money (MTN MoMo/Orange Money), livraison. Artisans gèrent produits (back-office simple). Marché évolue vite (fêtes, saisons). Justifier Agile/Scrum (150 mots max).

SOLUTION

Proposition de rédaction :

Au regard du contexte (budget serré, délai court 6 mois, besoins évolutifs saisonniers, équipe petite avec juniorité), l'approche Agile (Scrum) est la plus adaptée.

En premier lieu, l'instabilité des besoins (nouveaux produits, promotions fêtes, évolution back-office artisans) impose une gestion dynamique du périmètre. Le Product Backlog priorisé par le Product Owner (représentant coopérative) permet d'intégrer ces changements à chaque Sprint sans avenant contractuel coûteux, contrairement au modèle en cascade.

Par ailleurs, la contrainte de délai court et de montée en compétence des juniors trouve réponse dans les Sprints courts (2 semaines) et l'intégration continue. Cela force une livraison précoce d'un MVP (Catalogue + Panier Sprint 1-2), valide l'architecture technique (API Mobile Money) dès le départ, et permet au senior d'encadrer via les revues de code (Definition of Done) plutôt que par une spécification figée.

Enfin, la nécessité de valider l'ergonomie back-office avec les artisans (utilisateurs non techniques) est satisfaite par les Sprint Reviews régulières. Le logiciel opérationnel (Valeur Agile #2) remplace la documentation abstraite pour la validation.

Cela suppose toutefois la disponibilité continue d'un Product Owner légitime au sein de la coopérative. En synthèse, Scrum offre le meilleur compromis entre time-to-market, maîtrise des coûts et adaptation métier pour ce projet.

ATTENTION Les 5 erreurs à éviter pour le BEPC

1. Confondre « Itératif » et « Incrémental ». Itératif = on affine le tout (prototype → V1 → V2 complète). Incrémental = on construit par morceaux qui s'ajoutent (Module A → A+B → A+B+C). Agile/Scrum combine les deux. Ne pas écrire « L'itératif livre des morceaux ». Dire : « L'agile combine itérations (cycles courts) et incréments (livraisons partielles fonctionnelles). »

2. Oublier la rédaction anticipée des plans de test dans le Cycle en V. Dire « On fait les tests après le codage » est une erreur majeure pour le V. Règle : Plan Test Acceptation pendant Specs Utilisateur, Plan Test Système pendant Specs Système, Plan Test Unitaires pendant Conception Détaillée.
3. Justifier par des arguments génériques non liés au sujet. Ex : « Agile car c'est moderne/flexible » → 0 point. Obligatoire : Contexte → Contrainte → Caractéristique Méthode → Bénéfice. Ex : « Besoins changeants → Backlog dynamique → Adaptation sans coûts excessifs. »
4. Confondre « Recette » et « Tests Système ». Recette = faite par/avec le client (PO) sur pré-prod, données réelles, pour dire « Je valide/Je paie ». Tests Système = faits par l'équipe technique pour vérifier conformité specs techniques (perf, sécurité, IHM). Acteurs et critères différents.
5. Négliger la Maintenance. Le cycle ne s'arrête pas au déploiement. Citer les 3 types : Corrective (bugs), Adaptative (maj OS/navigateur/API), Évolutive (nouvelles fonctions). Rappel : maintenance évolutive = souvent 60–80% du coût total (Loi de Lehman).

EXERCICE 7 Entraînement : Choix d'approche pour la gestion d'un cybercafé

Contexte : Le gérant du cybercafé « Connexion Plus » à Bafoussam veut moderniser son système. Il a un cahier des charges très précis et stable (règles de facturation fixes, impression tickets, gestion abonnements mensuels) validé depuis 6 mois. Il a un budget fixe (1 000 000 FCFA) et une date d'ouverture impérative dans 4 mois. L'équipe de développement (3 devs expérimentés en C/SQLite) maîtrise parfaitement la technologie. Le gérant ne sera disponible que pour la validation finale (recette).

1. Quelle approche recommandez-vous : Cascade, Cycle en V, ou Agile (Scrum)?
2. Justifiez votre choix en utilisant la structure en 4 arguments de la méthode ci-dessus (Thèse, Arg 1, Arg 2, Arg 3, Conclusion).

CORRIGÉ Exercice : Cybercafé « Connexion Plus »

1. Choix : Cycle en V (ou Cascade pur, mais V préféré pour la traçabilité tests). 2. Justification structurée : Thèse : Au regard du contexte (besoins stables/figés, budget/délai fixes, équipe expérimentée, technologie maîtrisée, client peu disponible), le Cycle en V est la démarche la plus adaptée.

Argument 1 (Stabilité besoins / Réglementation implicite) : En premier lieu, la stabilité totale des besoins (cahier des charges précis validé depuis 6 mois, règles de gestion fixes) impose une approche séquentielle où les spécifications sont figées dès le départ. Le

Cycle en V, par sa branche descendante rigoureuse (CDC → Specs Tech → Conception), garantit que le code produit correspond exactement aux exigences validées, sans dérive de périmètre (scope creep) possible en Agile.

Argument 2 (Équipe / Technologie / Contractuel) : Par ailleurs, l'expertise de l'équipe sur la stack technique (C/SQLite) et la contrainte contractuelle forte (budget/délai fixes, client absent) trouvent réponse dans la planification détaillée et les jalons de validation formelle du Cycle en V. Les spécifications techniques détaillées servent de référentiel stable pour les 3 développeurs (parallélisation du travail par modules) et de base contractuelle pour la recette finale, ce qu'un Product Backlog dynamique ne permet pas.

Argument 3 (Criticité / Qualité) : Enfin, la criticité métier (facturation, argent) est satisfaite par la traçabilité exigences-tests propre au V. La rédaction anticipée des plans de test (acceptation, système, unitaires) en parallèle des specs assure une couverture de test rigoureuse, réduisant le risque de bugs financiers en production, là où l'Agile pourrait privilégier la vélocité sur l'exhaustivité des tests formels.

Conclusion : En synthèse, le Cycle en V offre le meilleur compromis entre maîtrise des coûts/délais (planification fixe), garantie de conformité (traçabilité V) et qualité (tests précoces) pour ce projet à spécifications stables et contrainte contractuelle forte.

16 EXERCICES

17 INTRODUCTION AUX APPROCHES DE DÉVELOPPEMENT LOGICIEL

17.1 Logiciel et cycle de développement : premières définitions

DÉFINITION Logiciel

Un **logiciel** est un ensemble de programmes, de procédures, de règles et de documentation associés au fonctionnement d'un système de traitement de l'information. Contrairement au matériel (hardware), il est immatériel. On distingue généralement les logiciels système (OS, pilotes), les logiciels applicatifs (traitement de texte, gestion de notes) et les logiciels de programmation (compilateurs, EDI).

DÉFINITION Cycle de vie d'un logiciel (Cycle de développement)

Le **cycle de vie d'un logiciel** (ou **cycle de développement**) désigne l'ensemble structuré des étapes traversées par un logiciel, depuis la naissance de l'idée initiale (expression du besoin) jusqu'à son retrait définitif (désinstallation, archivage), en passant par sa conception, sa réalisation, son déploiement, son utilisation et sa maintenance. C'est un cadre méthodologique pour maîtriser la complexité, la qualité, les coûts et les délais.

À RETENIR Pourquoi modéliser le développement ?

Développer un logiciel sans méthode conduit souvent à : dépassement des délais et du budget, logiciel inadapté aux besoins réels, code ingérable ("spaghetti code"), maintenance impossible. Le cycle de vie impose une discipline d'ingénierie : on ne code pas avant d'avoir analysé et conçu.

LESAVAIS-TU ? Contexte camerounais

Au Cameroun, de nombreux projets informatiques (gestion de cybercafés, logiciels de paie pour la fonction publique, systèmes d'information scolaires comme SIGE ou GEP) souffrent encore d'un manque de rigueur méthodologique. L'adoption de cycles de vie

structurés (V, Agile) par les SSII (Sociétés de Services en Ingénierie Informatique) locales et les administrations (MINPOSTEL, MINESEC) est un enjeu majeur pour la souveraineté numérique et la qualité des services publics en ligne.

17.2 Les étapes du cycle de vie d'un logiciel (Modèle classique)

Quelle que soit l'approche choisie (cascade, V, agile), six activités fondamentales existent. Dans les approches séquentielles, elles forment des **phases** successives ; dans les approches itératives, elles se répètent à chaque itération.

PROPRIÉTÉ Les six étapes fondamentales

1. Analyse des besoins (Étude préliminaire) : Rencontre avec le client, compréhension du métier, étude de l'existant, faisabilité (technique, économique, juridique), rédaction du **cahier des charges** (ou spécifications fonctionnelles) validé par le client.
2. Conception (Spécifications techniques) : Translation du quoi (besoins) en comment (solution technique). Architecture logicielle (modulaire, couches), choix technologiques (langage, SGBD, framework), modélisation des données (MCD/MLD), algorithmes, maquettes d'interface (IHM). Livrable : Dossier de conception technique.
3. Programmation (Implémentation / Codage) : Écriture du code source dans le langage choisi (C, JavaScript, Python, HTML/CSS...), respect des normes de codage (indentation, commentaires, conventions de nommage), gestion de version (Git).
4. Tests (Validation et Vérification - V&V) : Vérification que le logiciel est bien construit (conformité à la conception) et validation qu'il répond aux besoins utilisateur. Niveaux : tests unitaires (modules isolés), tests d'intégration (interfaces entre modules), tests système (global), recette client (acceptation).
5. Déploiement (Installation / Mise en production) : Installation sur l'environnement cible (serveur, postes clients, store mobile), migration des données (si existant), formation des utilisateurs, documentation utilisateur/administrateur, bascule (cut-over).
6. Maintenance : Activités post-livraison. Quatre types : corrective (corriger des bugs), adaptative (adapter à un nouvel OS, une nouvelle loi, une nouvelle version de bibliothèque), évolutive (ajouter des fonctionnalités demandées), préventive (refactoring, optimisation, mise à jour de sécurité pour éviter de futures pannes).

EXEMPLE Exemple : Gestion de notes au Collège de Ngaoundéré

- Analyse : Le proviseur exprime le besoin : « Je veux éditer les bulletins officiels MINESEC automatiquement ». Cahier des charges : gestion élèves, matières, coefficients,

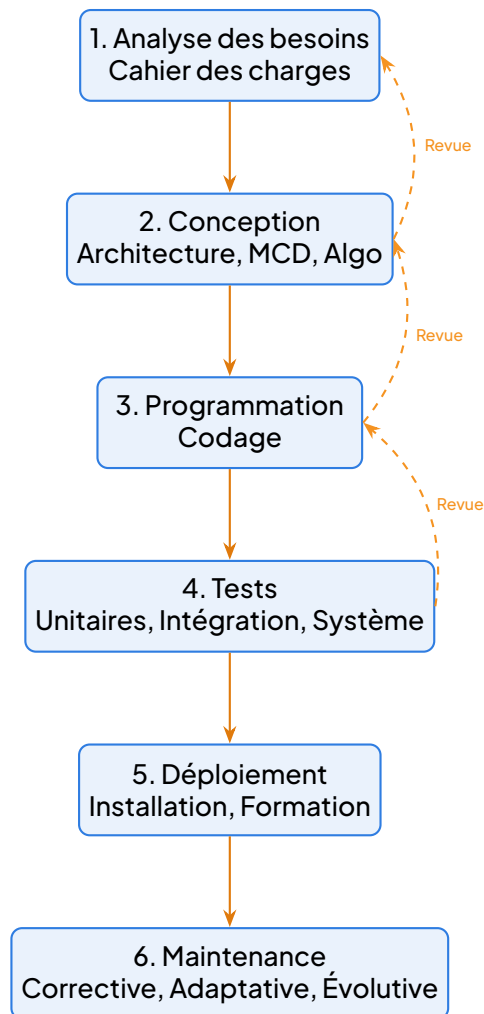
notes, moyennes, rangs, bulletins, PV.

- Conception : MCD (Eleve, Matiere, Note, Trimestre), choix : Application Web (PHP/JS ou C/GTK), base MySQL, algorithme calcul moyenne pondérée.
- Programmation : Code C pour le module de calcul, HTML/JS pour l'interface saisie professeurs.
- Tests : Test unitaire `calculerMoyenne()`, test intégration « Saisie note → Calcul → Affichage bulletin », recette : le proviseur valide un bulletin type.
- Déploiement : Installation sur serveur LAN du collège, formation des professeurs principaux.
- Maintenance : Changement coefficient EPS (adaptative), ajout export Excel (évolutive), correction bug affichage accents (corrective).

17.3 L'approche en cascade (Waterfall)

DÉFINITION Cycle en cascade (Waterfall)

L'**approche en cascade** (modèle de Royce, 1970) organise les six étapes de manière linéaire et séquentielle. Chaque phase doit être totalement terminée et validée (revue, signature) avant de passer à la suivante. Le retour en arrière est coûteux, voire impossible (comme une cascade : l'eau ne remonte pas).



Modèle en cascade (Waterfall) : enchaînement séquentiel strict avec revues de validation (portes de qualité).

PROPRIÉTÉ Caractéristiques de l'approche en cascade

- Documentation exhaustive produite en amont (CDC, Dossier Conception).
- Jalons (milestones) figés : validation formelle à chaque passage de phase.
- Visibilité du planning et du budget dès le début (si besoins stables).
- Rigidité : difficile d'intégrer un changement de besoin en cours de route.
- Risque tardif : le client ne voit le logiciel fonctionnel qu'à la fin (phase test/déploiement). Erreurs de conception découvertes tard = coût exponentiel.

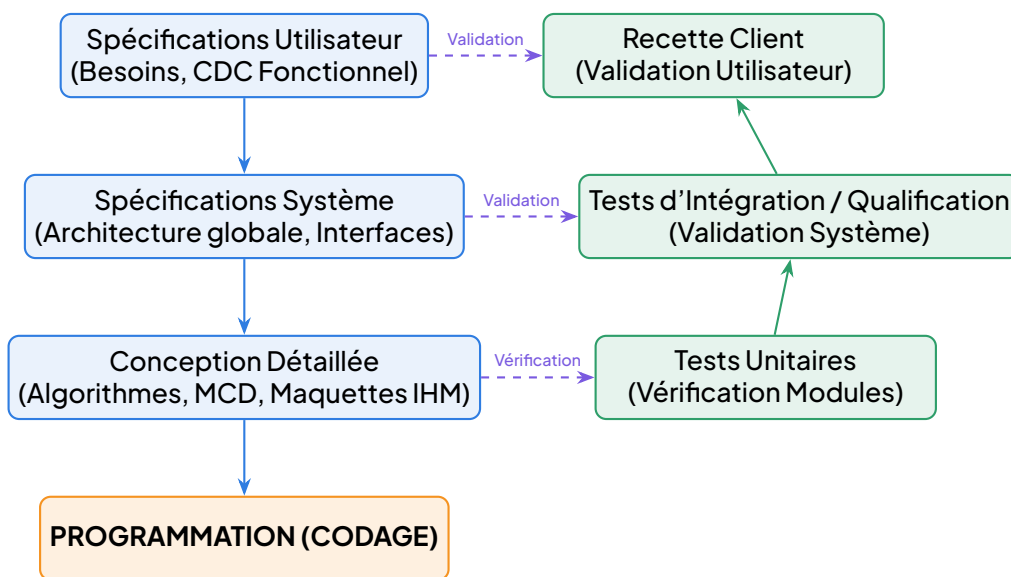
À RETENIR Quand choisir le cascade ?

Projet à besoins stables, clairs, complets et immuables (ex. logiciel de paie réglementaire, calculateur scientifique, système embarqué critique). Contraintes contractuelles fortes (marché public à prix/délai fixes). Équipe expérimentée sur le domaine.

17.4 Le cycle en V**DÉFINITION** Cycle en V (Modèle en V)

Le **cycle en V** (standard français NF X 06-051, utilisé en défense, aéronautique, administration) est une variante rigoureuse du cascade qui met l'accent sur la correspondance symétrique entre les phases de spécification (branche descendante : Quoi? Comment?) et les phases de validation/vérification (branche montante : Est-ce le bon produit? Est-ce le produit bien fait?). Le point bas du V est le codage.

BRANCHE DESCENDANTE : Spécification **BRANCHE MONTANTE : Validation / Vérification**



Le cycle en V : symétrie entre spécification (gauche) et validation/vérification (droite).

PROPRIÉTÉ Correspondance des niveaux (Règle fondamentale du V)

- Spécifications Utilisateur (Besoins) ↔ Recette Client (Validation d'acceptation) : « Le

logiciel fait-il ce que le client a demandé? » (Ex. : « Le bulletin est-il conforme au format MINESEC? »).

- Spécifications Système (Architecture) ↔ Tests d'Intégration / Qualification : « Les modules interagissent-ils correctement? Le système tient-il la charge? » (Ex. : « La saisie note → calcul → base de données fonctionne-t-elle? »).
- Conception Détaillée (Algo, MCD) ↔ Tests Unitaires : « Chaque fonction/classe respecte-t-elle son algorithme? » (Ex. : « `calculerMoyenne()` gère-t-elle les coefficients nuls? »).
- Codage : Point bas, vérifié par revues de code et analyse statique.

ATTENTION Vérification vs Validation (V&V)

Ne pas confondre :

- Vérification (Branche montante basse) : « Avons-nous bien construit le produit? » (Conformité à la conception technique). Ex. : Tests unitaires, revues de code.
- Validation (Branche montante haute) : « Avons-nous construit le bon produit? » (Conformité aux besoins utilisateur). Ex. : Recette client, tests système.

17.5 Approches itératives et agiles

DÉFINITION Développement itératif et incrémental

Au lieu de tout livrer à la fin (Big Bang), on découpe le projet en **itérations** (cycles courts, ex. 2 à 4 semaines). Chaque itération traverse toutes les étapes (Analyse → Conception → Code → Test) pour produire un **incrément** : une version partielle mais fonctionnelle, testée et potentiellement livrable du logiciel. Le produit grandit par ajouts successifs.

DÉFINITION Approche Agile (Manifeste Agile, 2001)

L'**approche agile** est un ensemble de méthodes (Scrum, XP, Kanban) fondées sur 4 valeurs : Individus et interactions > Processus et outils ; Logiciel opérationnel > Documentation exhaustive ; Collaboration client > Négociation contractuelle ; Adaptation au changement > Suivi d'un plan. Principes clés : livraisons fréquentes, accueil favorable au changement, équipe auto-organisée, rétrospection continue.

EXEMPLE Scrum : le cadre agile le plus utilisé

- Rôles : **Product Owner (PO)** (représente le client, gère le Product Backlog, priorise la valeur), **Scrum Master** (facilitateur, protège l'équipe, retire les obstacles), **Équipe de développement** (auto-organisée, 3–9 pers., pluridisciplinaire).
- Artefacts : Product Backlog (liste priorisée des **User Stories** = besoins utilisateur), Sprint Backlog (sélection pour le sprint en cours), Incrément (produit fini du sprint).
- Cérémonies (Événements) : **Sprint** (itération de durée fixe, 2–4 sem.), **Sprint Planning** (planification : quoi et comment), **Daily Scrum** (point quotidien 15 min : quoi fait ? quoi ferai ? blocages ?), **Sprint Review** (démonstration incrément + feedback PO/client), **Sprint Retrospective** (amélioration processus équipe).

PROPRIÉTÉ Comparatif synthétique : Cascade vs Agile

Critère	Cascade (Waterfall)	Agile (Scrum)
Besoins	Stables, connus, figés (CDC complet)	Évolutifs, découverts progressivement
Planification	Globale, détaillée au début	Adaptative, par Sprint (Rolling wave)
Livraison	Unique (Big Bang) à la fin	Fréquente (incrément chaque Sprint)
Client	Disponible au début (CDC) et à la fin (Recette)	Impliqué en continu (PO, Revues Sprint)
Risques	Découverts tard (coût élevé)	Identifiés et traités tôt
Documentation	Lourde, contractuelle	Légère, « juste assez », vivante
Projet type	Réglementaire, embarqué, migration connue	Web, Mobile, Startup, Innovation, UI/UX incertaine

17.6 Choisir et justifier une démarche de développement**MÉTHODE** Comment choisir son approche ?

1. Analyser la stabilité des besoins : Sont-ils figés par la loi/contrat (Cascade) ou sujets à évolution/conflit (Agile) ?
2. Évaluer la visibilité technique : Architecture connue (Cascade) ou exploration techno-

logique nécessaire (Agile)?

3. Contraintes contractuelles : Marché public à forfait (Cascade souvent imposé) vs Contrat d'objectif / Time & Material (Agile possible).
4. Relation client : Client distant/indisponible (Cascade) vs Client disponible/engagé (Agile).
5. Criticité / Sécurité : Vie humaine, finance critique → V / Cascade rigoureux. Innovation, Time-to-market → Agile.
6. Équipe : Grande équipe distribuée, juniors (Cadre structurant Cascade) vs Petite équipe collocalisée, seniors (Autonomie Agile).

À RETENIR Hybridation réelle

Dans la pratique professionnelle (au Cameroun comme ailleurs), on rencontre souvent des modèles hybrides : « Wagile » (Waterfall-Agile). Ex. : Cadrage contractuel en Cascade (CDC, Budget, Planning macro, Architecture) + Développement par Sprints Agiles pour les modules fonctionnels. L'essentiel est de justifier ses choix pédagogiques ou professionnels.

17.7 Je vérifie mes connaissances

EXERCICE 8 QCM : notions de base

Pour chaque question, une seule réponse est correcte. Recopie le numéro et la lettre choisie.

1. Le cycle de développement d'un logiciel correspond à :
 - A. La période pendant laquelle le logiciel est utilisé par le client.
 - B. L'ensemble des étapes depuis l'idée initiale jusqu'au retrait du logiciel.
 - C. Uniquement la phase d'écriture du code source.
 - D. La phase de test et de correction des bugs.
2. Dans le cycle en cascade (waterfall), les étapes :
 - A. Se chevauchent et peuvent être revisitées à tout moment.
 - B. S'enchaînent séquentiellement, chaque étape devant être validée avant de passer à la suivante.
 - C. Sont réalisées par itérations courtes de deux à quatre semaines.
 - D. Ne comprennent pas de phase de maintenance.

3. L'approche agile se caractérise principalement par :
 - A. Une planification complète et figée dès le début du projet.
 - B. L'absence de documentation technique.
 - C. Le développement itératif, incrémental et l'adaptation au changement.
 - D. Une livraison unique du produit final après plusieurs mois.
4. La recette client (ou validation) dans le cycle en V correspond à :
 - A. Les tests unitaires effectués par les développeurs.
 - B. La vérification que le logiciel répond aux besoins exprimés dans le cahier des charges.
 - C. L'installation du logiciel sur les serveurs de production.
 - D. La correction des anomalies détectées en phase de maintenance.

EXERCICE 9 Vrai ou Faux justifié

Indique si chaque affirmation est Vraie ou Fausse et justifie ta réponse en une phrase.

1. La phase de maintenance ne concerne que la correction des bugs découverts après la livraison.
2. Dans le cycle en V, la branche descendante correspond aux phases de validation et la branche montante aux phases de vérification.
3. L'approche en cascade est idéale pour un projet dont les besoins sont mal définis et susceptibles d'évoluer fréquemment.
4. Le cycle de vie d'un logiciel s'arrête au moment du déploiement chez le client.

EXERCICE 10 Définitions et vocabulaire

Donne une définition précise et concise pour chacun des termes suivants :

1. Cycle de vie d'un logiciel (ou cycle de développement).
2. Approche en cascade (Waterfall).
3. Approche agile (ex. Scrum).
4. Cahier des charges (ou spécifications fonctionnelles).

EXERCICE 11 Association : étapes et descriptions

Le tableau ci-dessous présente, dans le désordre, les six étapes classiques du cycle de vie d'un logiciel et leur description. Recopie le tableau en reliant chaque étape à sa description correcte (utilise des flèches ou numérote les descriptions de 1 à 6).

Étape	Description
1. Analyse des besoins	A. Écriture du code source dans le langage choisi.
2. Conception	B. Mise à disposition du logiciel pour les utilisateurs finaux.
3. Programmation (Implémentation)	C. Correction d'anomalies, améliorations, adaptation à un nouvel OS.
4. Tests (Validation/Vérification)	D. Étude de la faisabilité, rédaction du cahier des charges.
5. Déploiement (Installation)	E. Définition de l'architecture, choix techniques, algorithmes, MCD.
6. Maintenance	F. Exécution de scénarios de test, recette, correction des bugs.

17.8 Je m'entraîne**EXERCICE 12 Le cycle en V : complétion et explication**

On considère le schéma simplifié du cycle en V ci-dessous (représenté textuellement). Certaines étapes ont été effacées (marquées ???).

BRANCHE DESCENDANTE (Spécification/Conception)	BRANCHE MONTANTE (Validation/Vérification)
Spécifications utilisateur (Besoins)	??? (Recette client)
↓	↑
Spécifications système (Cahier des charges)	??? (Tests d'intégration)
↓	↑
Conception détaillée (Architecture, Algo)	??? (Tests unitaires)
↓	↑
Programmation (Codage)	(Point bas du V)

1. Complète le schéma en indiquant le nom des trois étapes manquantes côté montant (branche droite).

2. Explique la relation fondamentale qui lie chaque niveau de la branche descendante à son niveau symétrique sur la branche montante (ex. : Spécifications utilisateur ↔ Recette client).
3. Quel est le rôle précis de la recette client ? Qui la réalise généralement ?

EXERCICE 13 Identification d'étapes dans des situations concrètes

Pour chacune des situations suivantes, identifie l'étape du cycle de vie concernée (choisis parmi : Analyse, Conception, Programmation, Tests, Déploiement, Maintenance) et justifie brièvement.

1. L'équipe projet rencontre le directeur d'un collège pour lister les fonctionnalités attendues pour la gestion des notes (bulletins, moyennes, absences) et rédige un document validé par le directeur.
2. Un développeur écrit la fonction `calculerMoyenne(eleve)` en langage C en respectant l'algorithme défini lors de la conception.
3. L'équipe installe l'application sur le serveur de l'établissement, configure la base de données MySQL et forme les professeurs principaux à la saisie des notes.
4. Six mois après la mise en service, le ministère change le mode de calcul des moyennes (nouveaux coefficients). L'équipe modifie le code et redéploie une mise à jour.
5. Le chef de projet dessine le MCD (Modèle Conceptuel de Données) de la base (tables Eleve, Matiere, Note, Trimestre) et définit les interfaces utilisateur (maquettes des écrans).
6. Un testeur exécute un scénario : « Saisir une note de 25/20 pour un élève » et vérifie que le système affiche un message d'erreur « Note invalide ».

EXERCICE 14 Tableau comparatif : Cascade vs Agile

Complète le tableau comparatif suivant en indiquant pour chaque critère la caractéristique de l'approche en cascade et celle de l'approche agile.

Critère de comparaison	Approche en Cascade (Waterfall)	Approche Agile (ex. Scrum)
Souplesse face aux changements de besoins		
Implication du client pendant le projet		
Moment où le client voit un logiciel fonctionnel		
Gestion des risques (découverte tardive d'erreurs)		
Documentation produite		
Type de projet le plus adapté		

EXERCICE 15 Choix et justification d'une approche

Pour chacun des trois projets décrits ci-dessous, choisis l'approche de développement la plus adaptée (Cascade ou Agile) et justifie ton choix par deux arguments précis liés au contexte du projet.

1. **Projet A** : Logiciel de paie pour une administration camerounaise. Les règles de calcul (cotisations CNPS, impôt sur le revenu, grille indiciaire) sont fixées par la loi et le décret en vigueur. Elles changent très rarement (une fois par an maximum par loi de finances). Le cahier des charges est complet et stable dès le départ. La livraison est imposée pour le 1er janvier prochain.
2. **Projet B** : Application mobile de commande pour un restaurant « Fast-Food » à Douala. Le gérant veut tester rapidement un prototype avec ses clients. Les fonctionnalités (fidélité, paiement Mobile Money, livraison) ne sont pas toutes définies ; elles évolueront selon les retours utilisateurs. Le gérant souhaite une première version utilisable sous 6 semaines.
3. **Projet C** : Site web vitrine de l'école « Lycée Bilingue de Bafoussam ». Le contenu (texte, photos, histoire de l'école, contacts) est fourni par le proviseur et ne bougera pas pendant l'année. Le design est validé sur maquettes. Pas de fonctionnalité complexe (pas de connexion, pas de base de données dynamique). Livraison demandée pour la rentrée.

17.9 Je me prépare au Probatoire

EXERCICE 16 Cas pratique : Application de gestion des notes d'un collègue

Contexte : Le proviseur du Collège Polyvalent de Ngaoundéré souhaite informatiser la

gestion des notes pour remplacer les registres papier. Les besoins principaux sont : gestion des élèves (inscription, classes), des matières et coefficients (par série), saisie des notes par les professeurs (par séquence et par examen), calcul automatique des moyennes et rangs, édition des bulletins officiels (format MINESEC) et des procès-verbaux de délibération. Le proviseur a validé un cahier des charges détaillé et stable. L'équipe de développement (3 personnes) a 4 mois pour livrer l'application avant le premier trimestre. Le budget est figé.

Travail demandé :

1. Cite dans l'ordre les six étapes du cycle de vie classique d'un logiciel que l'équipe devra traverser.
2. Parmi les approches Cascade et Agile, laquelle recommandes-tu pour ce projet? Coche la case correspondante.
☐ Cascade ☐ Agile
3. Justifie ton choix en trois arguments tirés exclusivement du contexte ci-dessus (stabilité des besoins, contrainte de temps, nature du livrable, relation client).
4. Décris brièvement ce qui se passe concrètement lors de l'étape Tests (Recette) pour ce projet précis (donne 2 exemples de tests à effectuer).
5. L'application est déployée. Six mois plus tard, le MINESEC publie une circulaire modifiant le coefficient de l'EPS de 1 à 2 pour toutes les séries. Quelle phase du cycle de vie est concernée? Quel type de maintenance s'agit-il (corrective, adaptative, évolutive, préventive)? Justifie.

EXERCICE 17 Cas pratique : Évolution d'un logiciel de cybercafé

Contexte : « CyberNet » est un cybercafé à Yaoundé. Il y a deux ans, une équipe a développé en approche cascade un logiciel de gestion (contrôle du temps, facturation, impression). Le logiciel fonctionne bien mais l'interface est vieillotte. Le gérant veut aujourd'hui une version 2.0 avec : interface moderne (tableau de bord temps réel), gestion des abonnements mensuels, paiement par Mobile Money (MTN MoMo / Orange Money), statistiques de fréquentation par heure. Le gérant n'est pas sûr de l'ergonomie idéale pour le tableau de bord et veut pouvoir tester des maquettes. Il souhaite une première version utilisable (MVP) dans 8 semaines pour l'été.

Travail demandé :

1. Explique pourquoi l'approche cascade utilisée pour la V1 serait risquée pour cette V2.0 (2 arguments).
2. L'équipe décide d'adopter Scrum (approche agile) pour la V2.0.
 - a) Qu'appelle-t-on un Sprint? Quelle est sa durée typique?
 - b) Cite trois cérémonies (réunions) obligatoires dans Scrum et donne l'objectif de

chacune en une phrase.

c) Quel est le rôle du Product Owner dans ce projet précis (CyberNet V2.0) ?

3. Le gérant (Product Owner) a priorisé le Product Backlog suivant :

(a) Connexion / Déconnexion client + chronomètre.

(b) Intégration paiement Mobile Money.

(c) Tableau de bord gérant (stats temps réel).

(d) Gestion abonnements mensuels.

(e) Impression ticket de caisse.

L'équipe estime sa vélocité à 2 « User Stories » par sprint de 2 semaines. Prévois le contenu des deux premiers sprints (Sprint 1 et Sprint 2) en listant les User Stories sélectionnées. Justifie l'ordre choisi.

4. À la fin du Sprint 1, l'équipe présente l'incrément au gérant. Ce dernier n'est pas satisfait de l'affichage du chronomètre (trop petit, police illisible). Comment l'approche Agile permet-elle de traiter ce retour immédiatement ? Quel artefact Scrum est mis à jour ?

EXERCICE 18 Cas pratique : Système d'information « Bibliothèque Scolaire » et cycle en V

Contexte : Le lycée veut un logiciel pour gérer sa bibliothèque : catalogue des ouvrages (titre, auteur, ISBN, catégorie, nombre d'exemplaires), gestion des adhérents (élèves, professeurs), emprunts/retours avec alertes de retard, recherche multicritères, édition de l'inventaire annuel. Le projet suit un cycle en V.

Travail demandé :

1. Sur la branche descendante du V, on trouve : « Spécifications Utilisateur », « Spécifications Système », « Conception Détaillée ». Donne un exemple concret de livrable (document) produit à chacune de ces trois étapes pour le projet « Bibliothèque ».
2. Complète le tableau ci-dessous en associant chaque niveau de la branche descendante à l'activité de validation/vérification correspondante sur la branche montante, et donne un exemple de test pour la bibliothèque.

Branche Descendante (Spécification)	Branche Montante (Validation/Vérification)
Spécifications Utilisateur (Besoins)	???
Spécifications Système (Cahier des charges)	???
Conception Détaillée (Architecture, Algo, MCD)	???

3. Lors de la recette client, le bibliothécaire teste le scénario : « Emprunter un ouvrage dont tous les exemplaires sont déjà sortis ». Le système affiche « Stock épuisé » et bloque l'emprunt. Ce test valide-t-il une exigence fonctionnelle ou une exigence non-fonctionnelle ? Justifie.
4. Après un an d'utilisation, le disque dur du serveur tombe en panne. La base de données est perdue. Heureusement, une sauvegarde quotidienne existe. Le rétablissement du service relève de quelle phase du cycle de vie ? Cite deux mesures de prévention (maintenance préventive) qui auraient pu limiter l'impact de cette panne matérielle.

18 CORRIGÉS DES EXERCICES**CORRIGÉ Exercice 1 — QCM : notions de base**

1. Réponse B. Le cycle de développement couvre toutes les étapes, de l'idée initiale (expression du besoin) jusqu'au retrait du logiciel. La réponse A ne décrit que la période d'exploitation, la C que la programmation, la D que les tests : ce ne sont que des phases partielles.
2. Réponse B. Dans le modèle en cascade, les phases s'enchaînent de façon linéaire et séquentielle : chaque phase doit être totalement terminée et validée avant de passer à la suivante. La réponse A décrit un modèle souple, la C décrit les itérations agiles, la D est fausse car la maintenance existe bien.
3. Réponse C. L'approche agile repose sur le développement itératif et incrémental (cycles courts) et l'adaptation au changement. La A décrit le cascade ; la B est fausse (l'agilité préfère une documentation « juste assez », pas zéro documentation) ; la D décrit la livraison « Big Bang » du cascade.
4. Réponse B. La recette client est la validation : on vérifie que le logiciel répond aux besoins exprimés dans le cahier des charges (« avons-nous construit le bon produit ? »). La A correspond aux tests unitaires (vérification), la C au déploiement, la D à la maintenance corrective.

Point de vigilance : bien distinguer vérification (le produit est-il bien construit ?) et validation (est-ce le bon produit ?), confusion classique au BEPC.

CORRIGÉ Exercice 2 — Vrai ou Faux justifié

1. Faux. La maintenance comporte quatre types : corrective (corriger les bugs), mais aussi adaptative (adapter à un nouvel OS ou une nouvelle loi), évolutive (ajouter des fonctionnalités) et préventive (éviter de futures pannes). Elle ne se limite donc pas à la correction de bugs.
2. Faux. C'est l'inverse : la branche descendante correspond aux phases de spécification et conception (besoins, architecture, conception détaillée), tandis que la branche montante correspond aux phases de validation et vérification (tests unitaires, tests d'intégration, recette client).
3. Faux. Le cascade exige des besoins stables, clairs et complets dès le départ ; il est très rigide face aux changements. Pour des besoins mal définis et évolutifs, c'est l'approche agile qui est adaptée.
4. Faux. Le cycle de vie continue après le déploiement : il comprend l'exploitation et la

maintenance, et ne s'achève qu'au retrait définitif du logiciel (désinstallation, archivage).

CORRIGÉ Exercice 3 — Définitions et vocabulaire

1. Cycle de vie d'un logiciel : ensemble structuré des étapes traversées par un logiciel, depuis la naissance de l'idée (expression du besoin) jusqu'à son retrait définitif, en passant par la conception, la programmation, les tests, le déploiement et la maintenance.
2. Approche en cascade (Waterfall) : modèle de développement dans lequel les étapes s'enchaînent de manière linéaire et séquentielle ; chaque phase doit être totalement terminée et validée avant de passer à la suivante, le retour en arrière étant très coûteux.
3. Approche agile : ensemble de méthodes (ex. Scrum) fondées sur le développement itératif et incrémental : le projet est découpé en cycles courts (sprints) qui produisent chacun une version partielle mais fonctionnelle du logiciel, avec une forte collaboration du client et une grande capacité d'adaptation au changement.
4. Cahier des charges : document rédigé lors de l'analyse des besoins qui décrit de façon précise et complète les fonctionnalités attendues du logiciel, les contraintes (délais, budget, techniques) et qui est validé par le client ; il sert de référence pour la suite du projet.

Point de vigilance : une définition doit être précise et concise : une à deux phrases suffisent, avec les mots-clés soulignés dans le cours (séquentiel, itératif, incrémental, validé par le client...).

CORRIGÉ Exercice 4 — Association : étapes et descriptions

Les bonnes associations sont :

Étape	Description	Justification
1. Analyse des besoins	D	On étudie la faisabilité et on rédige le cahier des charges.
2. Conception	E	On définit l'architecture, les choix techniques, les algorithmes et le MCD.
3. Programmation	A	On écrit le code source dans le langage choisi (C, HTML/JS...).
4. Tests (V&V)	F	On exécute des scénarios de test, on fait la recette, on corrige les bugs.
5. Déploiement	B	On installe le logiciel et on le met à disposition des utilisateurs finaux.
6. Maintenance	C	On corrige les anomalies, on améliore, on adapte à un nouvel environnement.

Astuce : retiens l'ordre logique « Analyser → Concevoir → Programmer → Tester → Déployer → Maintenir » : chaque étape prépare la suivante.

CORRIGÉ Exercice 5 — Le cycle en V : complétion et explication

- Les trois étapes manquantes de la branche montante (de haut en bas) sont :
 - Spécifications utilisateur ↔ Recette client (tests de validation / d'acceptation) ;
 - Spécifications système ↔ Tests d'intégration (et de qualification système) ;
 - Conception détaillée ↔ Tests unitaires.
- Relation fondamentale : chaque niveau de la branche descendante (spécification) prépare et correspond au niveau symétrique de la branche montante (test). Le test en face vérifie que ce qui a été spécifié à ce niveau est bien réalisé :
 - Spécifications utilisateur ↔ Recette client : on valide que le logiciel répond aux besoins exprimés (« avons-nous construit le bon produit ? ») ;
 - Spécifications système ↔ Tests d'intégration : on vérifie que les modules interagissent correctement conformément à l'architecture prévue ;
 - Conception détaillée ↔ Tests unitaires : on vérifie que chaque module ou fonction respecte son algorithme (« le produit est-il bien construit ? »).
- Rôle de la recette client : c'est la phase de validation finale où l'on s'assure que le logiciel livré est conforme au cahier des charges et acceptable par l'utilisateur. Elle est réalisée par le client (ou ses représentants, ex. le proviseur et les professeurs principaux), souvent assisté de l'équipe de développement, avant l'acceptation définitive du produit.

CORRIGÉ Exercice 6 — Identification d'étapes dans des situations concrètes

1. Analyse des besoins. On rencontre le client (le directeur), on recueille les fonctionnalités attendues et on rédige un document de référence (cahier des charges) validé par lui.
2. Programmation (Implémentation). Le développeur écrit le code source de la fonction `calculerMoyenne(eleve)` en langage C, à partir de l'algorithme conçu précédemment.
3. Déploiement. On installe l'application sur le serveur de l'établissement, on configure la base de données et on forme les utilisateurs (professeurs principaux).
4. Maintenance. Le logiciel est déjà en service ; on l'adapte à une nouvelle règle (changement des coefficients) : il s'agit plus précisément d'une maintenance adaptative.
5. Conception. On modélise les données (MCD : Eleve, Matiere, Note, Trimestre) et on définit l'architecture et les maquettes d'écrans : c'est la traduction du « quoi » en « comment ».
6. Tests (Vérification). Le testeur exécute un scénario précis (note de 25/20) et vérifie que le système réagit correctement (message « Note invalide ») : c'est un test de conformité au comportement prévu.

Point de vigilance : pour identifier l'étape, repère les mots déclencheurs : « rencontre du client / cahier des charges » = Analyse ; « MCD / maquettes » = Conception ; « écrit le code » = Programmation ; « scénario de test » = Tests ; « installe / forme » = Déploiement ; « après mise en service, on modifie » = Maintenance.

CORRIGÉ Exercice 7 — Tableau comparatif : Cascade vs Agile

Critère	Cascade (Waterfall)	Agile (Scrum)
Souplesse face aux changements	Très faible : besoins figés dans le cahier des charges, tout changement est coûteux.	Forte : le changement est accueilli favorablement à chaque sprint.
Implication du client	Au début (cahier des charges) et à la fin (recette) seulement.	En continu : le client (Product Owner) participe à chaque sprint.
Moment où le client voit un logiciel fonctionnel	À la fin du projet (livraison unique « Big Bang »).	Très tôt : un incrément fonctionnel à la fin de chaque sprint (2 à 4 semaines).
Gestion des risques	Erreurs découvertes tardivement (phase de tests), coût de correction élevé.	Risques identifiés et traités tôt, grâce aux livraisons et retours fréquents.
Documentation	Lourde, exhaustive et contractuelle (CDC, dossier de conception).	Légère, « juste assez », évolutive.
Type de projet adapté	Besoins stables et connus : logiciel de paie réglementaire, système critique.	Besoins évolutifs ou incertains : applications web/mobile, innovation, prototypes.

CORRIGÉ**Exercice 8 — Choix et justification d'une approche**

1. Projet A (paie administration) : approche en CASCADE.

- Argument 1 : les besoins sont stables et figés par la loi (règles CNPS, impôt, grille indiciaire) ; le cahier des charges est complet dès le départ : condition idéale du cascade.
 - Argument 2 : la date de livraison est imposée (1er janvier) et le contexte est contractuel (administration) : le cascade offre une planification globale et une documentation exhaustive adaptée à un marché à délai fixe.
2. Projet B (application Fast-Food) : approche AGILE.
- Argument 1 : les besoins sont mal définis et évolutifs (fidélité, Mobile Money, livraison évolueront selon les retours des clients) : l'agilité accueille le changement.
 - Argument 2 : le gérant veut une première version utilisable en 6 semaines et tester un prototype : les livraisons fréquentes (incréments à chaque sprint) répondent exactement à ce besoin.
3. Projet C (site vitrine du lycée) : approche en CASCADE.
- Argument 1 : le contenu est fourni, figé et validé (textes, photos, maquettes) et n'évoluera pas : les besoins sont parfaitement stables.
 - Argument 2 : le projet est simple et sans fonctionnalité complexe (pas de base de données dynamique), avec une échéance fixe (la rentrée) : un enchaînement linéaire Analyse → Conception → Réalisation → Tests → Mise en ligne suffit, sans la lourdeur des cérémonies agiles.

Point de vigilance : la justification doit toujours s'appuyer sur des éléments du contexte (stabilité des besoins, délai, disponibilité du client), jamais sur une préférence gratuite.

CORRIGÉ Exercice 9 — Cas pratique : gestion des notes (type BEPC)

Barème indicatif (sur 10 pts) : Q1 : 2 pts ; Q2 : 1 pt ; Q3 : 3 pts (1 pt par argument) ; Q4 : 2 pts ; Q5 : 2 pts.

1. Les six étapes du cycle de vie classique, dans l'ordre :
- (a) Analyse des besoins (cahier des charges) ;
 - (b) Conception (architecture, MCD, algorithmes, maquettes) ;
 - (c) Programmation (écriture du code source) ;
 - (d) Tests (vérification et validation / recette) ;
 - (e) Déploiement (installation, formation des utilisateurs) ;
 - (f) Maintenance (corrections, adaptations, évolutions).
2. Approche recommandée : ☒ Cascade ☐ Agile
3. Justification (trois arguments tirés du contexte) :

- Besoins stables : le proviseur a validé un cahier des charges détaillé et stable ; les règles (format des bulletins MINESEC, calcul des moyennes) sont connues et figées.
 - Contrainte de temps et de budget figés : l'équipe dispose de 4 mois et d'un budget fixe ; le cascade permet une planification globale et prévisible dès le départ.
 - Livrable unique et contractualisé : le client (le proviseur) a exprimé des besoins clairs et n'a pas demandé à tester des prototypes ; une livraison unique après recette convient.
4. Lors de la recette (tests), le proviseur et les professeurs vérifient que l'application répond au cahier des charges. Exemples de tests :
- Saisir les notes d'une classe, lancer le calcul et vérifier que la moyenne pondérée d'un élève est correcte (contrôle manuel d'un cas) ;
 - Éditer un bulletin et vérifier qu'il est conforme au format officiel MINESEC (en-tête, matières, coefficients, rang, mention) ;
 - (autre exemple accepté) saisir une note supérieure à 20 et vérifier que le système la refuse.
5. La phase concernée est la maintenance. Il s'agit d'une maintenance adaptative : le logiciel, qui fonctionnait correctement, doit être adapté à un changement de son environnement (la nouvelle circulaire MINESEC modifiant le coefficient de l'EPS). Ce n'est ni un bug (corrective), ni une nouvelle fonctionnalité (évolutive), ni une prévention (préventive).

Points de vigilance : en Q3, n'accepte que des arguments présents dans l'énoncé ; en Q5, exiger la justification « changement de règle extérieure ⇒ adaptative ».

CORRIGÉ Exercice 10 — Cas pratique : évolution du logiciel de cybercafé

1. Le cascade serait risqué pour la V2.0 car :
- Besoins incertains : le gérant n'est pas sûr de l'ergonomie du tableau de bord et veut tester des maquettes ; or le cascade fige tout dans le cahier des charges initial et rend tout changement très coûteux.
 - Risque découvert tard : avec le cascade, le gérant ne verrait le logiciel qu'à la fin ; si l'interface ne lui plaît pas, il faudrait tout reprendre. De plus, l'exigence d'une version utilisable en 8 semaines (MVP) est incompatible avec une livraison unique en fin de projet.
2. Adoption de Scrum :
- a) Un Sprint est une itération : un cycle de développement court et de durée fixe, au bout duquel l'équipe livre un incrément (une version partielle mais fonctionnelle et

testée du logiciel). Durée typique : 2 à 4 semaines.

b) Trois cérémonies Scrum :

- Sprint Planning : réunion de planification où l'équipe choisit les User Stories du sprint et définit comment les réaliser ;
- Daily Scrum : point quotidien de 15 minutes où chaque membre dit ce qu'il a fait, ce qu'il fera et ses difficultés ;
- Sprint Review : démonstration de l'incrément au client (Product Owner) pour recueillir son retour ; (on accepte aussi la Sprint Retrospective : l'équipe analyse son fonctionnement pour s'améliorer).

c) Le Product Owner (ici le gérant de CyberNet, ou son représentant) représente le client : il définit et priorise le Product Backlog (la liste des fonctionnalités), tranche sur ce qui apporte le plus de valeur (ex. le paiement Mobile Money avant les statistiques) et valide les incréments à chaque Sprint Review.

3. Planification des deux premiers sprints (vélocité : 2 User Stories par sprint) :

- Sprint 1 : User Story 1 (Connexion/Déconnexion + chronomètre) et User Story 5 (Impression ticket de caisse).
- Sprint 2 : User Story 2 (Paiement Mobile Money) et User Story 3 (Tableau de bord gérant).

Justification : la connexion/chronomètre est le cœur du métier du cybercafé (sans elle, rien ne fonctionne) et l'impression du ticket complète le cycle de facturation de base : on obtient dès le Sprint 1 un produit minimal utilisable (MVP). Le paiement Mobile Money et le tableau de bord, prioritaires pour le gérant, sont traités ensuite ; les abonnements (moins urgents) viendront au Sprint 3. (Toute planification respectant la priorité métier et la vélocité de 2 US/sprint est acceptable si elle est justifiée.)

4. Grâce à l'agilité, le retour du gérant est traité immédiatement : la correction « agrandir l'affichage du chronomètre » est ajoutée et priorisée dans le Product Backlog (artefact mis à jour par le Product Owner), puis planifiée dès le Sprint 2 lors du Sprint Planning. On n'attend pas la fin du projet : le changement est accueilli favorablement, contrairement au cascade.

CORRIGÉ Exercice 11 — Cas pratique : bibliothèque scolaire et cycle en V

1. Exemples de livrables de la branche descendante :

- Spécifications Utilisateur : le cahier des charges fonctionnel validé par le bibliothécaire (ex. « le système doit gérer les emprunts, les retours et alerter en cas de retard »).
- Spécifications Système : le dossier d'architecture : choix techniques (application

de bureau ou web, SGBD MySQL), description des modules (catalogue, adhérents, prêts) et de leurs interfaces.

- Conception Détaillée : le MCD/MLD de la base (tables *Ouvrage*, *Adherent*, *Emprunt*), les algorithmes (ex. calcul de la date de retour, détection des retards) et les maquettes d'écrans.

2. Tableau complété :

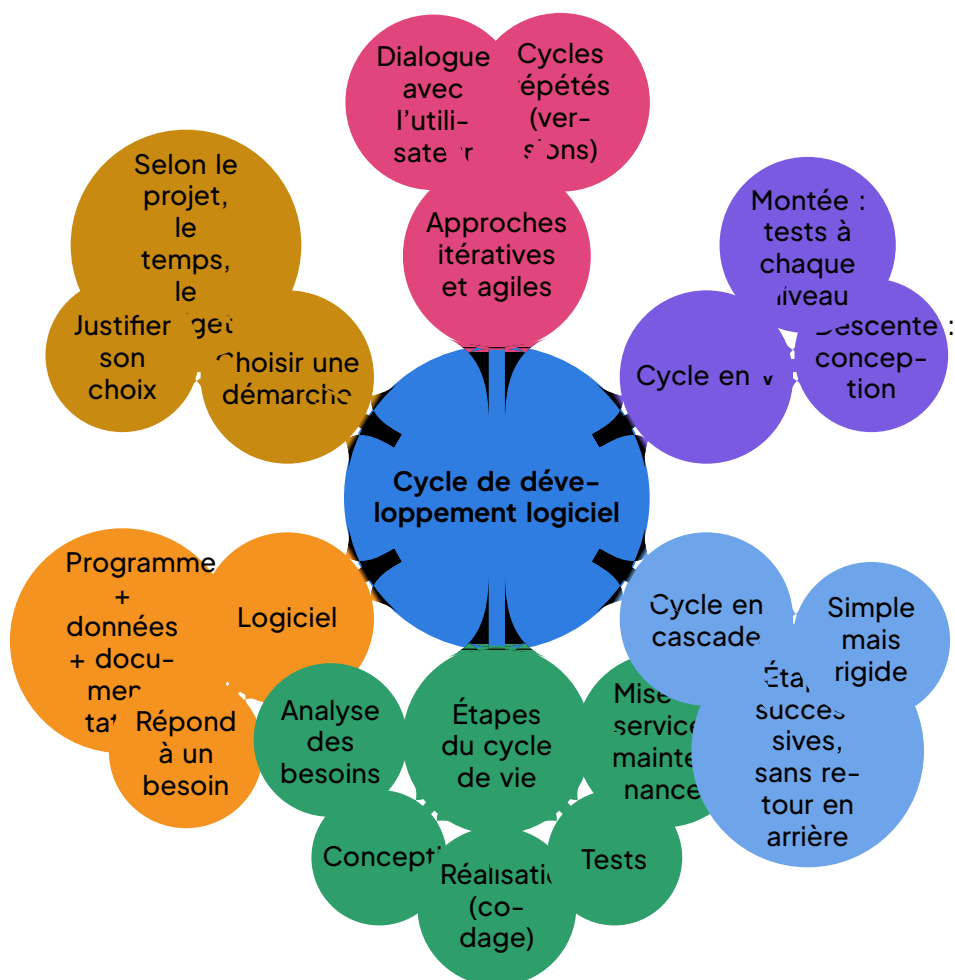
Branche descendante	Branche montante	Exemple de test « Bibliothèque »
Spécifications Utilisateur	Recette client (validation)	Le bibliothécaire vérifie qu'il peut enregistrer un emprunt et éditer l'inventaire annuel conformément au cahier des charges.
Spécifications Système	Tests d'intégration / qualification	Vérifier que le module « Emprunt » met bien à jour le stock dans la base et déclenche l'alerte de retard (modules qui communiquent).
Conception Détaillée	Tests unitaires	Tester seule la fonction <code>calculerDateRetour()</code> : pour un emprunt le 02/03, elle doit renvoyer le 16/03 (durée de 14 jours).

3. Ce test valide une exigence fonctionnelle : il vérifie un comportement attendu du logiciel (un service rendu : bloquer l'emprunt quand le stock est épuisé), et non une qualité comme la rapidité, la sécurité ou l'ergonomie (qui seraient des exigences non-fonctionnelles).
4. Le rétablissement du service après la panne relève de la phase de maintenance (ici maintenance corrective : réparer et restaurer le service). Deux mesures de maintenance préventive auraient limité l'impact :

- Mettre en place des sauvegardes automatiques externalisées (copie quotidienne sur un autre serveur ou dans le cloud, et non sur le même disque);
- Utiliser un disque dur redondant (système RAID) ou un onduleur, et surveiller l'état du matériel pour remplacer un disque défaillant avant la panne.

Point de vigilance : dans le cycle en V, chaque test de la branche montante doit correspondre exactement au niveau de spécification symétrique : recette ↔ besoins, intégration ↔ architecture, tests unitaires ↔ conception détaillée.

19 FICHE BILAN



Carte mentale de la leçon : les approches de développement logiciel.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- Un **logiciel** est un ensemble de programmes, de données et de documents permettant à un ordinateur d'accomplir des tâches (ex. : traitement de texte, logiciel de gestion des notes d'un collègue).
- Le **cycle de développement** (ou cycle de vie) d'un logiciel est la suite des étapes qui vont de l'idée du logiciel jusqu'à son utilisation et sa maintenance.
- Une **approche de développement** est une manière d'organiser ces étapes : en cascade, en V, itérative ou agile.

Les étapes du cycle de vie (à connaître par cœur, dans l'ordre)

N°	Étape	Ce qu'on y fait
1	Analyse des besoins	Comprendre ce que veut l'utilisateur ; rédiger le cahier des charges.
2	Conception	Préparer la solution : organisation du logiciel, écrans, algorithmes.
3	Réalisation (codage)	Écrire les programmes dans un langage (C, JavaScript...).
4	Tests	Vérifier que le logiciel fonctionne et corriger les erreurs.
5	Mise en service et maintenance	Installer le logiciel chez l'utilisateur, puis le corriger et l'améliorer.

Les grandes approches comparées

Approche	Principe	Limite / avantage
Cascade	Les étapes se suivent dans l'ordre, sans retour en arrière.	Simple à suivre, mais une erreur découverte tard coûte cher.
Cycle en V	À chaque étape de conception correspond une étape de tests.	Tests bien préparés, mais peu souple.
Itérative / agile	On développe par petits cycles répétés, avec des versions successives et l'avis de l'utilisateur.	Souple et adaptée aux besoins qui changent ; demande une bonne organisation.

Méthode express : choisir et justifier une démarche

1. Identifier le projet : taille, besoins clairs ou évolutifs, délais, budget.
2. Besoins stables et bien définis → cascade ou cycle en V.
3. Besoins évolutifs, utilisateur disponible → approche itérative ou agile.
4. Rédiger la justification en citant le principe de l'approche choisie et son avantage pour la situation.

À RETENIR

Checklist avant le BEPC

- ☐ Je sais définir un logiciel et donner deux exemples de la vie courante.
- ☐ Je sais expliquer ce qu'est le cycle de développement d'un logiciel.
- ☐ Je connais les 5 étapes du cycle de vie, dans l'ordre.
- ☐ Je sais dire ce qu'on fait à chaque étape (analyse, conception, codage, tests, main-)

tenance).

- ☐ Je sais décrire le principe du cycle en cascade.
- ☐ Je sais expliquer pourquoi le cycle en cascade est dit « rigide ».
- ☐ Je sais décrire le cycle en V et le rôle des tests dans ce modèle.
- ☐ Je sais expliquer le principe des approches itératives et agiles.
- ☐ Je sais comparer deux approches et citer un avantage et une limite de chacune.
- ☐ Je sais choisir une démarche adaptée à une situation concrète (ex. : logiciel de gestion d'une bibliothèque scolaire) et justifier mon choix par écrit.



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+